

Chapter 4. Threads

This chapter explains the data structures and algorithms that deal with threads and thread scheduling in Windows. The first section shows how to create threads. Then the internals of threads and thread scheduling are described. The chapter concludes with a discussion of thread pools.

Creating threads

Before discussing the internal structures used to manage threads, let's take a look at creating threads from an API perspective to give a sense of the steps and arguments involved.

The simplest creation function in user mode is `CreateThread`. This function creates a thread in the current process, accepting the following arguments:

- **An optional security attributes structure** This specifies the security descriptor to attach to the newly created thread. It also specifies whether the thread handle is to be created as inheritable. (Handle inheritance is discussed in Chapter 8, “System mechanisms,” in *Windows Internals Part 2*.)
- **An optional stack size** If zero is specified, a default is taken from the executable's header. This always applies to the first thread in a user-mode process. (Thread's stack is discussed further in [Chapter 5](#), “[Memory management](#).”)
- **A function pointer** This serves as the entry point for the new thread's execution.
- **An optional argument** This is to pass to the thread's function.
- **Optional flags** One controls whether the thread starts suspended (`CREATE_SUSPENDED`). The other controls the interpretation of the stack size argument (initial committed size or maximum reserved size).

On successful completion, a non-zero handle is returned for the new thread and, if requested by the caller, the unique thread ID.

An extended thread creation function is `CreateRemoteThread`. This function accepts an extra argument (the first), which is a handle to a target process where the thread is to be created. You can use this function to inject a thread into another process. One common use of this technique is for a debugger to force a break in a debugged process. The debugger injects the thread, which immediately causes a breakpoint by calling the `DebugBreak` function. Another common use of this technique is for one process to obtain internal information about another process, which is easier when running within the target process context (for example, the entire address space is visible). This could be done for legitimate or malicious purposes.

To make `CreateRemoteThread` work, the process handle must have been obtained with enough access rights to allow such operation. As an extreme example, protected processes cannot be injected in this way because handles to such processes can be obtained with very limited rights only.

The final function worth mentioning here is `CreateRemoteThreadEx`, which is a superset of `CreateThread` and `CreateRemoteThread`. In fact, the implementation of `CreateThread` and `CreateRemoteThread` simply calls `CreateRemoteThreadEx` with the appropriate defaults. `CreateRemoteThreadEx` adds the ability to provide an attribute list (similar to the `STARTUPINFOEX` structure's role with an additional member over `STARTUPINFO` when creating processes). Examples of attributes include setting the ideal processor and group affinity (both discussed later in this chapter).

If all goes well, `CreateRemoteThreadEx` eventually calls `NtCreateThreadEx` in `Ntdll.dll`. This makes the usual transition to kernel mode, where execution continues in the executive function `NtCreateThreadEx`. There, the kernel mode part of thread creation occurs (described later in this chapter, in the “[Birth of a thread](#)” section).

Creating a thread in kernel mode is achieved with the `PsCreateSystemThread` function (documented in the WDK). This is useful for drivers that need independent work to be processes within the system process (meaning it’s not associated with any particular process). Technically, the function can be used to create a thread under any process, which is less useful for drivers.

Exiting a kernel thread’s function does not automatically destroy the thread object. Instead, drivers must call `PsTerminateSystemThread` from within the thread function to properly terminate the thread. Consequently, this function never returns.

Thread internals

This section discusses the internal structures used within the kernel (and some in user mode) to manage a thread. Unless explicitly stated otherwise, you can assume that anything in this section applies to both user-mode threads and kernel-mode system threads.

Data structures

At the operating-system (OS) level, a Windows thread is represented by an executive thread object. The executive thread object encapsulates an `ETHREAD` structure, which in turn contains a `KTHREAD` structure as its first member. These are illustrated in [Figure 4-1](#) (`ETHREAD`) and [Figure 4-2](#) (`KTHREAD`). The `ETHREAD` structure and the other structures it points to exist in the system address space. The only exception is the thread environment block (TEB), which exists in the process address space (similar to a PEB, because user-mode components need to access it).

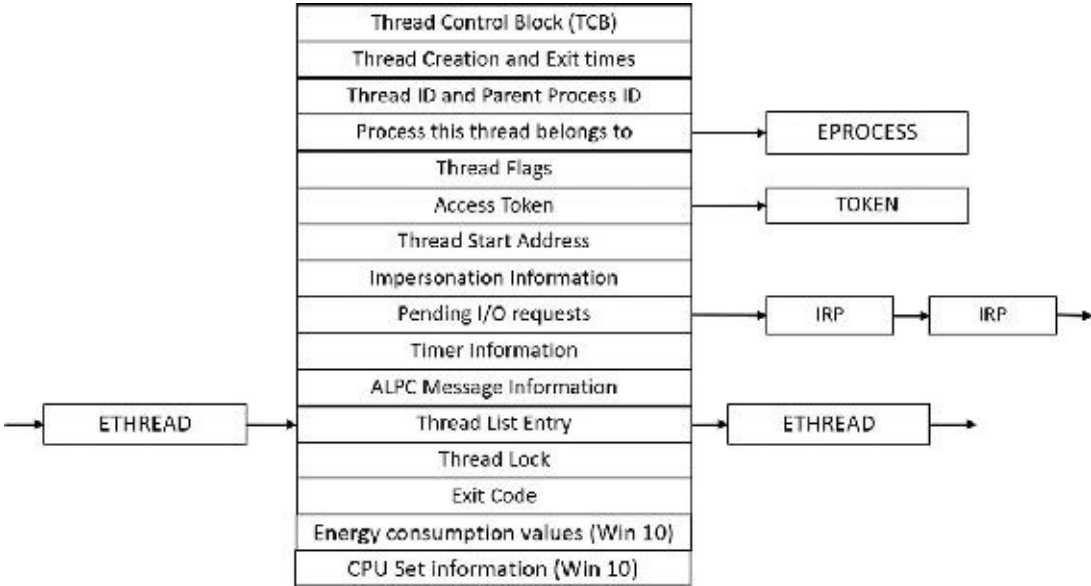


FIGURE 4-1 Important fields of the executive thread structure (`ETHREAD`).

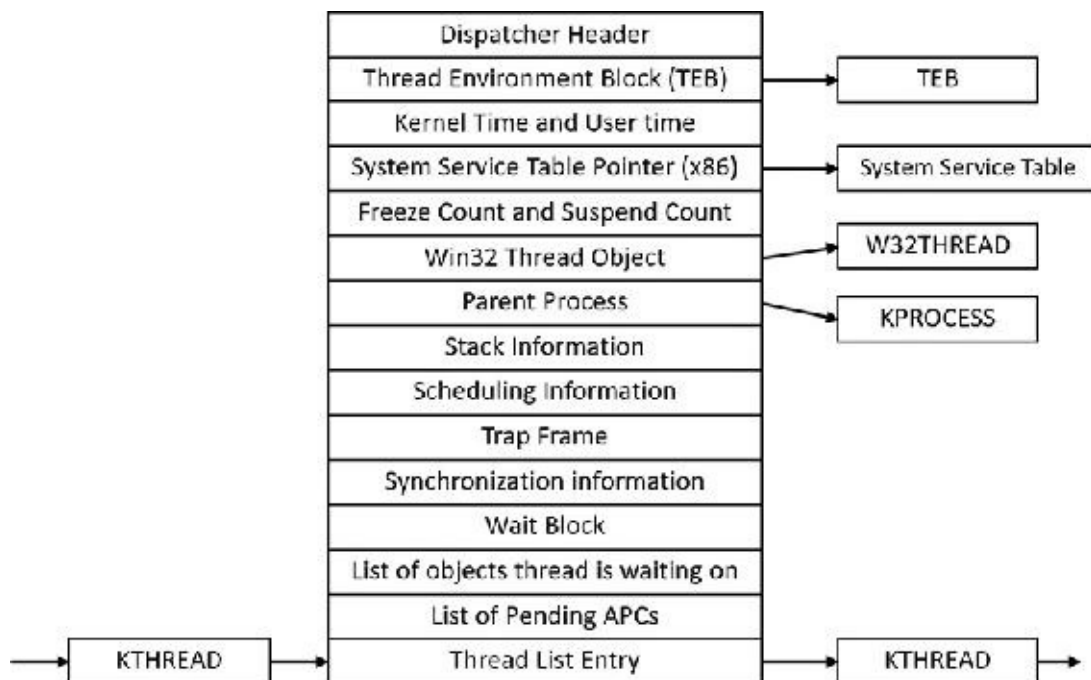


FIGURE 4-2 Important fields of the kernel thread structure (KTHREAD).

The Windows subsystem process (Csrss) maintains a parallel structure for each thread created in a Windows subsystem application, called the `CSR_THREAD`. For threads that have called a Windows subsystem `USER` or `GDI` function, the kernel-mode portion of the Windows subsystem (Win32k.sys) maintains a per-thread data structure (`W32THREAD`) that the `KTHREAD` structure points to.



Note

The fact that the executive, high-level, graphics-related, Win32k thread structure is pointed to by `KTHREAD` instead of the `ETHREAD` appears to be a layer violation or oversight in the standard kernel's abstraction architecture. The scheduler and other low-level components do not use this field.

Most of the fields illustrated in [Figure 4-1](#) are self-explanatory. The first member of the `ETHREAD` is called `Tcb`. This is short for *thread control block*, which is a structure of type `KTHREAD`. Following that are the thread identification information, the process identification information (including a pointer to the owning process so that its environment information can be accessed), security information in the form of a pointer to the access token and impersonation information, fields relating to Asynchronous Local Procedure Call (ALPC) messages, pending I/O requests (IRPs) and Windows 10-specific fields related to power management (described in [Chapter 6](#), “[I/O system](#)”) and CPU Sets (described later in this chapter). Some of these key fields are covered in more detail elsewhere in this book. For more details on the internal structure of an `ETHREAD` structure, you can use the kernel debugger `dt` command to display its format.

Let's take a closer look at two of the key thread data structures referred to in the preceding text: `ETHREAD` and `KTHREAD`. The `KTHREAD` structure (which is the `Tcb` member of the `ETHREAD`) contains information that the Windows kernel needs to perform thread scheduling, synchronization, and timekeeping functions.

EXPERIMENT: Displaying ETHREAD and KTHREAD structures

You can display the ETHREAD and KTHREAD structures with the `dt` command in the kernel debugger. The following output shows the format of an ETHREAD on a 64-bit Windows 10 system:

[Click here to view code image](#)

```
lkd> dt nt!_ethread
+0x000 Tcb                : _KTHREAD
+0x5d8 CreateTime         : _LARGE_INTEGER
+0x5e0 ExitTime           : _LARGE_INTEGER
...
+0x7a0 EnergyValues       : Ptr64 _THREAD_ENERGY_VALUES
+0x7a8 CmCellReferences   : Uint4B
+0x7b0 SelectedCpuSets    : Uint8B
+0x7b0 SelectedCpuSetsIndirect : Ptr64 Uint8B
+0x7b8 Silo               : Ptr64 _EJOB
```

You can display the KTHREAD with a similar command or by typing `dt nt!_ETHREAD Tcb`, as shown in the experiment “Displaying the format of an EPROCESS structure” in [Chapter 3](#), “[Processes and jobs](#).”

[Click here to view code image](#)

```
lkd> dt nt!_kthread
+0x000 Header             : _DISPATCHER_HEADER
+0x018 SListFaultAddress  : Ptr64 Void
+0x020 QuantumTarget      : Uint8B
+0x028 InitialStack       : Ptr64 Void
+0x030 StackLimit         : Ptr64 Void
+0x038 StackBase          : Ptr64 Void
+0x040 ThreadLock         : Uint8B
+0x048 CycleTime          : Uint8B
+0x050 CurrentRunTime     : Uint4B
...
+0x5a0 ReadOperationCount : Int8B
+0x5a8 WriteOperationCount : Int8B
+0x5b0 OtherOperationCount : Int8B
+0x5b8 ReadTransferCount  : Int8B
+0x5c0 WriteTransferCount  : Int8B
+0x5c8 OtherTransferCount  : Int8B
+0x5d0 QueuedScb          : Ptr64 _KSCB
```

EXPERIMENT: Using the kernel debugger !thread command

The kernel debugger `!thread` command dumps a subset of the information in the thread data structures. Some key elements of the information the kernel debugger displays can’t be displayed by any utility, including the following information:

- Internal structure addresses

- Priority details
- Stack information
- The pending I/O request list
- For threads in a wait state, the list of objects the thread is waiting for

To display thread information, use either the `!process` command (which displays all the threads of a process after displaying the process information) or the `!thread` command with the address of a thread object to display a specific thread.

Let's find all instances of explorer.exe:

[Click here to view code image](#)

```
lkd> !process 0 0 explorer.exe
PROCESS fffffe00017f3e7c0
    SessionId: 1  Cid: 0b7c    Peb: 00291000  ParentCid: 0c34
    DirBase: 19b264000  ObjectTable: fffffc00007268cc0  HandleCount:
2248.
    Image: explorer.exe

PROCESS fffffe00018c817c0
    SessionId: 1  Cid: 23b0    Peb: 00256000  ParentCid: 03f0
    DirBase: 2d4010000  ObjectTable: fffffc0001aef0480  HandleCount:
2208.
    Image: explorer.exe
```

We'll select one of the instances and show its threads:

[Click here to view code image](#)

```
lkd> !process fffffe00018c817c0 2
PROCESS fffffe00018c817c0
    SessionId: 1  Cid: 23b0    Peb: 00256000  ParentCid: 03f0
    DirBase: 2d4010000  ObjectTable: fffffc0001aef0480  HandleCount:
2232.
    Image: explorer.exe

        THREAD fffffe0001ac3c080  Cid 23b0.2b88  Teb:
00000000000257000 Win32Thread:
fffffe0001570ca20 WAIT: (UserRequest) UserMode Non-Alertable
        fffffe0001b6eb470  SynchronizationEvent

        THREAD fffffe0001af10800  Cid 23b0.2f40  Teb:
00000000000265000 Win32Thread:
fffffe000156688a0 WAIT: (UserRequest) UserMode Non-Alertable
        fffffe000172ad4f0  SynchronizationEvent
        fffffe0001ac26420  SynchronizationEvent

        THREAD fffffe0001b69a080  Cid 23b0.2f4c  Teb:
00000000000267000 Win32Thread:
fffffe000192c5350 WAIT: (UserRequest) UserMode Non-Alertable
```

```
fffffe00018d83c00 SynchronizationEvent
fffffe0001552ff40 SynchronizationEvent
```

...

```
THREAD fffffe00023422080 Cid 23b0.3d8c Teb:
000000000003cf000 Win32Thread:
fffffe0001eccd790 WAIT: (WrQueue) UserMode Alertable
fffffe0001aec9080 QueueObject
```

```
THREAD fffffe00023f23080 Cid 23b0.3af8 Teb:
000000000003d1000 Win32Thread:
00000000000000000 WAIT: (WrQueue) UserMode Alertable
fffffe0001aec9080 QueueObject
```

```
THREAD fffffe000230bf800 Cid 23b0.2d6c Teb:
000000000003d3000 Win32Thread:
00000000000000000 WAIT: (WrQueue) UserMode Alertable
fffffe0001aec9080 QueueObject
```

```
THREAD fffffe0001f0b5800 Cid 23b0.3398 Teb:
000000000003e3000 Win32Thread:
00000000000000000 WAIT: (UserRequest) UserMode Alertable
fffffe0001d19d790 SynchronizationEvent
fffffe00022b42660 SynchronizationTimer
```

The list of threads is truncated for the sake of space. Each thread shows its address (ETHREAD), which can be passed to the `!thread` command; its client ID (Cid)—process ID and thread ID (the process ID for all the preceding threads is the same, as they are part of the same explorer.exe process); the Thread Environment Block (TEB, discussed momentarily); and the thread state (most should be in the `Wait` state, with the reason for the wait in parentheses). The next line may show a list of synchronization objects the threads is waiting on.

To get more information on a specific thread, pass its address to the `!thread` command:

[Click here to view code image](#)

```
lkd> !thread fffffe0001d45d800
THREAD fffffe0001d45d800 Cid 23b0.452c Teb: 0000000000026d000
Win32Thread:
fffffe0001aace630 WAIT: (UserRequest) UserMode Non-Alertable
fffffe00023678350 NotificationEvent
fffffe00022aeb370 Semaphore Limit 0xfffff
fffffe000225645b0 SynchronizationEvent
Not impersonating
DeviceMap fffffc00004f7ddb0
Owning
Process fffffe00018c817c0 Image: explorer.exe
Attached Process N/A Image: N/A
```

```

Wait Start TickCount          7233205          Ticks: 270 (0:00:00:04.218)
Context Switch Count          6570             IdealProcessor: 7
UserTime                      00:00:00.078
KernelTime                    00:00:00.046
Win32 Start Address 0c
Stack Init fffffd000271d4c90 Current fffffd000271d3f80
Base fffffd000271d5000 Limit fffffd000271cf000 Call 0000000000000000
Priority 9 BasePriority 8 PriorityDecrement 0 IoPriority 2
PagePriority 5
GetContextState failed, 0x80004001
Unable to get current machine context, HRESULT 0x80004001
Child-SP          RetAddr          : Args to
Child              : Call Site
fffffd000'271d3fc0 ffffff803'bef086ca : 00000000'00000000
00000000'00000001
00000000'00000000 00000000'00000000 : nt!KiSwapContext+0x76
fffffd000'271d4100 ffffff803'bef08159 : fffffe000'1d45d800
fffff803'00000000
fffffe000'1aec9080 00000000'0000000f : nt!KiSwapThread+0x15a
fffffd000'271d41b0 ffffff803'bef09cfe : 00000000'00000000
00000000'00000000
fffffe000'0000000f 00000000'00000003 : nt!KiCommitThreadWait+0x149
fffffd000'271d4240 ffffff803'bf2a445d : fffffd000'00000003
fffffd000'271d43c0
00000000'00000000 ffffff960'00000006 :
nt!KewaitForMultipleObjects+0x24e
fffffd000'271d4300 ffffff803'bf2fa246 : ffffff803'bf1a6b40
fffffd000'271d4810
fffffd000'271d4858 fffffe000'20aeca60 :
nt!ObwaitForMultipleObjects+0x2bd
fffffd000'271d4810 ffffff803'befdefa3 : 00000000'00000fa0
fffff803'bef02aad
fffffe000'1d45d800 00000000'1e22f198 :
nt!NtwaitForMultipleObjects+0xf6
fffffd000'271d4a90 00007ffe'f42b5c24 : 00000000'00000000
00000000'00000000
00000000'00000000 00000000'00000000 : nt!KiSystemServiceCopyEnd+0x13
(TrapFrame @
fffffd000'271d4b00)
00000000'1e22f178 00000000'00000000 : 00000000'00000000
00000000'00000000
00000000'00000000 00000000'00000000 : 0x00007ffe'f42b5c24

```

There is a lot of information about the thread, such as its priority, stack details, user and kernel times, and much more. You'll look at many of these details throughout this chapter and in [Chapter 5](#), “[Memory management](#),” and [Chapter 6](#), “[I/O system](#).”

EXPERIMENT: Viewing thread information with tlist

The following output is the detailed display of a process produced by using `tlist` in the Debugging Tools for Windows. (Make sure you run `tlist` from the same “bitness” as the target process.)

Notice that the thread list shows `Win32StartAddr`. This is the address passed to the `CreateThread` function by the application. All the other utilities (except Process Explorer) that show the thread start address show the actual start address (a function in `Ntdll.dll`), not the application-specified start address.

The following output is from running `tlist` on Word 2016 (truncated):

[Click here to view code image](#)

```
C:\Dbg\x86>tlist winword
120 WINWORD.EXE          Chapter04.docm - Word
  CWD:      C:\Users\pavel\Documents\
  CmdLine:  "C:\Program Files (x86)\Microsoft
Office\Root\Office16\WINWORD.EXE" /n
"D:\OneDrive\WindowsInternalsBook\7thEdition\Chapter04\Chapter04.docm
VirtualSize:   778012 KB   PeakVirtualSize:   832680 KB
WorkingSetSize:185336 KB   PeakWorkingSetSize:227144 KB
NumberOfThreads: 45
12132 Win32StartAddr:0x00921000 LastErr:0x00000000 State:Waiting
15540 Win32StartAddr:0x6cc2fdd8 LastErr:0x00000000 State:Waiting
7096 Win32StartAddr:0x6cc3c6b2 LastErr:0x00000006 State:Waiting
17696 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
17492 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
4052 Win32StartAddr:0x70aa5cf7 LastErr:0x00000000 State:Waiting
14096 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
6220 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
7204 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
1196 Win32StartAddr:0x6ea016c0 LastErr:0x00000057 State:Waiting
8848 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
3352 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
11612 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
17420 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
13612 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
15052 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
...
12080 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
9456 Win32StartAddr:0x77c1c6d0 LastErr:0x00002f94 State:Waiting
9808 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
16208 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
9396 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
2688 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
9100 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
18364 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
11180 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
16.0.6741.2037 shp 0x00920000 C:\Program Files (x86)\Microsoft
```

```

Office\Root\
Office16\WINWORD.EXE
  10.0.10586.122 shp  0x77BF0000  C:\windows\SYSTEM32\ntdll.dll
    10.0.10586.0 shp  0x75540000  C:\windows\SYSTEM32\KERNEL32.DLL
  10.0.10586.162 shp  0x77850000  C:\windows\SYSTEM32\KERNELBASE.dll
    10.0.10586.63 shp  0x75AF0000  C:\windows\SYSTEM32\ADVAPI32.dll
  . . .
    10.0.10586.0 shp  0x68540000  C:\Windows\SYSTEM32\VssTrace.DLL
    10.0.10586.0 shp  0x5C390000  C:\Windows\SYSTEM32\adsldpc.dll
  10.0.10586.122 shp  0x5DE60000  C:\Windows\SYSTEM32\taskschd.dll
    10.0.10586.0 shp  0x5E3F0000  C:\Windows\SYSTEM32\srmstormod.dll
    10.0.10586.0 shp  0x5DCA0000  C:\Windows\SYSTEM32\srmscan.dll
    10.0.10586.0 shp  0x5D2E0000  C:\Windows\SYSTEM32\msdrm.dll
    10.0.10586.0 shp  0x711E0000  C:\Windows\SYSTEM32\srm_ps.dll
    10.0.10586.0 shp  0x56680000  C:\windows\System32\OpcServices.dll
                          0x5D240000  C:\Program Files (x86)\Common
Files\Microsoft
Shared\Office16\WXPNSE.DLL
  16.0.6701.1023 shp  0x77E80000  C:\Program Files (x86)\Microsoft
Office\Root\Office16\GR00VEEX.DLL
    10.0.10586.0
shp  0x693F0000  C:\windows\system32\dataexchange.dll

```

The TEB, illustrated in [Figure 4-3](#), is one of the data structures explained in this section that exists in the process address space (as opposed to the system space). Internally, it is made up of a header called the *Thread Information Block (TIB)*, which mainly existed for compatibility with OS/2 and Win9x applications. It also allows exception and stack information to be kept into a smaller structure when creating new threads by using an initial TIB.

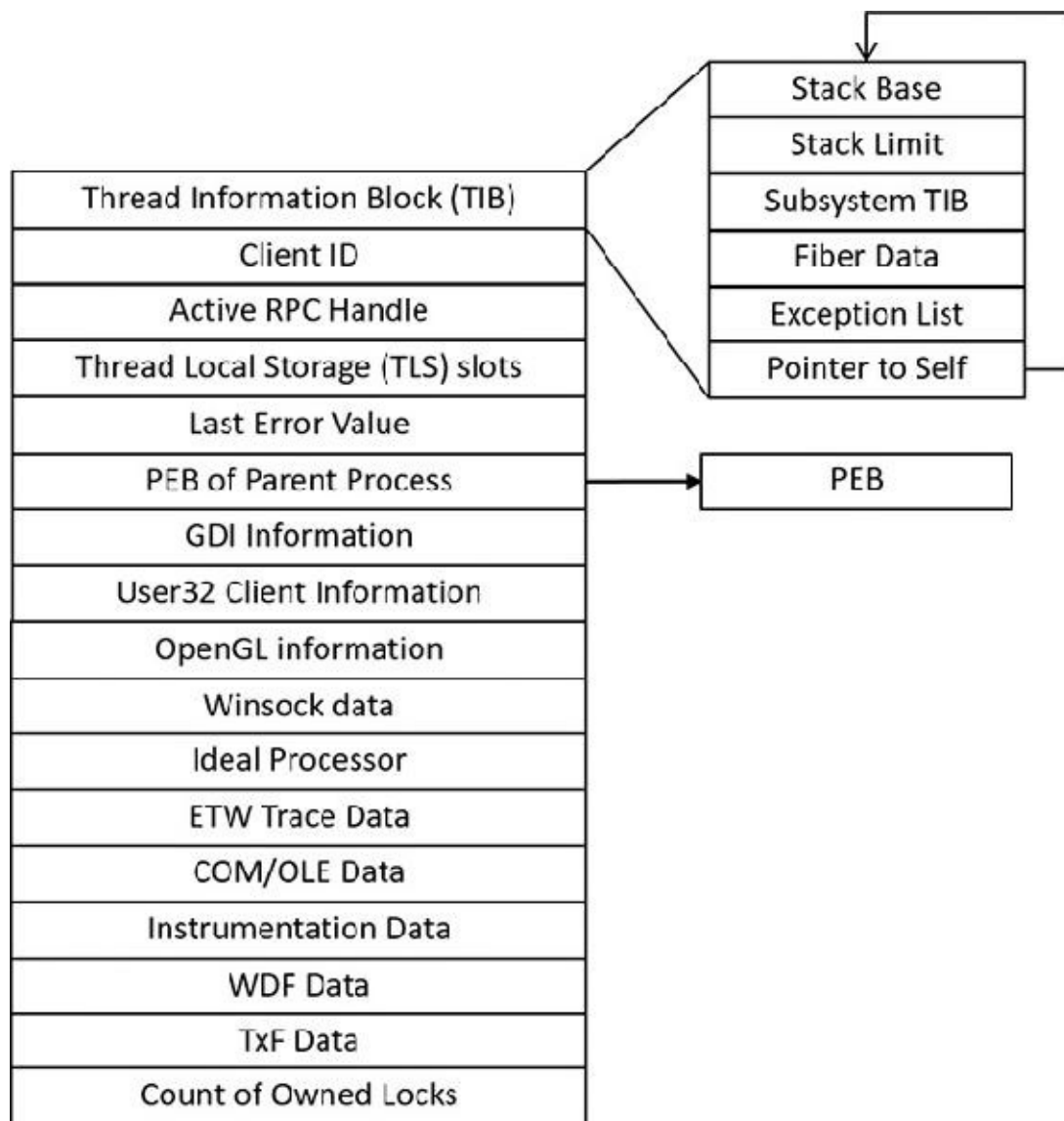


FIGURE 4-3 Important fields of the thread environment block.

The TEB stores context information for the image loader and various Windows DLLs. Because these components run in user mode, they need a data structure writable from user mode. That's why this structure exists in the process address space instead of in the system space, where it would be writable only from kernel mode. You can find the address of the TEB with the kernel debugger `!thread` command.

EXPERIMENT: Examining the TEB

You can dump the TEB structure with the `!teb` command in the kernel or user-mode debugger. The command can be used on its own to dump the TEB for the current thread of the debugger or with a TEB address to get it for an arbitrary thread. In case of a kernel debugger, the current process must be set before issuing the command on a TEB address so that the correct process context is used.

To view the TEB with a user-mode debugger, follow these steps. (You'll learn how to view the TEB using a kernel debugger in the next experiment.)

1. Open WinDbg.
2. Open the **File** menu and choose **Run Executable**.
3. Navigate to `c:\windows\system32\notepad.exe`. The debugger should break at the initial breakpoint.

4. Issue the **!teb** command to view the TEB of the only thread existing at the moment (the example is from 64 bit Windows):

[Click here to view code image](#)

```
0:000> !teb
TEB at 000000ef125c1000
ExceptionList:      0000000000000000
StackBase:          000000ef12290000
StackLimit:         000000ef1227f000
SubSystemTib:       0000000000000000
FiberData:          00000000000001e0
ArbitraryUserPointer: 0000000000000000
Self:               000000ef125c1000
EnvironmentPointer: 0000000000000000
ClientId:           000000000000021bc . 00000000000001b74
RpcHandle:          0000000000000000
Tls Storage:        00000266e572b600
PEB Address:        000000ef125c0000
LastErrorValue:     0
LastStatusValue:    0
Count Owned Locks:  0
HardErrorMode:      0
```

5. Enter the **g** command or press **F5** to proceed with running Notepad.
6. In Notepad, open the **File** menu and choose **Open**. Then click **Cancel** to dismiss the Open File dialog box.
7. Press **Ctrl+Break** or open the **Debug** menu and choose **Break** to forcibly break into the process.
8. Enter the **~** (tilde) command to show all threads in the process. You should see something like this:

[Click here to view code image](#)

```
0:005> ~
 0 Id: 21bc.1b74 Suspend: 1 Teb: 000000ef'125c1000 Unfrozen
 1 Id: 21bc.640 Suspend: 1 Teb: 000000ef'125e3000 Unfrozen
 2 Id: 21bc.1a98 Suspend: 1 Teb: 000000ef'125e5000 Unfrozen
 3 Id: 21bc.860 Suspend: 1 Teb: 000000ef'125e7000 Unfrozen
 4 Id: 21bc.28e0 Suspend: 1 Teb: 000000ef'125c9000 Unfrozen
. 5 Id: 21bc.23e0 Suspend: 1 Teb: 000000ef'12400000 Unfrozen
 6 Id: 21bc.244c Suspend: 1 Teb: 000000ef'125eb000 Unfrozen
 7 Id: 21bc.168c Suspend: 1 Teb: 000000ef'125ed000 Unfrozen
 8 Id: 21bc.1c90 Suspend: 1 Teb: 000000ef'125ef000 Unfrozen
 9 Id: 21bc.1558 Suspend: 1 Teb: 000000ef'125f1000 Unfrozen
10 Id: 21bc.a64 Suspend: 1 Teb: 000000ef'125f3000 Unfrozen
11 Id: 21bc.20c4 Suspend: 1 Teb: 000000ef'125f5000 Unfrozen
12 Id: 21bc.1524 Suspend: 1 Teb: 000000ef'125f7000 Unfrozen
13 Id: 21bc.1738 Suspend: 1 Teb: 000000ef'125f9000 Unfrozen
```

```
14 Id: 21bc.f48 Suspend: 1 Teb: 000000ef'125fb000 Unfrozen
15 Id: 21bc.17bc Suspend: 1 Teb: 000000ef'125fd000 Unfrozen
```

9. Each thread shows its TEB address. You can examine a specific thread by specifying its TEB address to the `!teb` command. Here's an example for thread 9 from the preceding output:

[Click here to view code image](#)

```
0:005> !teb 000000ef'125f1000
TEB at 000000ef125f1000
ExceptionList: 0000000000000000
StackBase: 000000ef13400000
StackLimit: 000000ef133ef000
SubSystemTib: 0000000000000000
FiberData: 00000000000001e0
ArbitraryUserPointer: 0000000000000000
Self: 000000ef125f1000
EnvironmentPointer: 0000000000000000
ClientId: 00000000000021bc . 00000000000001558
RpcHandle: 0000000000000000
Tls Storage: 00000266ea1af280
PEB Address: 000000ef125c0000
LastErrorValue: 0
LastStatusValue: c0000034
Count Owned Locks: 0
HardErrorMode: 0
```

10. Of course, it's possible to view the actual structure with the TEB address (truncated to conserve space):

[Click here to view code image](#)

```
0:005> dt ntdll!_teb 000000ef'125f1000
+0x000 NtTib : _NT_TIB
+0x038 EnvironmentPointer : (null)
+0x040 ClientId : _CLIENT_ID
+0x050 ActiveRpcHandle : (null)
+0x058 ThreadLocalStoragePointer : 0x00000266'ea1af280 Void
+0x060 ProcessEnvironmentBlock : 0x000000ef'125c0000 _PEB
+0x068 LastErrorValue : 0
+0x06c CountOfOwnedCriticalSections : 0
...
+0x1808 LockCount : 0
+0x180c WowTebOffset : 0n0
+0x1810 ResourceRetValue : 0x00000266'ea2a5e50 Void
+0x1818 ReservedForWdf : (null)
+0x1820 ReservedForCrt : 0
+0x1828 EffectiveContainerId : _GUID {00000000-0000-0000-0000-000000000000}
```

EXPERIMENT: Examining the TEB with a kernel debugger

Follow these steps to view the TEB with a kernel debugger:

1. Find the process for which a thread's TEB is of interest. For example, the following looks for explorer.exe processes and lists its threads with basic information (truncated):

[Click here to view code image](#)

```
lkd> !process 0 2 explorer.exe
PROCESS fffffe0012bea7840
    SessionId: 2  Cid: 10d8    Peb: 00251000  ParentCid: 10bc
    DirBase: 76e12000  ObjectTable:
fffffc000e1ca0c80  HandleCount: <Data Not
Accessible>
    Image: explorer.exe

    THREAD fffffe0012bf53080  Cid 10d8.10dc  Teb:
0000000000252000
Win32Thread: fffffe0012c1532f0 WAIT: (WrUserRequest) UserMode
Non-Alertable
    fffffe0012c257fe0  SynchronizationEvent

    THREAD fffffe0012a30f080  Cid 10d8.114c  Teb:
0000000000266000
Win32Thread: fffffe0012c2e9a20 WAIT: (UserRequest) UserMode
Alertable
    fffffe0012bab85d0  SynchronizationEvent

    THREAD fffffe0012c8bd080  Cid 10d8.1178  Teb:
000000000026c000
Win32Thread: fffffe0012a801310 WAIT: (UserRequest) UserMode
Alertable
    fffffe0012bfd9250  NotificationEvent
    fffffe0012c9512f0  NotificationEvent
    fffffe0012c876b80  NotificationEvent
    fffffe0012c010fe0  NotificationEvent
    fffffe0012d0ba7e0  NotificationEvent
    fffffe0012cf9d1e0  NotificationEvent
    ...

    THREAD fffffe0012c8be080  Cid 10d8.1180  Teb:
0000000000270000
Win32Thread: 0000000000000000 WAIT: (UserRequest) UserMode
Alertable
    fffff80156946440  NotificationEvent

    THREAD fffffe0012afd4040  Cid 10d8.1184  Teb:
0000000000272000
Win32Thread: fffffe0012c7c53a0 WAIT: (UserRequest) UserMode Non-
```

Alertable

```
fffffe0012a3dafa0 NotificationEvent
fffffe0012c21ee70 Semaphore Limit 0xffff
fffffe0012c8db6f0 SynchronizationEvent
```

```
THREAD fffffe0012c88a080 Cid 10d8.1188 Teb:
00000000000274000
Win32Thread: 0000000000000000 WAIT: (UserRequest) UserMode
Alertable
```

```
fffffe0012afd4920 NotificationEvent
fffffe0012c87b480 SynchronizationEvent
fffffe0012c87b400 SynchronizationEvent
```

...

2. If more than one explorer.exe process exists, select one arbitrarily for the following steps.
3. Each thread shows the address of its TEB. Because the TEB is in user space, the address has meaning only in the context of the relevant process. You need to switch to the process/thread as seen by the debugger. Select the first thread of explorer because its kernel stack is probably resident in physical memory. Otherwise, you'll get an error.

[Click here to view code image](#)

```
lkd> .thread /p fffffe0012bf53080
Implicit thread is now fffffe001'2bf53080
Implicit process is now fffffe001'2bea7840
```

4. This switches the context to the specified thread (and by extension, the process). Now you can use the !teb command with the TEB address listed for that thread:

[Click here to view code image](#)

```
lkd> !teb 00000000000252000
TEB at 00000000000252000
ExceptionList: 0000000000000000
StackBase: 00000000000d0000
StackLimit: 00000000000c2000
SubSystemTib: 0000000000000000
FiberData: 00000000000001e00
ArbitraryUserPointer: 0000000000000000
Self: 00000000000252000
EnvironmentPointer: 0000000000000000
ClientId: 00000000000010d8 . 000000000000010dc
RpcHandle: 0000000000000000
Tls Storage: 0000000009f73f30
PEB Address: 00000000000251000
LastErrorValue: 0
LastStatusValue: c0150008
Count Owned Locks: 0
HardErrorMode: 0
```

The CSR_THREAD, illustrated in [Figure 4-4](#) is analogous to the data structure of CSR_PROCESS, but it's applied to threads. As you might recall, this is maintained by each Csrss process within a session and identifies the Windows subsystem threads running within it. CSR_THREAD stores a handle that Csrss keeps for the thread, various flags, the client ID (thread ID and process ID), and a copy of the thread's creation time. Note that threads are registered with Csrss when they send their first message to Csrss, typically due to some API that requires notifying Csrss of some operation or condition.

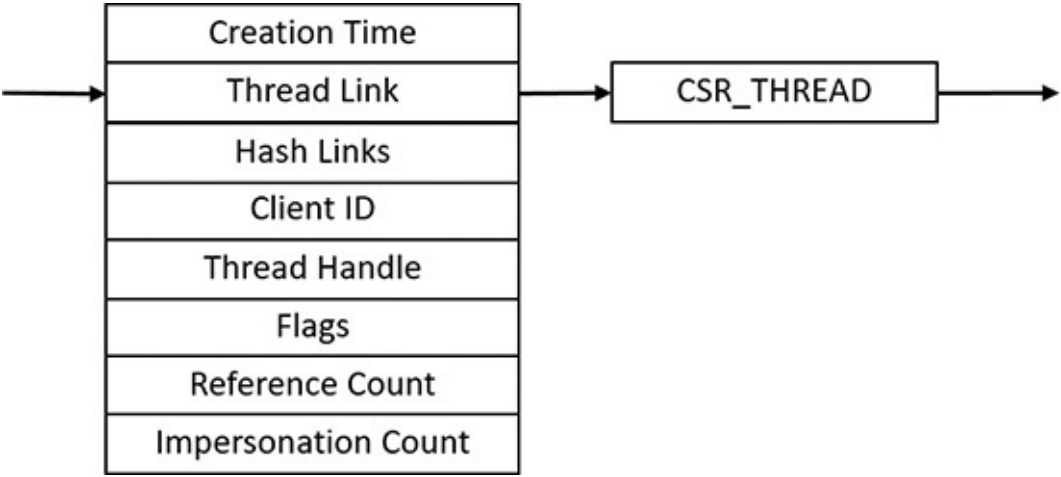


FIGURE 4-4 Fields of the CSR thread.

EXPERIMENT: Examining the CSR_THREAD

You can dump the CSR_THREAD structure with the dt command in the kernel-mode debugger while the debugger context is set to a Csrss process. Follow the instructions in the experiment “[Examining the CSR_PROCESS](#)” in [Chapter 3](#) to perform this operation. Here is an example output from Windows 10 x64 system:

[Click here to view code image](#)

```
lkd> dt csrss!_csr_thread
+0x000 CreateTime      : _LARGE_INTEGER
+0x008 Link            : _LIST_ENTRY
+0x018 HashLinks       : _LIST_ENTRY
+0x028 ClientId        : _CLIENT_ID
+0x038 Process         : Ptr64 _CSR_PROCESS
+0x040 ThreadHandle    : Ptr64 Void
+0x048 Flags           : Uint4B
+0x04c ReferenceCount  : Int4B
+0x050 ImpersonateCount : Uint4B
```

Finally, the W32THREAD structure, illustrated in [Figure 4-5](#), is analogous to the data structure of W32PROCESS, but it's applied to threads. This structure mainly contains information useful for the GDI subsystem (brushes and Device Context attributes) and DirectX, as well as for the User Mode Print Driver (UMPD) framework that vendors use to write user-mode printer drivers. Finally, it contains a rendering state useful for desktop compositing and anti-aliasing.

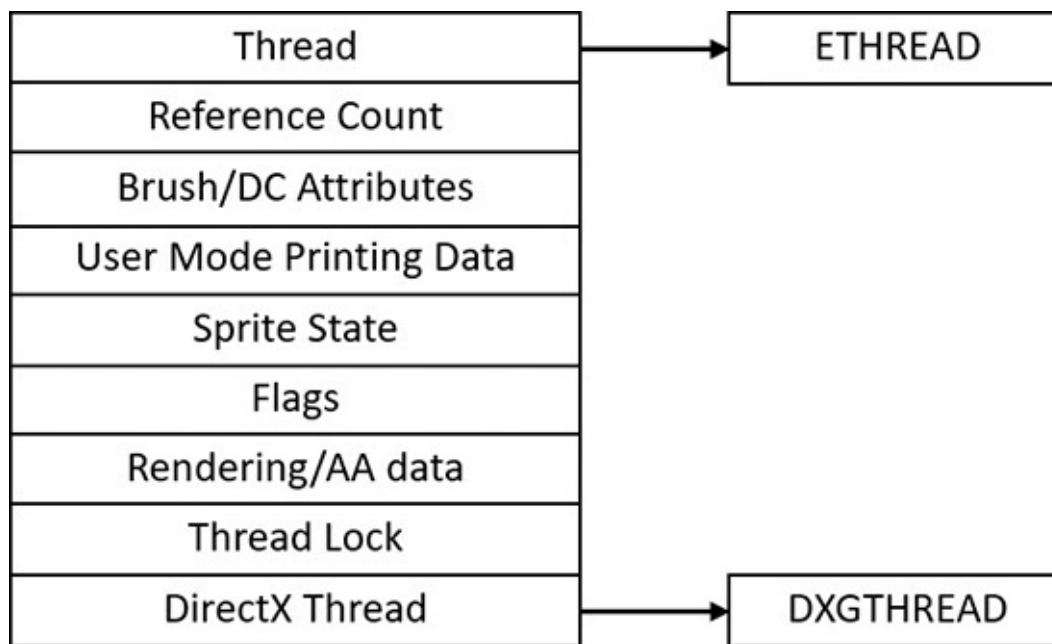


FIGURE 4-5 Fields of the Win32k thread.

Birth of a thread

A thread's life cycle starts when a process (in the context of some thread, such as the thread running the main function) creates a new thread. The request filters down to the Windows executive, where the process manager allocates space for a thread object and calls the kernel to initialize the thread control block (KTHREAD). As mentioned, the various thread-creation functions eventually end up at `CreateRemoteThreadEx`. The following steps are taken inside this function in `Kernel32.dll` to create a Windows thread:

1. The function converts the Windows API parameters to native flags and builds a native structure describing object parameters (`OBJECT_ATTRIBUTES`, described in Chapter 8 in Part 2).
2. It builds an attribute list with two entries: client ID and TEB address. (For more information on attribute lists, see the section "[Flow of CreateProcess](#)" in [Chapter 3](#).)
3. It determines whether the thread is created in the calling process or another process indicated by the handle passed in. If the handle is equal to the pseudo handle returned from `GetCurrentProcess` (with a value of `-1`), then it's the same process. If the process handle is different, it could still be a valid handle to the same process, so a call is made to `NtQueryInformationProcess` (in `Ntdll`) to find out whether that is indeed the case.
4. It calls `NtCreateThreadEx` (in `Ntdll`) to make the transition to the executive in kernel mode and continues inside a function with the same name and arguments.
5. `NtCreateThreadEx` (inside the executive) creates and initializes the user-mode thread context (its structure is architecture-specific) and then calls `PspCreateThread` to create a suspended executive thread object. (For a description of the steps performed by this function, see the descriptions of stage 3 and stage 5 in [Chapter 3](#) in the section "[Flow of CreateProcess](#).") Then the function returns, eventually ending back in user mode at `CreateRemoteThreadEx`.
6. `CreateRemoteThreadEx` allocates an activation context for the thread used by side-by-side assembly support. It then queries the activation stack to see if it requires activation and activates it if needed. The activation stack pointer is saved in the new thread's TEB.
7. Unless the caller created the thread with the `CREATE_SUSPENDED` flag set, the thread is now

resumed so that it can be scheduled for execution. When the thread starts running, it executes the steps described in [Chapter 3](#) in the section “[Stage 7: performing process initialization in the context of the new process](#)” before calling the actual user’s specified start address.

8. The thread handle and the thread ID are returned to the caller.

Examining thread activity

Examining thread activity is especially important if you are trying to determine why a process that is hosting multiple services is running (such as Svchost.exe, Dllhost.exe, or Lsass.exe) or why a process has stopped responding.

There are several tools that expose various elements of the state of Windows threads: WinDbg (in user-process attach and kernel-debugging mode), Performance Monitor, and Process Explorer. (The tools that show thread-scheduling information are listed in the section “[Thread scheduling](#).”)

To view the threads in a process with Process Explorer, select a process and double-click it to open its Properties dialog box. Alternatively, right-click the process and select the **Properties** menu item. Then click the **Threads** tab. This tab shows a list of the threads in the process and four columns of information for each thread: its ID, the percentage of CPU consumed (based on the refresh interval configured), the number of cycles charged to the thread, and the thread start address. You can sort by any of these four columns.

New threads that are created are highlighted in green, and threads that exit are highlighted in red. (To configure the highlight duration, open the **Options** menu and choose **Difference Highlight Duration**.) This might be helpful to discover unnecessary thread creation occurring in a process. (In general, threads should be created at process startup, not every time a request is processed inside a process.)

As you select each thread in the list, Process Explorer displays the thread ID, start time, state, CPU time counters, number of cycles charged, number of context switches, the ideal processor and its group, and the I/O priority, memory priority, and base and current (dynamic) priority. There is a Kill button, which terminates an individual thread, but this should be used with extreme care. Another option is the Suspend button, which prevents the thread from forward execution and thus prevents a runaway thread from consuming CPU time. However, this can also lead to deadlocks and should be used with the same care as the Kill button. Finally, the Permissions button allows you to view the security descriptor of the thread. (See [Chapter 7](#), “[Security](#),” for more information on security descriptors.)

Unlike Task Manager and all other process/processor monitoring tools, Process Explorer uses the clock cycle counter designed for thread run-time accounting (described later in this chapter) instead of the clock interval timer, so you will see a significantly different view of CPU consumption using Process Explorer. This is because many threads run for such a short time that they are seldom (if ever) the currently running thread when the clock interval timer interrupt occurs. As a result, they are not charged for much of their CPU time, leading clock-based tools to perceive a CPU usage of 0 percent. On the other hand, the total number of clock cycles represents the actual number of processor cycles that each thread in the process accrued. It is independent of the clock interval timer’s resolution because the count is maintained internally by the processor at each cycle and updated by Windows at each interrupt entry. (A final accumulation is done before a context switch.)

The thread start address is displayed in the form *module!function*, where *module* is the name of the .EXE or .DLL. The function name relies on access to symbol files for the module (see the section “[Experiment: Viewing process details with Process Explorer](#)” in [Chapter 1](#), “[Concepts and tools](#)”). If you are unsure what the module is, click the **Module** button to open an Explorer file Properties dialog box for the module containing the thread’s start address (for example, the .EXE or .DLL).



Note

For threads created by the Windows `CreateThread` function, Process Explorer displays the function passed to `CreateThread`, not the actual thread start function. This is because all Windows threads start at a common thread startup wrapper function (`RtlUserThreadStart` in `Ntdll.dll`). If Process Explorer showed the actual start address, most threads in processes would appear to have started at the same address, which would not be helpful in trying to understand what code the thread was executing. However, if Process Explorer can't query the user-defined startup address (such as in the case of a protected process), it will show the wrapper function, so you will see all threads starting at `RtlUserThreadStart`.

The thread start address displayed might not be enough information to pinpoint what the thread is doing and which component within the process is responsible for the CPU consumed by the thread. This is especially true if the thread start address is a generic startup function—for example, if the function name does not indicate what the thread is actually doing. In this case, examining the thread stack might answer the question. To view the stack for a thread, double-click the thread of interest (or select it and click the **Stack** button). Process Explorer displays the thread's stack (both user and kernel, if the thread was in kernel mode).



Note

While the user-mode debuggers (WinDbg, Ntld, and Cdb) permit you to attach to a process and display the user stack for a thread, Process Explorer shows both the user and kernel stack in one easy click of a button. You can also examine user and kernel thread stacks using WinDbg in local kernel debugging mode, as the next two experiments demonstrate.

When looking at 32-bit processes running on 64-bit systems as a Wow64 process (see Chapter 8 in Part 2 for more information on Wow64), Process Explorer shows both the 32-bit and 64-bit stack for threads. Because at the time of the real (64 bit) system call, the thread has been switched to a 64-bit stack and context, simply looking at the thread's 64-bit stack would reveal only half the story—the 64-bit part of the thread, with Wow64's thunking code. So, when examining Wow64 processes, be sure to take into account both the 32-bit and 64-bit stacks.

EXPERIMENT: Viewing a thread stack with a user-mode debugger

Follow these steps to attach WinDbg to a process and view thread information and its stack:

1. Run `notepad.exe` and `WinDbg.exe`.
2. In WinDbg, open the **File** menu and select **Attach to Process**.
3. Find the `notepad.exe` instance and click **OK** to attach. The debugger should break into Notepad.
4. List the existing threads in the process with the `~` command. Each thread shows its debugger ID, the client ID (`ProcessID.ThreadID`), its suspend count (this should be 1 most of the time, as it is suspended because of the breakpoint), the TEB address, and whether it has been frozen using a debugger command.

[Click here to view code image](#)

```
0:005> ~
  0 Id: 612c.5f68 Suspend: 1 Teb: 00000022'41da2000 Unfrozen
  1 Id: 612c.5564 Suspend: 1 Teb: 00000022'41da4000 Unfrozen
  2 Id: 612c.4f88 Suspend: 1 Teb: 00000022'41da6000 Unfrozen
  3 Id: 612c.5608 Suspend: 1 Teb: 00000022'41da8000 Unfrozen
  4 Id: 612c.cf4 Suspend: 1 Teb: 00000022'41daa000 Unfrozen
.  5 Id: 612c.9f8 Suspend: 1 Teb: 00000022'41db0000 Unfrozen
```

5. Notice the dot preceding thread 5 in the output. This is the current debugger thread. Issue the **k** command to view the call stack:

[Click here to view code image](#)

```
0:005> k
# Child-SP          RetAddr          Call Site
00 00000022'421ff7e8 00007ff8'504d9031 ntdll!DbgBreakPoint
01 00000022'421ff7f0 00007ff8'501b8102
ntdll!DbgUiRemoteBreakin+0x51
02 00000022'421ff820 00007ff8'5046c5b4
KERNEL32!BaseThreadInitThunk+0x22
03 00000022'421ff850 00000000'00000000
ntdll!RtlUserThreadStart+0x34
```

6. The debugger injected a thread into Notepad's process that issues a breakpoint instruction (DbgBreakPoint). To view the call stack of another thread, use the **~nk** command, where **n** is the thread number as seen by WinDbg. (This does not change the current debugger thread.) Here's an example for thread 2:

[Click here to view code image](#)

```
0:005> ~2k
# Child-SP          RetAddr          Call Site
00 00000022'41f7f9e8 00007ff8'5043b5e8
ntdll!ZwWaitForWorkViaWorkerFactory+0x14
01 00000022'41f7f9f0 00007ff8'501b8102
ntdll!TppWorkerThread+0x298
02 00000022'41f7fe00 00007ff8'5046c5b4
KERNEL32!BaseThreadInitThunk+0x22
03 00000022'41f7fe30 00000000'00000000
ntdll!RtlUserThreadStart+0x34
```

7. To switch the debugger to another thread, use the **~ns** command (again, **n** is the thread number). Let's switch to thread 0 and show its stack:

[Click here to view code image](#)

```
0:005> ~0s
USER32!ZwUserGetMessage+0x14:
00007ff8'502e21d4 c3          ret
0:000> k
# Child-SP          RetAddr          Call Site
00 00000022'41e7f048 00007ff8'502d3075
```

```

USER32!ZwUserGetMessage+0x14
01 00000022'41e7f050 00007ff6'88273bb3 USER32!GetMessageW+0x25
02 00000022'41e7f080 00007ff6'882890b5 notepad!WinMain+0x27b
03 00000022'41e7f180 00007ff8'341229b8
notepad!__mainCRTStartup+0x1ad
04 00000022'41e7f9f0 00007ff8'5046c5b4
KERNEL32!BaseThreadInitThunk+0x22
05 00000022'41e7fa20 00000000'00000000
ntdll!RtlUserThreadStart+0x34

```

8. Note that even though a thread might be in kernel mode at the time, a user-mode debugger shows its last function that's still in user mode (ZwUserGetMessage in the preceding output).

EXPERIMENT: Viewing a thread stack with a local kernel-mode debugger

In this experiment, you'll use a local kernel debugger to view a thread's stack (both user mode and kernel mode). The experiment uses one of Explorer's threads, but you can try it with other processes or threads.

1. Show all the processes running the image explorer.exe. (Note that you may see more than one instance of Explorer if the Launch Folder Windows in a Separate Process option in Explorer options is selected. One process manages the desktop and taskbar, while the other manages Explorer windows.)

[Click here to view code image](#)

```

lkd> !process 0 0 explorer.exe
PROCESS fffffe00197398080
    SessionId: 1  Cid: 18a0    Peb: 00320000  ParentCid: 1840
    DirBase: 17c028000  ObjectTable:
fffffc000bd4aa880  HandleCount: <Data
Not Accessible>
    Image: explorer.exe

PROCESS fffffe00196039080
    SessionId: 1  Cid: 1f30    Peb: 00290000  ParentCid: 0238
    DirBase: 24cc7b000  ObjectTable:
fffffc000bbbef740  HandleCount: <Data
Not Accessible>
    Image: explorer.exe

```

2. Select one instance and show its thread summary:

[Click here to view code image](#)

```

lkd> !process fffffe00196039080 2
PROCESS fffffe00196039080
    SessionId: 1  Cid: 1f30    Peb: 00290000  ParentCid: 0238
    DirBase: 24cc7b000  ObjectTable:
fffffc000bbbef740  HandleCount: <Data

```

Not Accessible>
Image: explorer.exe

THREAD fffffe0019758f080 Cid 1f30.0718 Teb:
0000000000291000
Win32Thread: fffffe001972e3220 WAIT: (UserRequest) UserMode Non-
Alertable
fffffe00192c08150 SynchronizationEvent

THREAD fffffe00198911080 Cid 1f30.1aac Teb:
00000000002a1000
Win32Thread: fffffe001926147e0 WAIT: (UserRequest) UserMode Non-
Alertable
fffffe00197d6e150 SynchronizationEvent
fffffe001987bf9e0 SynchronizationEvent

THREAD fffffe00199553080 Cid 1f30.1ad4 Teb:
00000000002b1000
Win32Thread: fffffe0019263c740 WAIT: (UserRequest) UserMode Non-
Alertable
fffffe0019ac6b150 NotificationEvent
fffffe0019a7da5e0 SynchronizationEvent

THREAD fffffe0019b6b2800 Cid 1f30.1758 Teb:
00000000002bd000
Win32Thread: 0000000000000000 WAIT: (Suspended) KernelMode Non-
Alertable
SuspendCount 1
fffffe0019b6b2ae0 NotificationEvent

...

3. Switch to the context of the first thread in the process (you can select other threads):

[Click here to view code image](#)

```
lkd> .thread /p /r fffffe0019758f080
Implicit thread is now fffffe001'9758f080
Implicit process is now fffffe001'96039080
Loading User Symbols
.....
```

4. Now look at the thread to show its details and its call stack (addresses are truncated in the shown output):

[Click here to view code image](#)

```
lkd> !thread fffffe0019758f080
THREAD fffffe0019758f080 Cid 1f30.0718 Teb: 0000000000291000
Win32Thread :
fffffe001972e3220 WAIT : (UserRequest)UserMode Non - Alertable
```

fffffe00192c08150 SynchronizationEvent
Not impersonating
DeviceMap fffffc000b77f1f30
Owning Process fffffe00196039080 Image :
explorer.exe
Attached Process N / A Image : N / A
Wait Start TickCount 17415276 Ticks : 146 (0:00 : 00
: 02.281)
Context Switch Count 2788 IdealProcessor : 4
UserTime 00 : 00 : 00.031
KernelTime 00 : 00 : 00.000
*** WARNING : Unable to verify checksum for C :
\windows\explorer.exe
Win32 Start Address
explorer!wWinMainCRTStartup(0x00007ff7b80de4a0)
Stack Init fffffd0002727cc90 Current fffffd0002727bf80
Base fffffd0002727d000 Limit fffffd00027277000 Call
0000000000000000
Priority 8 BasePriority 8 PriorityDecrement 0 IoPriority 2
PagePriority 5

... Call Site
... nt!KiSwapContext + 0x76
... nt!KiSwapThread + 0x15a
... nt!KiCommitThreadWait + 0x149
... nt!KeWaitForSingleObject + 0x375
... nt!ObWaitForMultipleObjects + 0x2bd
... nt!NtWaitForMultipleObjects + 0xf6
... nt!KiSystemServiceCopyEnd + 0x13 (TrapFrame @
fffffd000'2727cb00)
... ntdll!ZwWaitForMultipleObjects + 0x14
... KERNELBASE!WaitForMultipleObjectsEx + 0xef
... USER32!RealMsgWaitForMultipleObjectsEx + 0xdb
... USER32!MsgWaitForMultipleObjectsEx + 0x152
... explorerframe!SHProcessMessagesUntilEventsEx + 0x8a
... explorerframe!SHProcessMessagesUntilEventEx + 0x22
... explorerframe!CE ExplorerHostCreator::RunHost + 0x6d
... explorer!wWinMain + 0xa04fd
... explorer!__wmainCRTStartup + 0x1d6

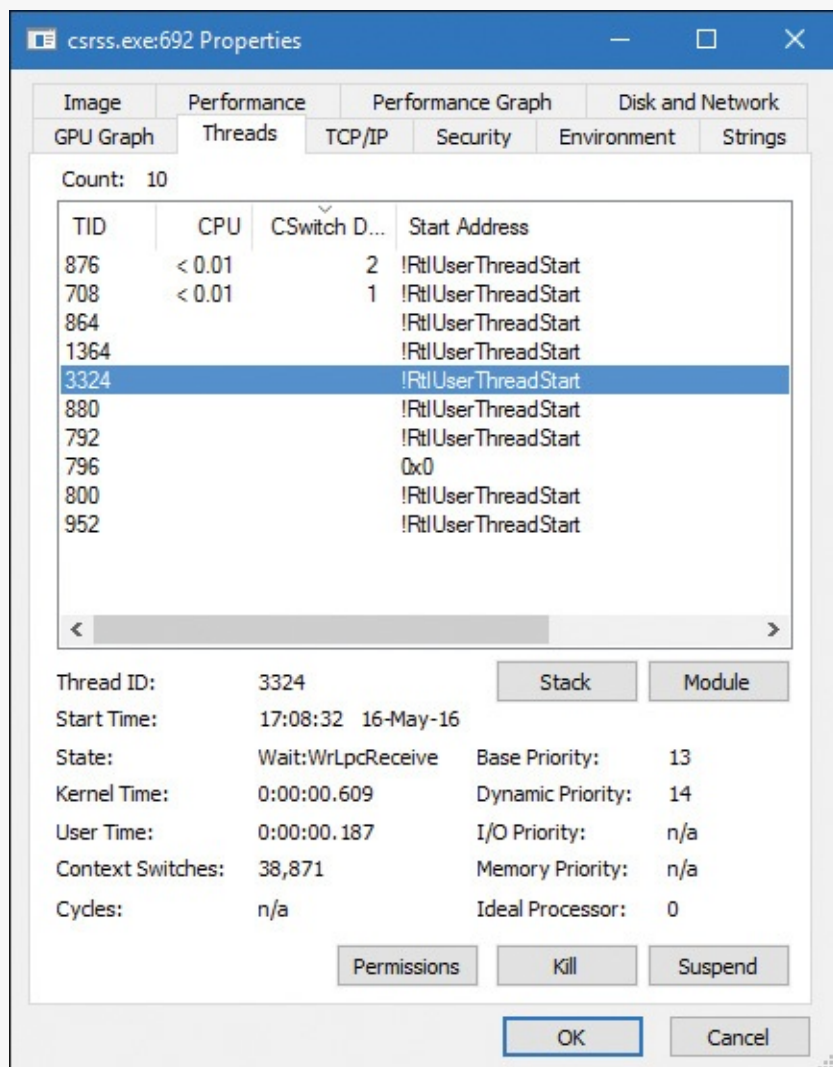
Limitations on protected process threads

As discussed in [Chapter 3](#), protected processes (classic protected or PPL) have several limitations in terms of which access rights will be granted, even to the users with the highest privileges on the system. These limitations also apply to threads inside such a process. This ensures that the actual code running inside the protected process cannot be hijacked or otherwise affected through standard Windows functions, which require access rights that are not granted for protected process threads. In fact, the only permissions granted are `THREAD_SUSPEND_RESUME` and `THREAD_SET/QUERY_LIMITED_INFORMATION`.

EXPERIMENT: Viewing protected process thread information with Process Explorer

In this experiment, you'll view protected process thread information. Follow these steps:

1. Find any protected or PPL process, such as the Audiodg.exe or Csrss.exe process inside the process list.
2. Open the process's Properties dialog box and click the **Threads** tab.



3. Process Explorer doesn't show the Win32 thread start address. Instead, it displays the standard thread start wrapper inside Ntdll.dll. If you click the **Stack** button, you'll get an error, because Process Explorer needs to read the virtual memory inside the protected process, which it can't do.
4. Note that although the base and dynamic priorities are shown, the I/O and memory priorities are not (nor is Cycles), which is another example of the limited access right `THREAD_QUERY_LIMITED_INFORMATION` versus full query information access right (`THREAD_QUERY_INFORMATION`).
5. Try to kill a thread inside a protected process. When you do, notice yet another access-denied error: recall the lack of `THREAD_TERMINATE` access.

Thread scheduling

This section describes the Windows scheduling policies and algorithms. The first subsection provides a condensed description of how scheduling works on Windows and a definition of key terms. Then Windows priority levels are described from both the Windows API and the Windows kernel points of view. After a review of the relevant Windows utilities and tools that relate to scheduling, the detailed data structures and algorithms that make up the Windows scheduling system are presented, including a description of common scheduling scenarios and how thread selection, as well as processor selection, occurs.

Overview of Windows scheduling

Windows implements a priority-driven, preemptive scheduling system. At least one of the highest-priority runnable (ready) threads always runs, with the caveat that certain high-priority threads ready to run might be limited by the processors on which they might be allowed or preferred to run on—phenomenon called *processor affinity*. Processor affinity is defined based on a given processor group, which collects up to 64 processors. By default, threads can run only on available processors within the processor group associated with the process. (This is to maintain compatibility with older versions of Windows, which supported only 64 processors). Developers can alter processor affinity by using the appropriate APIs or by setting an affinity mask in the image header, and users can use tools to change affinity at run time or at process creation. However, although multiple threads in a process can be associated with different groups, a thread on its own can run only on the processors available within its assigned group. Additionally, developers can choose to create group-aware applications, which use extended scheduling APIs to associate logical processors on different groups with the affinity of their threads. Doing so converts the process into a multigroup process that can theoretically run its threads on any available processor within the machine.

After a thread is selected to run, it runs for an amount of time called a *quantum*. A quantum is the length of time a thread is allowed to run before another thread at the same priority level is given a turn to run. Quantum values can vary from system to system and process to process for any of three reasons:

- System configuration settings (long or short quanta, variable or fixed quanta, and priority separation)
- Foreground or background status of the process
- Use of the job object to alter the quantum

These details are explained in the “Quantum” section later in this chapter.

A thread might not get to complete its quantum, however, because Windows implements a preemptive scheduler. That is, if another thread with a higher priority becomes ready to run, the currently running thread might be preempted before finishing its time slice. In fact, a thread can be selected to run next and be preempted before even beginning its quantum!

The Windows scheduling code is implemented in the kernel. There’s no single “scheduler” module or routine, however. The code is spread throughout the kernel in which scheduling-related events occur. The routines that perform these duties are collectively called the kernel’s *dispatcher*. The following events might require thread dispatching:

- A thread becomes ready to execute—for example, a thread has been newly created or has just been released from the wait state.
- A thread leaves the running state because its time quantum ends, it terminates, it yields execution, or it enters a wait state.

- A thread's priority changes, either because of a system service call or because Windows itself changes the priority value.
- A thread's processor affinity changes so that it will no longer run on the processor on which it was running.

At each of these junctions, Windows must determine which thread should run next on the logical processor that was running the thread, if applicable, or on which logical processor the thread should now run. After a logical processor has selected a new thread to run, it eventually performs a context switch to it. A *context switch* is the procedure of saving the volatile processor state associated with a running thread, loading another thread's volatile state, and starting the new thread's execution.

As noted, Windows schedules at the thread granularity level. This approach makes sense when you consider that processes don't run; rather, they only provide resources and a context in which their threads run. Because scheduling decisions are made strictly on a thread basis, no consideration is given to what process the thread belongs to. For example, if process A has 10 runnable threads, process B has 2 runnable threads, and all 12 threads are at the same priority, each thread would theoretically receive one-twelfth of the CPU time. That is, Windows wouldn't give 50 percent of the CPU to process A and 50 percent to process B.

Priority levels

To understand the thread-scheduling algorithms, one must first understand the priority levels that Windows uses. As illustrated in [Figure 4-6](#), Windows uses 32 priority levels internally, ranging from 0 to 31 (31 is the highest). These values divide up as follows:

- Sixteen real-time levels (16 through 31)
- Sixteen variable levels (0 through 15), out of which level 0 is reserved for the zero page thread (described in [Chapter 5](#)).

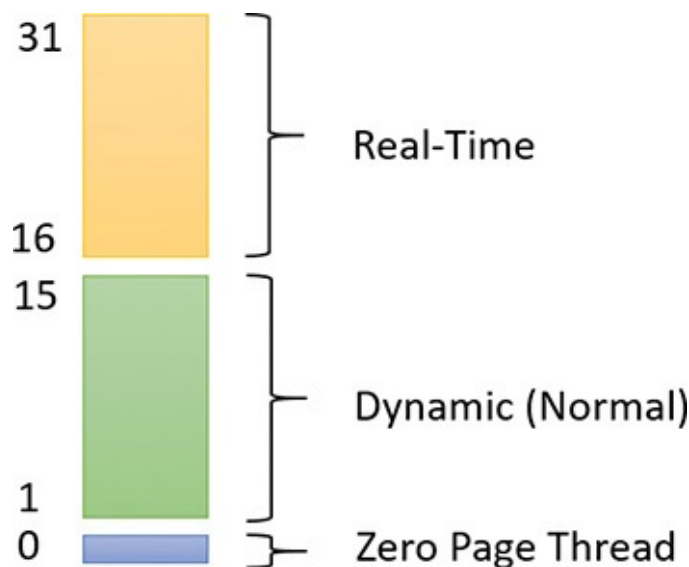


FIGURE 4-6 Thread priority levels.

Thread priority levels are assigned from two different perspectives: those of the Windows API and those of the Windows kernel. The Windows API first organizes processes by the priority class to which they are assigned at creation (the numbers in parentheses represent the internal `PROCESS_PRIORITY_CLASS` index recognized by the kernel):

- Real-Time (4)
- High (3)

- Above Normal (6)
- Normal (2)
- Below Normal (5)
- Idle (1)

The Windows API `SetPriorityClass` allows changing a process's priority class to one of these levels.

It then assigns a relative priority of the individual threads within those processes. Here, the numbers represent a priority delta that is applied to the process base priority:

- Time-Critical (15)
- Highest (2)
- Above-Normal (1)
- Normal (0)
- Below-Normal (-1)
- Lowest (-2)
- Idle (-15)

Time-Critical and Idle levels (+15 and -15) are called *saturation values* and represent specific levels that are applied rather than true offsets. These values can be passed to the `SetThreadPriority` Windows API to change a thread's relative priority.

Therefore, in the Windows API, each thread has a base priority that is a function of its process priority class and its relative thread priority. In the kernel, the process priority class is converted to a base priority by using the `PspPriorityTable` global array and the `PROCESS_PRIORITY_CLASS` indices shown earlier, which sets priorities of 4, 8, 13, 14, 6, and 10, respectively. (This is a fixed mapping that cannot be changed.) The relative thread priority is then applied as a differential to this base priority. For example, a Highest thread will receive a thread base priority of two levels higher than the base priority of its process.

This mapping from Windows priority to internal Windows numeric priority is shown graphically in [Figure 4-7](#) and textually in [Table 4-1](#).

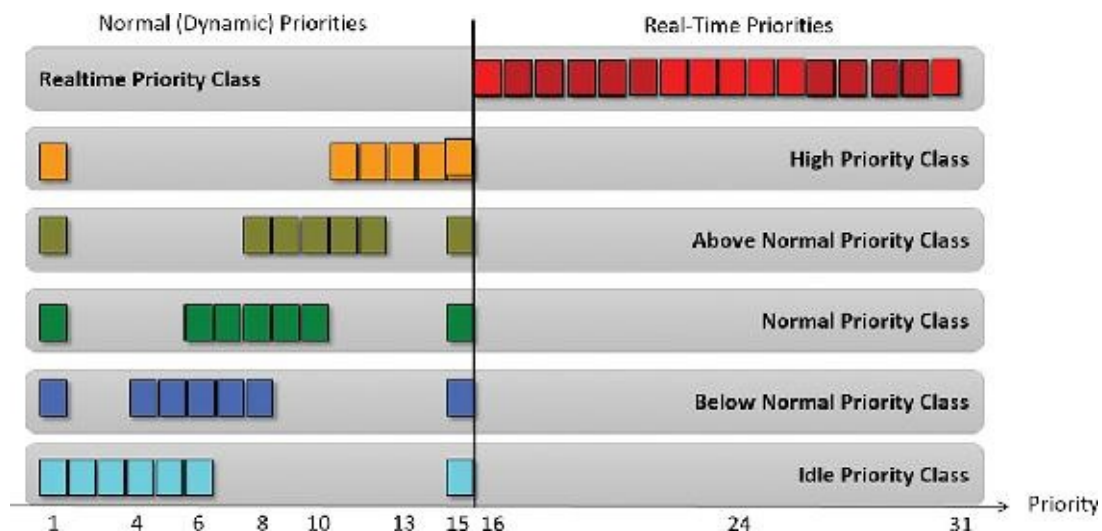


FIGURE 4-7 A graphic view of available thread priorities from a Windows API perspective.

Priority Class Relative Priority	Real-Time	High	Above-Normal	Normal	Below-Normal	Idle
Time Critical (+ Saturation)	31	15	15	15	15	15
Highest (+2)	26	15	12	10	8	6
Above Normal (+1)	25	14	11	9	7	5
Normal (0)	24	13	10	8	6	4
Below Normal (-1)	23	12	9	7	5	3
Lowest (-2)	22	11	8	6	4	2
Idle (-Saturation)	16	1	1	1	1	1

TABLE 4-1 Mapping Windows kernel priorities to the Windows API

You'll note that the Time-Critical and Idle relative thread priorities maintain their respective values regardless of the process priority class (unless it is Real-Time). This is because the Windows API requests saturation of the priority from the kernel, by passing in +16 or -16 as the requested relative priority. The formula used to get these values is as follows (HIGH_PRIORITY equals 31):

[Click here to view code image](#)

If Time-Critical: $((\text{HIGH_PRIORITY} + 1) / 2)$

If Idle: $-(\text{HIGH_PRIORITY} + 1) / 2$

These values are then recognized by the kernel as a request for saturation, and the `Saturation` field in `KTHREAD` is set. For positive saturation, this causes the thread to receive the highest possible priority within its priority class (dynamic or real-time); for negative saturation, it's the lowest possible one. Additionally, future requests to change the base priority of the process will no longer affect the base priority of these threads because saturated threads are skipped in the processing code.

As shown in [Table 4-1](#), threads have seven levels of possible priorities to set as viewed from the Windows API (six levels for the High priority class). The Real-Time priority class actually allows setting all priority levels between 16 and 31 (as shown in [Figure 4-7](#)). The values not covered by the standard constants shown in the table can be specified with the values -7, -6, -5, -4, -3, 3, 4, 5, and 6 as an argument to `SetThreadPriority`. (See the upcoming section "[Real-Time priorities](#)" for more information.)

Regardless of how the thread's priority came to be by using the Windows API (a combination of process priority class and a relative thread priority), from the point of view of the scheduler, only the final result matters. For example, priority level 10 can be obtained in two ways: a Normal priority class process (8) with a thread relative priority of Highest (+2), or an Above-Normal priority class process (10) and a Normal thread relative priority (0). From the scheduler's perspectives, these settings lead to the same value (10), so these threads are identical in terms of their priority.

Whereas a process has only a single base priority value, each thread has two priority values: current (dynamic) and base. Scheduling decisions are made based on the current priority. As explained in the upcoming section called "[Priority boosts](#)," under certain circumstances, the system increases the priority of threads in the dynamic range (1 through 15) for brief periods. Windows never adjusts the priority of threads in the Real-Time range (16 through 31), so they always have the same base and current priority.

A thread's initial base priority is inherited from the process base priority. A process, by default, inherits its base priority from the process that created it. You can override this behavior on the `Create-Process` function or by using the command-line `start` command. You can also change a process priority after it is created by using the `SetPriorityClass` function or by using various tools that

expose that function, such as Task Manager or Process Explorer. (Right-click on the process and choose a new priority class.) For example, you can lower the priority of a CPU-intensive process so that it does not interfere with normal system activities. Changing the priority of a process changes the thread priorities up or down, but their relative settings remain the same.

Normally, user applications and services start with a normal base priority, so their initial thread typically executes at priority level 8. However, some Windows system processes (such as the Session manager, Service Control Manager, and local security authentication process) have a base process priority slightly higher than the default for the Normal class (8). This higher default value ensures that the threads in these processes will all start at a higher priority than the default value of 8.

Real-Time priorities

You can raise or lower thread priorities within the dynamic range in any application. However, you must have the increase scheduling priority privilege (`SeIncreaseBasePriorityPrivilege`) to enter the Real-Time range. Be aware that many important Windows kernel-mode system threads run in the Real-Time priority range, so if threads spend excessive time running in this range, they might block critical system functions (such as in the memory manager, cache manager, or some device drivers).

Using the standard Windows APIs, once a process has entered the Real-Time range, all its threads (even Idle ones) must run at one of the Real-Time priority levels. It is thus impossible to mix real-time and dynamic threads within the same process through standard interfaces. This is because the `SetThreadPriority` API calls the native `NtSetInformationThread` API with the `ThreadBasePriority` information class, which allows priorities to remain only in the same range. Furthermore, this information class allows priority changes only in the recognized Windows API deltas of -2 to 2 (or Time-Critical/Idle) unless the request comes from CSRSS or another real-time process. In other words, this means that a real-time process can pick thread priorities anywhere between 16 and 31, even though the standard Windows API relative thread priorities would seem to limit its choices based on the table that was shown earlier.

As mentioned, calling `SetThreadPriority` with one of a set of special values causes a call to `NtSetInformationThread` with the `ThreadActualBasePriority` information class, the kernel base priority for the thread can be directly set, including in the dynamic range for a real-time process.



Note

The name *real-time* does not imply that Windows is a real-time OS in the common definition of the term. This is because Windows doesn't provide true, real-time OS facilities, such as guaranteed interrupt latency or a way for threads to obtain a guaranteed execution time. The term *real-time* really just means "higher than all the others."

Using tools to interact with priority

You can change (and view) the base-process priority with Task Manager and Process Explorer. You can kill individual threads in a process with Process Explorer (which should be done, of course, with extreme care).

You can view individual thread priorities with Performance Monitor, Process Explorer, or WinDbg. Although it might be useful to increase or decrease the priority of a process, it typically does not make sense to adjust individual thread priorities within a process because only a person who thoroughly

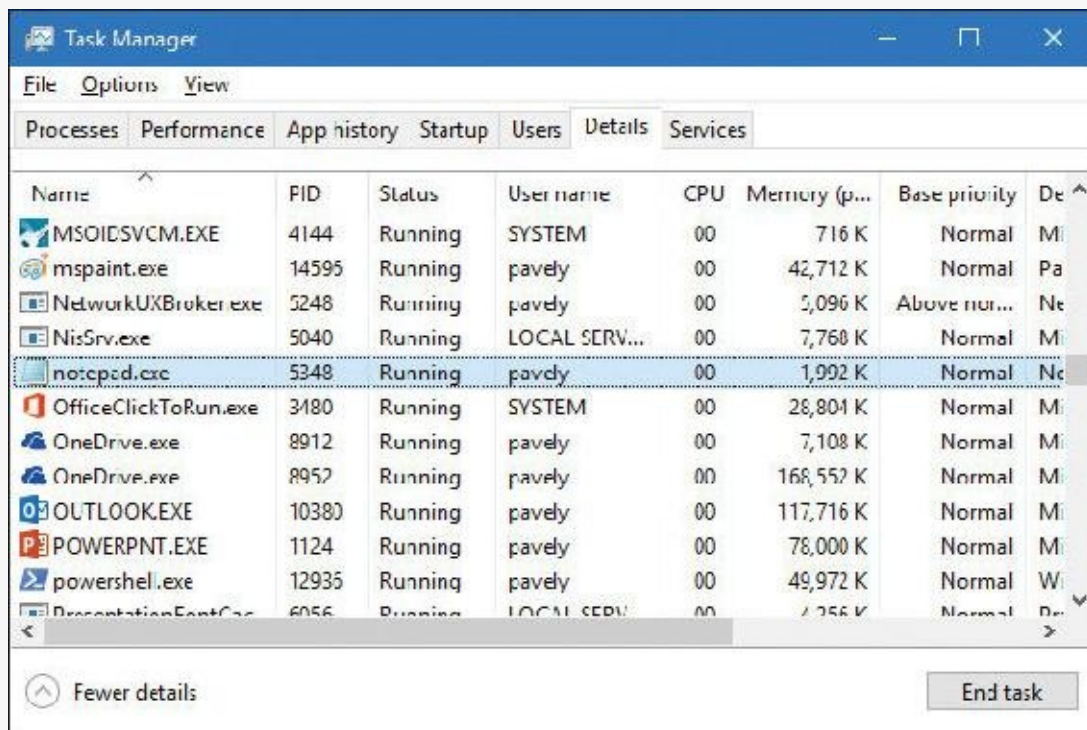
understands the program (in other words, the developer) would understand the relative importance of the threads within the process.

The only way to specify a starting priority class for a process is with the `start` command in the Windows command prompt. If you want to have a program start every time with a specific priority, you can define a shortcut to use the `start` command by beginning the command with `cmd /c`. This runs the command prompt, executes the command on the command line, and terminates the command prompt. For example, to run Notepad in the Idle-process priority, the command is `cmd /c start /low Notepad.exe`.

EXPERIMENT: Examining and specifying process and thread priorities

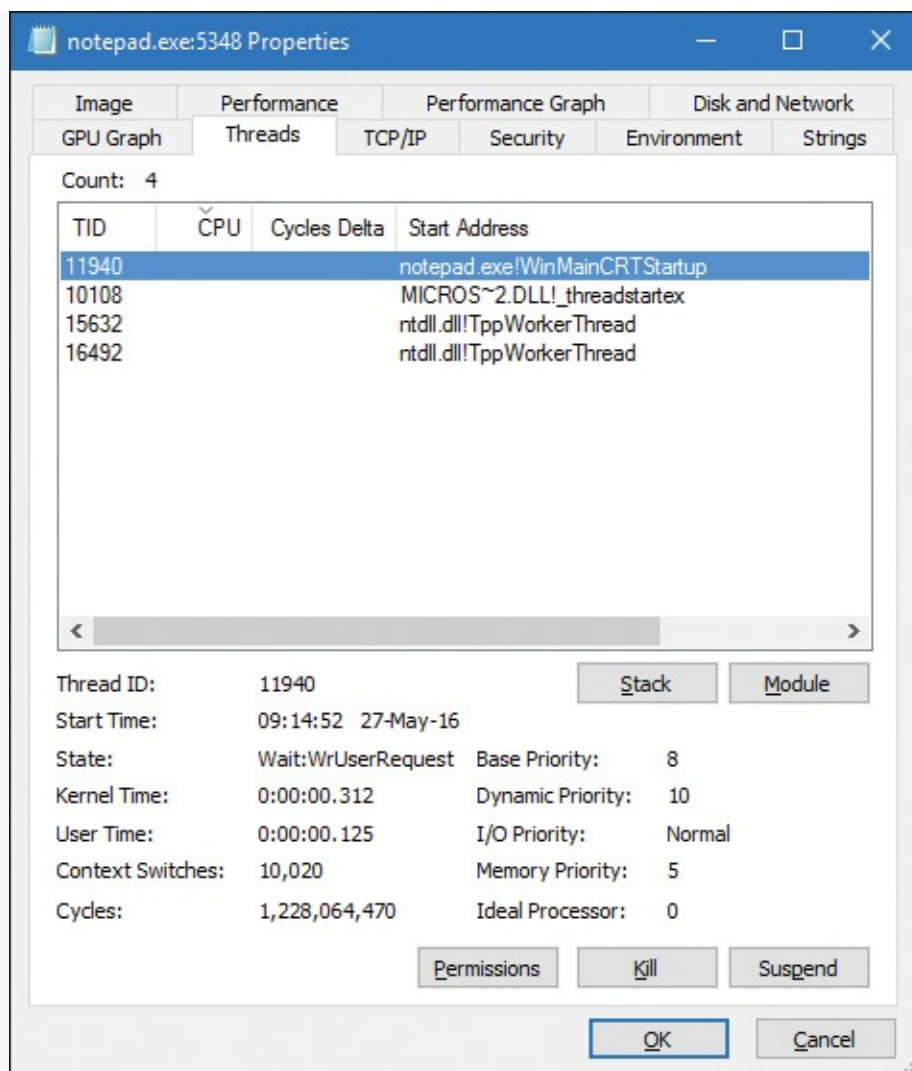
To examine and specify process and thread priorities, follow these steps:

1. Run notepad.exe normally—for example, by typing **Notepad** in a command window.
2. Open Task Manager and click to the **Details** tab.
3. Add a column named **Base Priority**. This is the name Task Manager uses for priority class.
4. Find Notepad in the list. You should see something like the following:

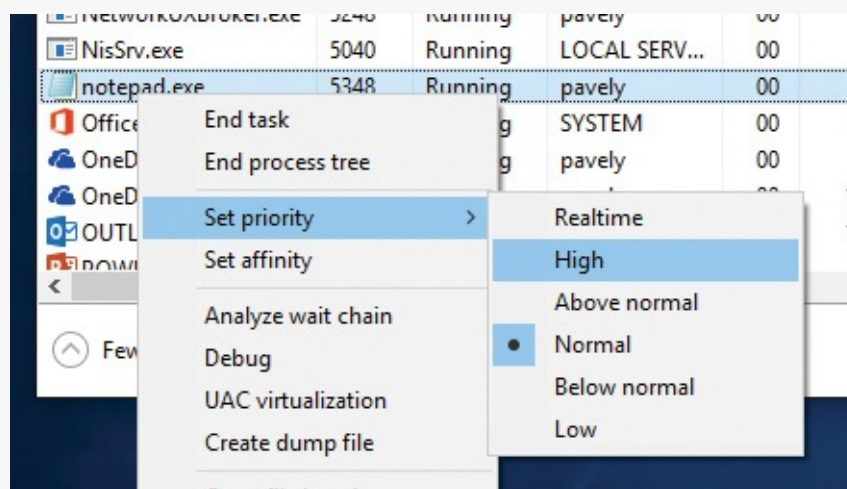


Name	PID	Status	User name	CPU	Memory (p...	Base priority	De ^
MSOISVCM.EXE	4144	Running	SYSTEM	00	716 K	Normal	Mi
mspaint.exe	14595	Running	pavely	00	42,712 K	Normal	Pa
NetworkUXBroker.exe	5248	Running	pavely	00	5,096 K	Above nur...	Ne
NisSrv.exe	5040	Running	LOCAL SERV...	00	7,768 K	Normal	Mi
notepad.exe	5348	Running	pavely	00	1,992 K	Normal	Ne
OfficeClickToRun.exe	3180	Running	SYSTEM	00	28,801 K	Normal	Mi
OneDrive.exe	8912	Running	pavely	00	7,108 K	Normal	Mi
OneDrive.exe	8952	Running	pavely	00	168,552 K	Normal	Mi
OUTLOOK.EXE	10380	Running	pavely	00	117,716 K	Normal	Mi
POWERPNT.EXE	1124	Running	pavely	00	78,000 K	Normal	Mi
powershell.exe	12935	Running	pavely	00	49,972 K	Normal	Wi
PresentationFontCap...	6056	Running	LOCAL SERV...	00	1,256 K	Normal	De

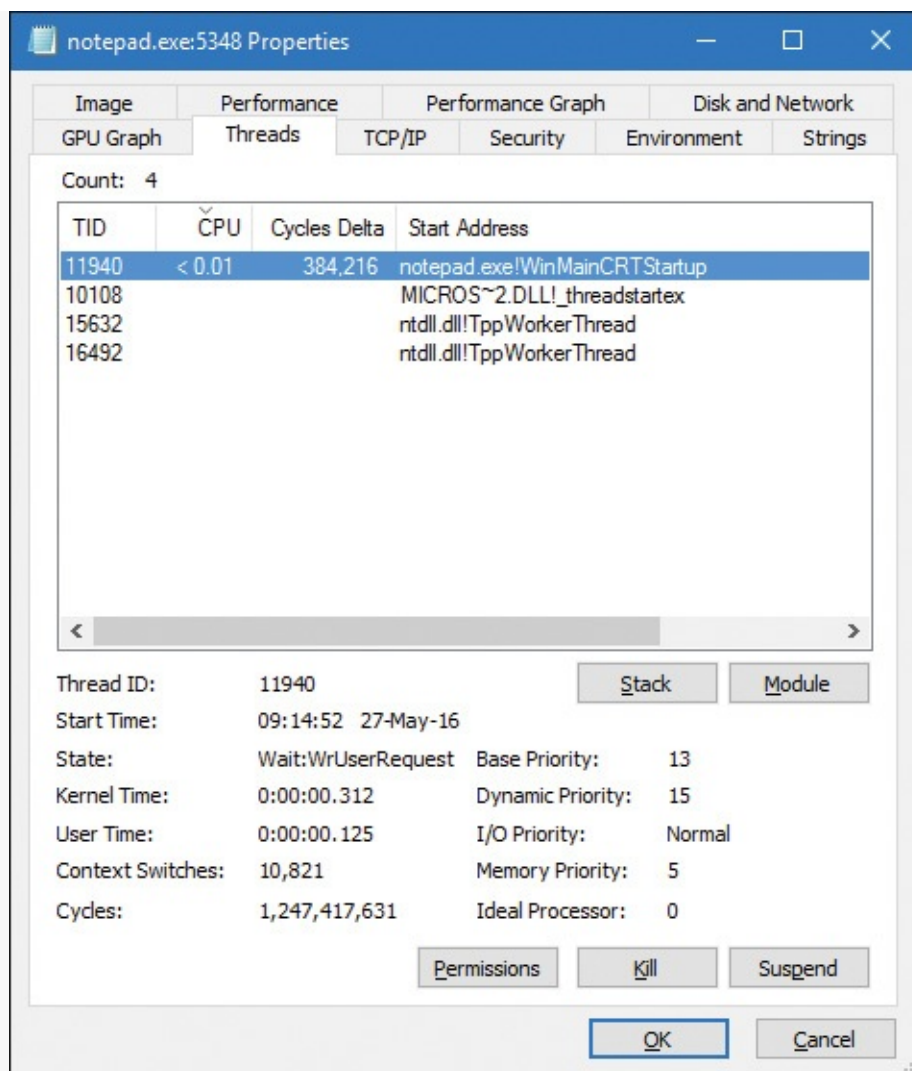
5. Notice the Notepad process running with the Normal priority class (8) and that Task Manager shows the Idle priority class as Low.
6. Open Process Explorer.
7. Double-click the **Notepad** process to show its Properties dialog box and click the **Threads** tab.
8. Select the first thread (if there's more than one). You should see something like this:



9. Notice the thread's priorities. Its base priority is 8 but its current (dynamic) priority is 10. (The reason for this priority boost is discussed in the upcoming "[Priority boosts](#)" section).
10. If you want, you can suspend and kill the thread. (Both operations must be used with caution, of course.)
11. In Task Manager, right-click the **Notepad** process, select **Set Priority**, and set the value to **High**, as shown here:



12. Accept the confirmation dialog box change and go back to Process Explorer. Notice that the thread's priority has jumped to the new base for High (13). The dynamic priority has made the same relative jump:



13. In Task Manager, change the priority class to **Realtime**. (You must be an administrator on the machine for this to succeed. Note that you can also make this change in Process Manager.)
14. In Process Manager, notice that the base and dynamic priorities of the thread are now 24. Recall that the kernel never applies priority boosts for threads in the Real-Time priority range.

notepad.exe:5348 Properties

Image Performance Performance Graph Disk and Network
GPU Graph Threads TCP/IP Security Environment Strings

Count: 4

TID	CPU	Cycles Delta	Start Address
11940	< 0.01	84,048	notepad.exe!WinMainCRTStartup
10108			MICROS~2.DLL!_threadstartex
15632			ntdll.dll!TppWorkerThread
16492			ntdll.dll!TppWorkerThread

Thread ID: 11940 Stack Module

Start Time: 09:14:52 27-May-16

State: Wait:WrUserRequest Base Priority: 24

Kernel Time: 0:00:00.312 Dynamic Priority: 24

User Time: 0:00:00.125 I/O Priority: Normal

Context Switches: 11,902 Memory Priority: 5

Cycles: 1,277,393,678 Ideal Processor: 0

Permissions Kill Suspend

OK Cancel

Windows System Resource Manager

Windows Server 2012 R2 Standard Edition and higher SKUs include an optionally installable component called *Windows System Resource Manager* (WSRM). It permits the administrator to configure policies that specify CPU utilization, affinity settings, and memory limits (both physical and virtual) for processes. In addition, WSRM can generate resource-utilization reports that can be used for accounting and verification of service-level agreements with users.

Policies can be applied for specific applications (by matching the name of the image with or without specific command-line arguments), users, or groups. The policies can be scheduled to take effect at certain periods or can be enabled all the time.

After you set a resource-allocation policy to manage specific processes, the WSRM service monitors CPU consumption of managed processes and adjusts process base priorities when those processes do not meet their target CPU allocations.

The physical memory limitation uses the function `SetProcessWorkingSetSizeEx` to set a hard-working set maximum. The virtual memory limit is implemented by the service checking the private virtual memory consumed by the processes. (See [Chapter 5](#) for an explanation of these memory limits.) If this limit is exceeded, WSRM can be configured to either kill the processes or write an entry to the event log. This behavior can be used to detect a process with a memory leak before it consumes all the available committed memory on the system. Note that WSRM memory limits do not apply to Address Windowing Extensions (AWE) memory, large page memory, or kernel memory (non-paged or paged pool). (See [Chapter 5](#) for more information on these terms.)

Thread states

Before looking at the thread-scheduling algorithms, you must understand the various execution states that a thread can be in. The thread states are as follows:

- **Ready** A thread in the ready state is waiting to execute or to be in-swapped after completing a wait. When looking for a thread to execute, the dispatcher considers only the threads in the ready state.
- **Deferred ready** This state is used for threads that have been selected to run on a specific processor but have not actually started running there. This state exists so that the kernel can minimize the amount of time the per-processor lock on the scheduling database is held.
- **Standby** A thread in this state has been selected to run next on a particular processor. When the correct conditions exist, the dispatcher performs a context switch to this thread. Only one thread can be in the standby state for each processor on the system. Note that a thread can be preempted out of the standby state before it ever executes (if, for example, a higher-priority thread becomes runnable before the standby thread begins execution).
- **Running** After the dispatcher performs a context switch to a thread, the thread enters the running state and executes. The thread's execution continues until its quantum ends (and another thread at the same priority is ready to run), it is preempted by a higher-priority thread, it terminates, it yields execution, or it voluntarily enters the waiting state.
- **Waiting** A thread can enter the waiting state in several ways: A thread can voluntarily wait for an object to synchronize its execution, the OS can wait on the thread's behalf (such as to resolve a paging I/O), or an environment subsystem can direct the thread to suspend itself. When the thread's wait ends, depending on its priority, the thread either begins running immediately or is moved back to the ready state.

- **Transition** A thread enters the transition state if it is ready for execution but its kernel stack is paged out of memory. After its kernel stack is brought back into memory, the thread enters the ready state. (Thread stacks are discussed in [Chapter 5](#).)
- **Terminated** When a thread finishes executing, it enters this state. After the thread is terminated, the executive thread object (the data structure in system memory that describes the thread) might or might not be deallocated. The object manager sets the policy regarding when to delete the object. For example, the object remains if there are any open handles to the thread. A thread can also enter the terminated state from other states if it's killed explicitly by some other thread—for example, by calling the `TerminateThread` Windows API.
- **Initialized** This state is used internally while a thread is being created.

[Figure 4-8](#) shows the main state transitions for threads. The numeric values shown represent the internal values of each state and can be viewed with a tool such as Performance Monitor. The ready and deferred ready states are represented as one. This reflects the fact that the deferred ready state acts as a temporary placeholder for the scheduling routines. This is true for the standby state as well. These states are almost always very short-lived. Threads in these states always transition quickly to ready, running, or waiting.

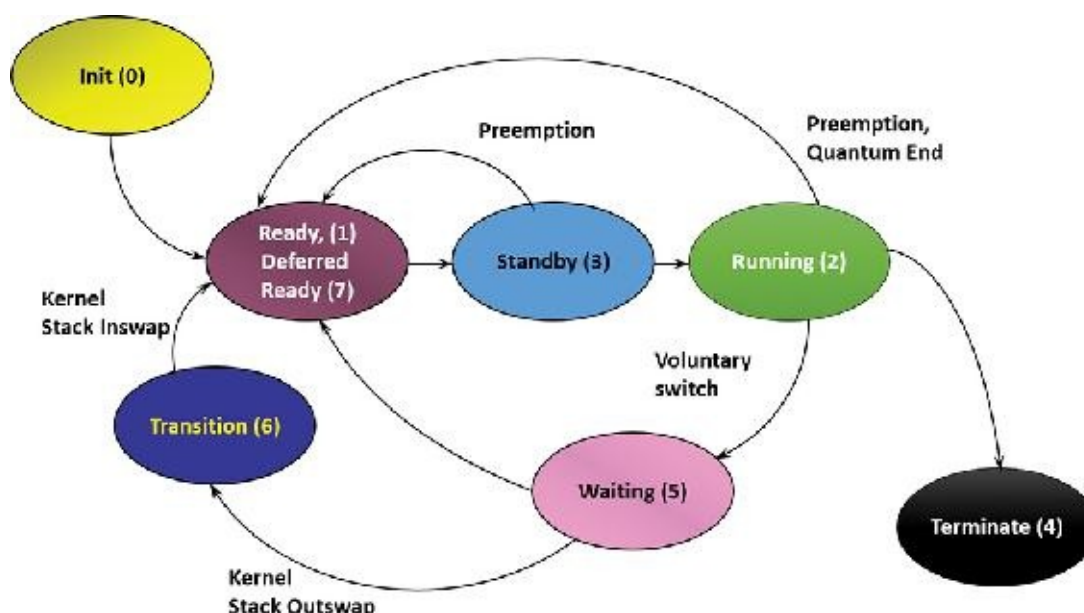


FIGURE 4-8 Thread states and transitions.

EXPERIMENT: Thread-scheduling state changes

You can watch thread-scheduling state changes with the Performance Monitor tool in Windows. This utility can be useful when you're debugging a multithreaded application and you're unsure about the state of the threads running in the process. To watch thread-scheduling state changes by using the Performance Monitor tool, follow these steps:

1. Download the CPU Stress tool from the book's downloadable resources.
2. Run `CPUSTRES.exe`. Thread 1 should be active.
3. Activate thread 2 by selecting it in the list and clicking the **Activate** button or by right-clicking it and selecting **Activate** from the context menu. The tool should look something like this:

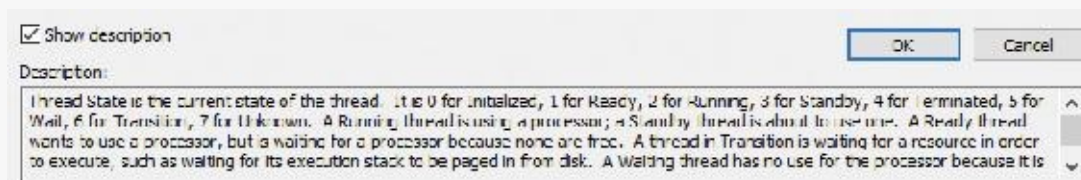
CPU Stress - Sysinternals www.sysinternals.com

File Process Thread Options Help

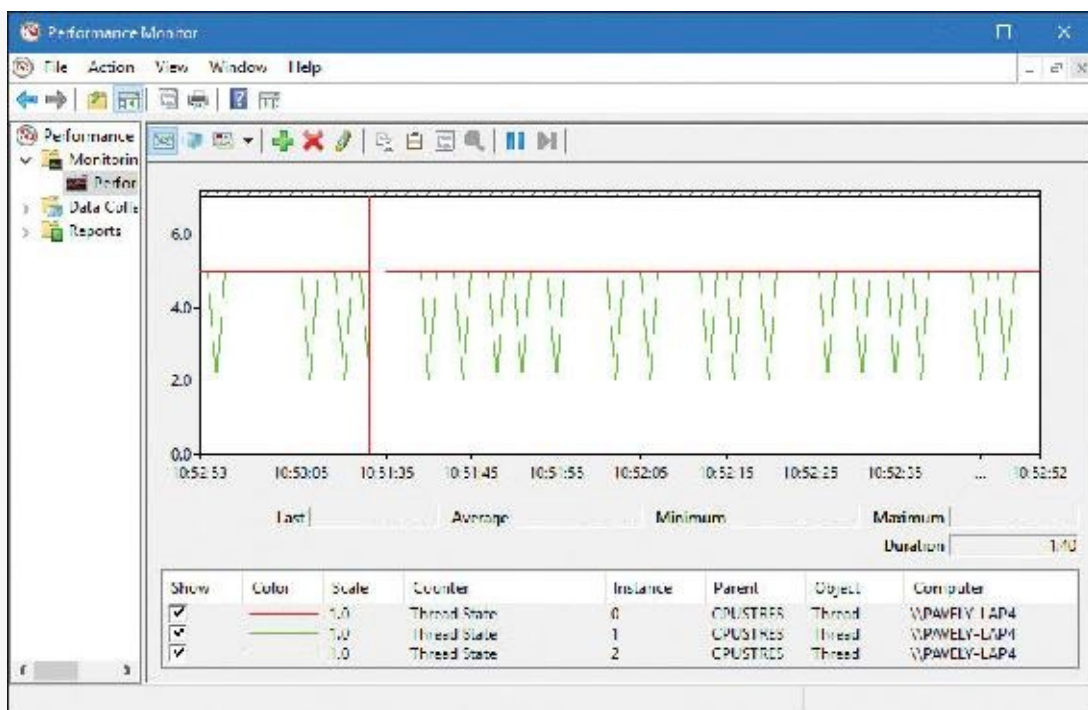
Process Affinity: 11111111
Process Priority Class: Normal

#	ID	Active?	Activity	Priority	Ideal CPU	Activity
1	2708	Yes	Low	Normal	3	11111111
2	14852	Yes	Low	Normal	5	11111111
3	16848		Low	Normal	7	11111111
4	14084		Low	Normal	0	11111111

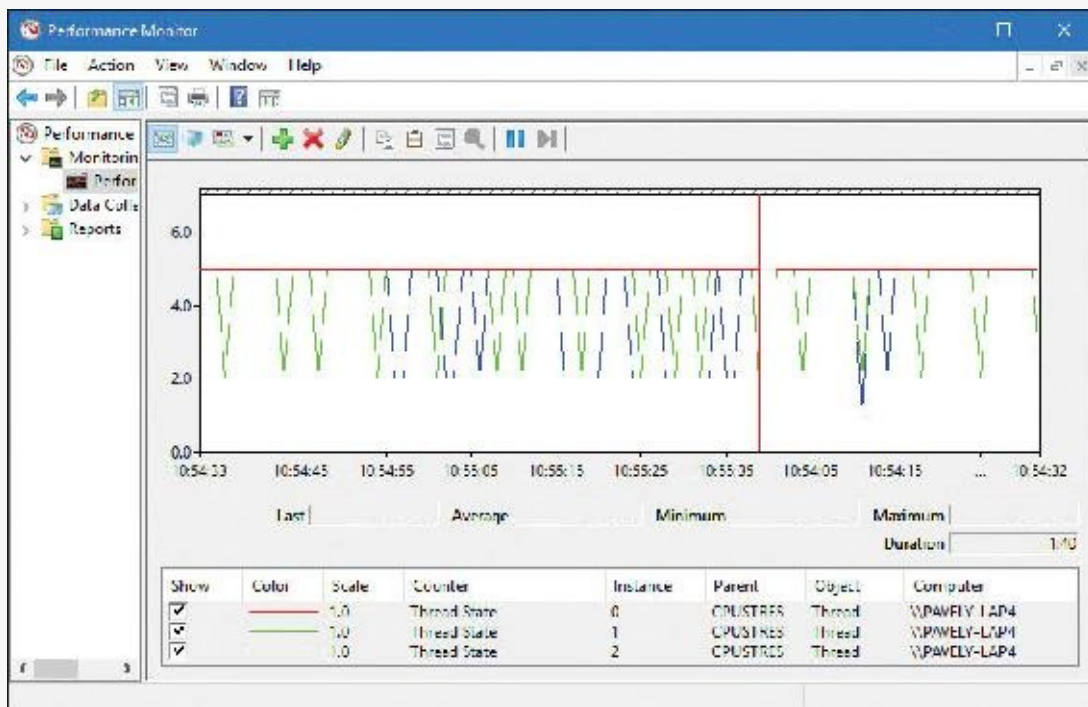
4. Click the **Start** button and type **perfmon** to start the Performance Monitor tool.
5. If necessary, select the chart view. Then remove the existing CPU counter.
6. Right-click the graph and choose **Properties**.
7. Click the **Graph** tab and change the chart vertical scale maximum to 7. (As you saw in [Figure 4-8](#), the various states are associated with numbers 0 through 7.) Then click **OK**.
8. Click the **Add** button on the toolbar to open the Add Counters dialog box.
9. Select the **Thread** performance object and then select the **Thread State** counter.
10. Select the **Show Description** check box to see the definition of the values:



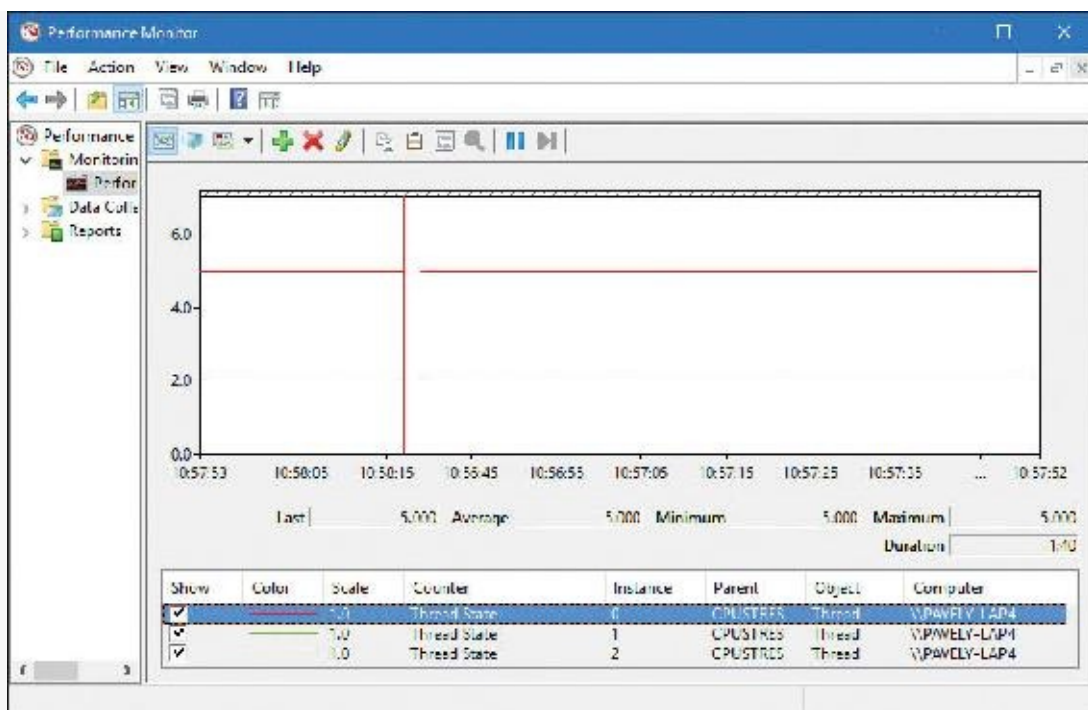
11. In the Instances box, select **<All instances>**. Then type **cpustres** and click **Search**.
12. Select the first three threads of cpustres (**cpustres/0**, **cpustres/1**, and **cpustres/2**) and click the **Add >>** button. Then click **OK**. Thread 0 should be in state 5 (waiting), because that's the GUI thread and it's waiting for user input. Threads 1 and 2 should be alternating between states 2 and 5 (running and waiting). (Thread 1 may be hiding thread 2 as they're running with the same activity level and the same priority.)



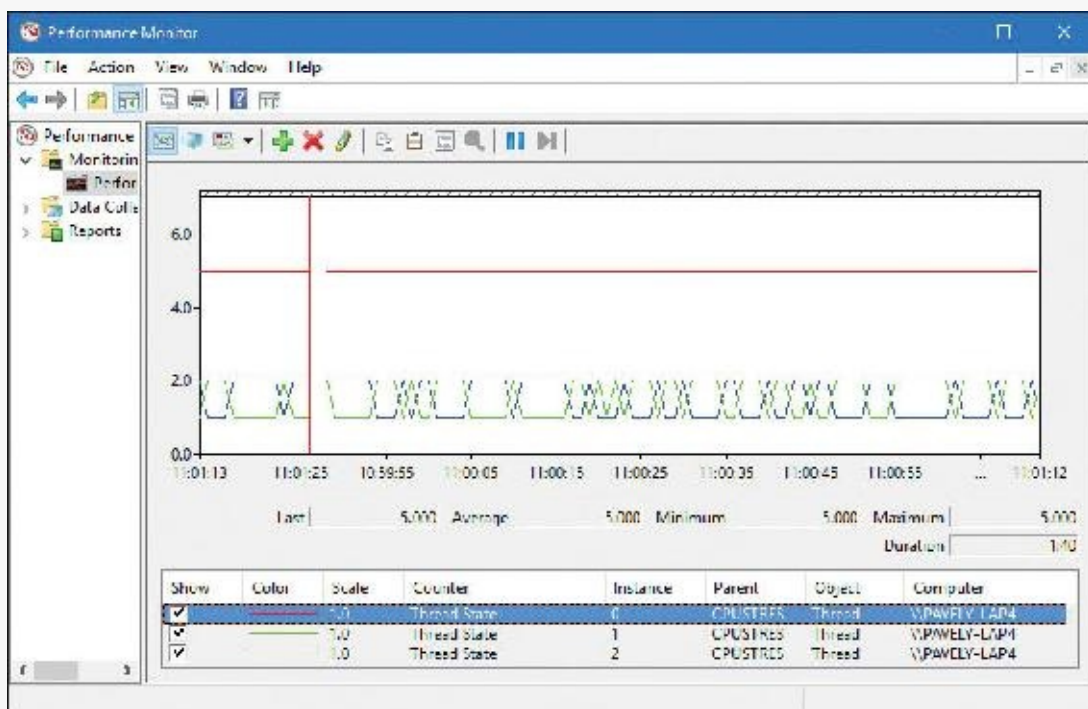
13. Go back to CPU Stress, right-click thread 2, and choose **Busy** from the activity context menu. You should see thread 2 in state 2 (running) more often than thread 1:



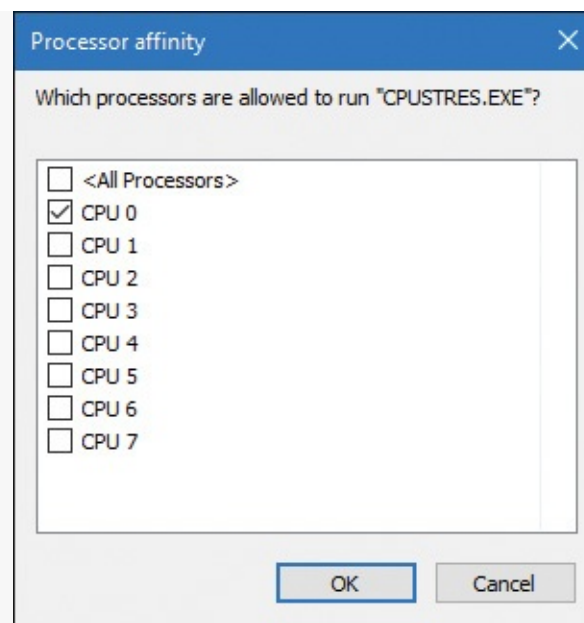
14. Right-click thread 1 and choose an activity level of **Maximum**. Then repeat this step for thread 2. Both threads now should be constantly in state 2 because they're running essentially an infinite loop:



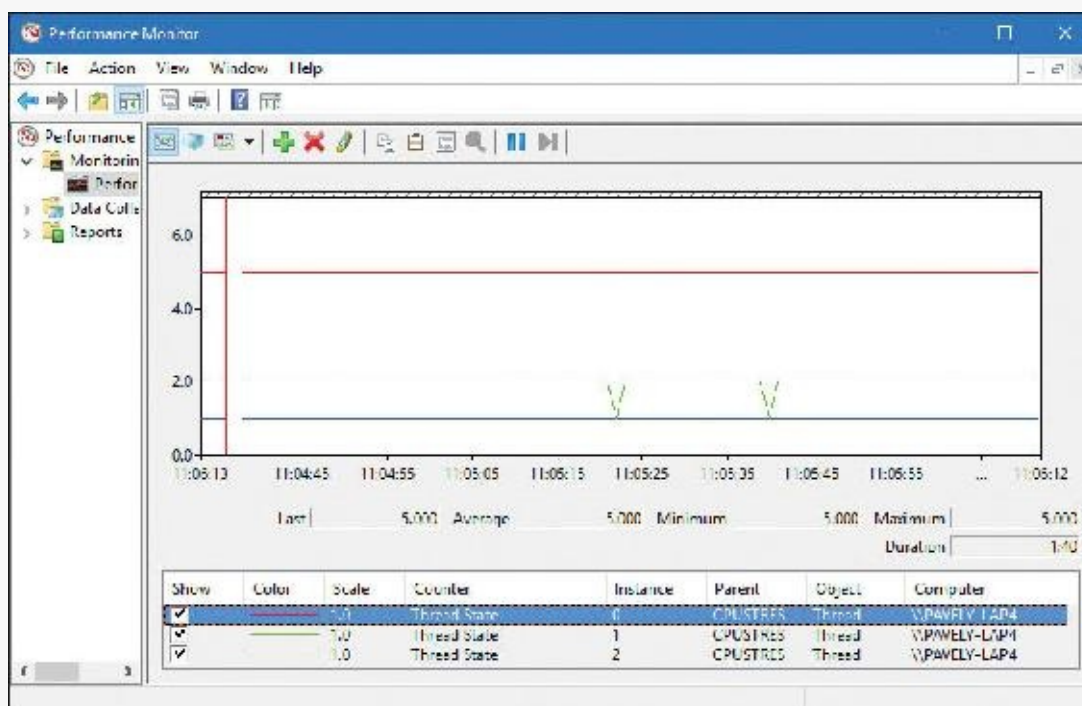
If you're trying this on a single processor system, you'll see something different. Because there is only one processor, only one thread can execute at a time, so you'll see the two threads alternating between states 1 (ready) and 2 (running):



15. If you're on a multiprocessor system (very likely), you can get the same effect by going to Task Manager, right-clicking the CPUSTRES process, selecting **Set Affinity**, and then select just one processor—it doesn't matter which one—as shown here. (You can also do it from CPU Stress by opening the Process menu and selecting **Affinity**.)



16. There's one more thing you can try. With this setting in place, go back to CPU Stress, right-click thread 1, and choose a priority of **Above Normal**. You'll see that thread 1 is running continuously (state 2) and thread 2 is always in the ready state (state 1). This is because there's only one processor, so in general, the higher priority thread wins out. From time to time, however, you'll see a change in thread 1's state to ready. This is because every 4 seconds or so, the starved thread gets a boost that enables it to run for a little while. (Often, this state change is not reflected by the graph because the granularity of Performance Monitor is limited to 1 second, which is too coarse.) This is described in more detail later in this chapter in the section "[Priority boosts](#)."



Dispatcher database

To make thread-scheduling decisions, the kernel maintains a set of data structures known collectively as the *dispatcher database*. The dispatcher database keeps track of which threads are waiting to execute and which processors are executing which threads.

To improve scalability, including thread-dispatching concurrency, Windows multiprocessor systems

have per-processor dispatcher ready queues and shared processor group queues, as illustrated in [Figure 4-9](#). In this way, each CPU can check its own shared ready queue for the next thread to run without having to lock the system-wide ready queues.

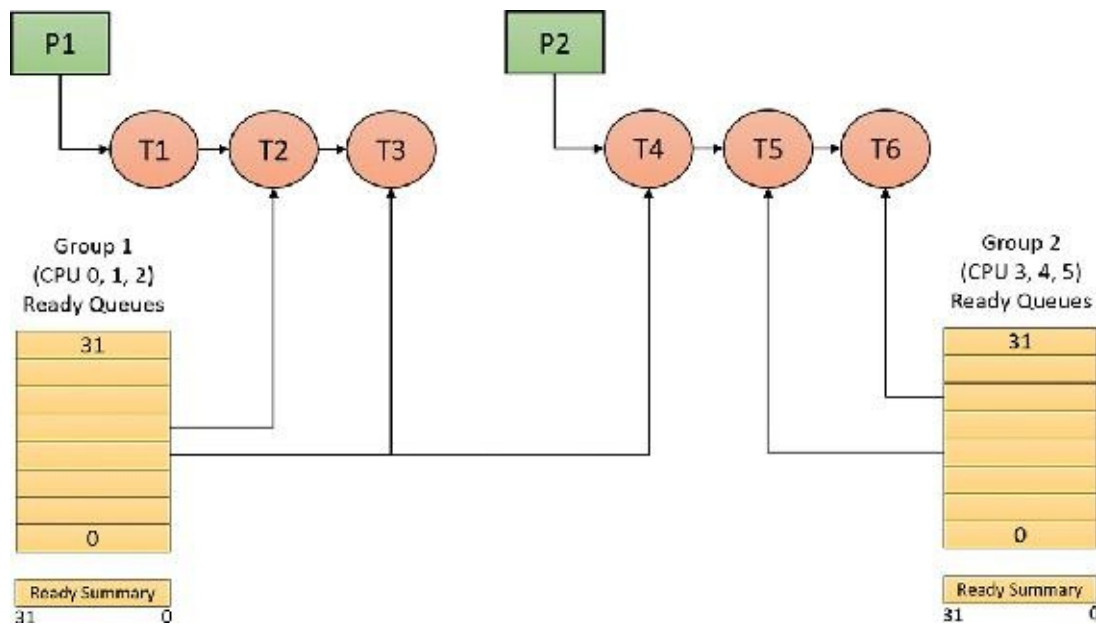


FIGURE 4-9 Windows multiprocessor dispatcher database. (This example shows six processors. *P* represents processes; *T* represents threads.)

Windows versions prior to Windows 8 and Windows Server 2012 used per-processor ready queues and a per-processor ready summary, which were stored as part of processor control block (PRCB) structure. (To see the fields in the PRCB, type `dt nt!_kprcb` in the kernel debugger.) Starting with Windows 8 and Windows Server 2012, a shared ready queue and ready summary are used for a group of processors. This enables the system to make better decisions about which processor to use next for that group of processors. (The per-CPU ready queues are still there and used for threads with affinity constraints.)



Note

Because the shared data structure must be protected (by a spinlock), the group should not be too large. That way, contention on the queues is insignificant. In the current implementation, the maximum group size is four logical processors. If the number of logical processors is greater than four, then more than one group would be created, and the available processors spread evenly. For example, on a six-processor system, two groups of three processors each would be created.

The ready queues, ready summary (described next), and some other information is stored in a kernel structure named `KSHARED_READY_QUEUE` that is stored in the PRCB. Although it exists for every processor, it's used only on the first processor of each processor group, sharing it with the rest of the processors in that group.

The dispatcher ready queues (`ReadListHead` in `KSHARED_READY_QUEUE`) contain the threads that are in the ready state, waiting to be scheduled for execution. There is one queue for each of the 32 priority levels. To speed up the selection of which thread to run or preempt, Windows maintains a 32-bit bitmask called the *ready summary* (`ReadySummary`). Each bit set indicates one or more threads in the ready queue for that priority level (bit 0 represents priority 0, bit 1 priority 1, and so on).

Instead of scanning each ready list to see whether it is empty or not (which would make scheduling decisions dependent on the number of different priority threads), a single bit scan is performed as a native processor command to find the highest bit set. Regardless of the number of threads in the ready queue, this operation takes a constant amount of time.

The dispatcher database is synchronized by raising IRQL to DISPATCH_LEVEL (2). (For an explanation of interrupt priority levels, or IRQLs, see [Chapter 6](#).) Raising IRQL in this way prevents other threads from interrupting thread dispatching on the processor because threads normally run at IRQL 0 or 1. However, more is required than just raising IRQL, because other processors can simultaneously raise to the same IRQL and attempt to operate on their dispatcher database. How Windows synchronizes access to the dispatcher database is explained later in this chapter in the section “[Multiprocessor systems](#).”

EXPERIMENT: Viewing ready threads

You can view the list of ready threads with the kernel-debugger `!ready` command. This command displays the thread or list of threads that are ready to run at each priority level. Here is an example generated on a 32-bit machine with four logical processors:

[Click here to view code image](#)

```
0: kd> !ready
KSHARED_READY_QUEUE 8147e800: (00) ****-----
SharedReadyQueue 8147e800: Ready Threads at priority 8
    THREAD 80af8bc0  Cid 1300.15c4  Teb: 7ffdb000 Win32Thread:
00000000 READY on
processor 80000002
    THREAD 80b58bc0  Cid 0454.0fc0  Teb: 7f82e000 Win32Thread:
00000000 READY on
processor 80000003
SharedReadyQueue 8147e800: Ready Threads at priority 7
    THREAD a24b4700  Cid 0004.11dc  Teb: 00000000 Win32Thread:
00000000 READY on
processor 80000001
    THREAD a1bad040  Cid 0004.096c  Teb: 00000000 Win32Thread:
00000000 READY on
processor 80000001
SharedReadyQueue 8147e800: Ready Threads at priority 6
    THREAD a1bad4c0  Cid 0004.0950  Teb: 00000000 Win32Thread:
00000000 READY on
processor 80000002
    THREAD 80b5e040  Cid 0574.12a4  Teb: 7fc33000 Win32Thread:
00000000 READY on
processor 80000000
SharedReadyQueue 8147e800: Ready Threads at priority 4
    THREAD 80b09bc0  Cid 0004.12dc  Teb: 00000000 Win32Thread:
00000000 READY on
processor 80000003
SharedReadyQueue 8147e800: Ready Threads at priority 0
    THREAD 82889bc0  Cid 0004.0008  Teb: 00000000 Win32Thread:
```

```
00000000 READY on
processor 80000000
Processor 0: No threads in READY state
Processor 1: No threads in READY state
Processor 2: No threads in READY state
Processor 3: No threads in READY state
```

The processor numbers have a 0x80000000 added to them, but the actual processor numbers are easy to see. The first line shows the address of the KSHARED_READY_QUEUE with the group number in parentheses (00 in the output) and then a graphic representation of the processors in this particular group (the four asterisks).

The last four lines seem odd, as they appear to indicate no ready threads, contradicting the preceding output. These lines indicate ready threads from the older DispatcherReadyListHead member of the PRCB because the per-processor ready queues are used for threads that have restrictive affinity (set to run on a subset of processors inside that processor group).

You can also dump the KSHARED_READY_QUEUE with the address given by the !ready command:

[Click here to view code image](#)

```
0: kd> dt nt!_KSHARED_READY_QUEUE 8147e800
+0x000 Lock                : 0
+0x004 ReadySummary        : 0x1d1
+0x008 ReadyListHead       : [32] _LIST_ENTRY [ 0x82889c5c -
0x82889c5c ]
+0x108 RunningSummary      : [32] "???"
+0x128 Span                : 4
+0x12c LowProcIndex        : 0
+0x130 QueueIndex          : 1
+0x134 ProcCount           : 4
+0x138 Affinity            : 0xf
```

The ProcCount member shows the processor count in the shared group (4 in this example). Also note the ReadySummary value, 0x1d1. This translates to 111010001 in binary. Reading the binary one bits from right to left, this indicates that threads exist in priorities 0, 4, 6, 7, 8, which match the preceding output.

Quantum

As mentioned earlier in the chapter, a *quantum* is the amount of time a thread is permitted to run before Windows checks to see whether another thread at the same priority is waiting to run. If a thread completes its quantum and there are no other threads at its priority, Windows permits the thread to run for another quantum.

On client versions of Windows, threads run for two clock intervals by default. On server systems, threads runs for 12 clock intervals by default. (We'll explain how to change these values in the "[Controlling the quantum](#)" section.) The rationale for the longer default value on server systems is to minimize context switching. By having a longer quantum, server applications that wake up because of a client request have a better chance of completing the request and going back into a wait state before their

quantum ends.

The length of the clock interval varies according to the hardware platform. The frequency of the clock interrupts is up to the HAL, not the kernel. For example, the clock interval for most x86 uniprocessors is about 10 milliseconds (note that these machines are no longer supported by Windows and are used here only for example purposes), and for most x86 and x64 multiprocessors it is about 15 milliseconds. This clock interval is stored in the kernel variable `KeMaximumIncrement` as hundreds of nanoseconds.

Although threads run in units of clock intervals, the system does not use the count of clock ticks as the gauge for how long a thread has run and whether its quantum has expired. This is because thread run-time accounting is based on processor cycles. When the system starts up, it multiplies the processor speed (CPU clock cycles per second) in hertz (Hz) by the number of seconds it takes for one clock tick to fire (based on the `KeMaximumIncrement` value described earlier) to calculate the number of clock cycles to which each quantum is equivalent. This value is stored in the kernel variable `KiCyclesPerClockQuantum`.

The result of this accounting method is that threads do not actually run for a quantum number based on clock ticks. Instead, they run for a quantum target, which represents an estimate of what the number of CPU clock cycles the thread has consumed should be when its turn would be given up. This target should be equal to an equivalent number of clock interval timer ticks. This is because, as you just saw, the calculation of clock cycles per quantum is based on the clock interval timer frequency, which you can check using the following experiment. Note, however, that because interrupt cycles are not charged to the thread, the actual clock time might be longer.

EXPERIMENT: Determining the clock interval frequency

The Windows `GetSystemTimeAdjustment` function returns the clock interval. To determine the clock interval, run the `clockres` tool from Sysinternals. Here's the output from a quad-core 64-bit Windows 10 system:

[Click here to view code image](#)

```
C:\>clockres
```

```
ClockRes v2.0 - View the system clock resolution  
Copyright (C) 2009 Mark Russinovich  
SysInternals - www.sysinternals.com
```

```
Maximum timer interval: 15.600 ms  
Minimum timer interval: 0.500 ms  
Current timer interval: 1.000 ms
```

The current interval may be lower than the maximum (default) clock interval because of multimedia timers. Multimedia timers are used with functions such as `timeBeginPeriod` and `timeSetEvent` that are used to receive callbacks with intervals of 1 millisecond (ms) at best. This causes a global reprogramming of the kernel interval timer, meaning the scheduler wakes up in more frequent intervals, which can degrade system performance. In any case, this does not affect quantum lengths, as described in the next section.

It's also possible to read the value using the kernel global variable `KeMaximumIncrement` as shown here (not the same system as the previous example):

[Click here to view code image](#)

```
0: kd> dd nt!KeMaximumIncrement L1  
814973b4 0002625a  
0: kd> ? 0002625a  
Evaluate expression: 156250 = 0002625a
```

This corresponds to the default of 15.6 ms.

Quantum accounting

Each process has a quantum reset value in the process control block (`KPROCESS`). This value is used when creating new threads inside the process and is duplicated in the thread control block (`KTHREAD`), which is then used when giving a thread a new quantum target. The quantum reset value is stored in terms of actual quantum units (we'll discuss what these mean soon), which are then multiplied by the number of clock cycles per quantum, resulting in the quantum target.

As a thread runs, CPU clock cycles are charged at different events, such as context switches, interrupts, and certain scheduling decisions. If, at a clock interval timer interrupt, the number of CPU clock cycles charged has reached (or passed) the quantum target, quantum end processing is triggered. If there is another thread at the same priority waiting to run, a context switch occurs to the next thread in the ready queue.

Internally, a quantum unit is represented as one-third of a clock tick. That is, one clock tick equals three quanta. This means that on client Windows systems, threads have a quantum reset value of 6 ($2 * 3$) and

that server systems have a quantum reset value of 36 ($12 * 3$) by default. For this reason, the `KiCyclesPerClockQuantum` value is divided by 3 at the end of the calculation previously described, because the original value describes only CPU clock cycles per clock interval timer tick.

The reason a quantum was stored internally as a fraction of a clock tick rather than as an entire tick was to allow for partial quantum decay-on-wait completion on versions of Windows prior to Windows Vista. Prior versions used the clock interval timer for quantum expiration. If this adjustment had not been made, it would have been possible for threads to never have their quantum reduced. For example, if a thread ran, entered a wait state, ran again, and entered another wait state but was never the currently running thread when the clock interval timer fired, it would never have its quantum charged for the time it was running. Because threads now have CPU clock cycles charged instead of quantum, and because this no longer depends on the clock interval timer, these adjustments are not required.

EXPERIMENT: Determining the clock cycles per quantum

Windows doesn't expose the number of clock cycles per quantum through any function. However, with the calculation and description we've given, you should be able to determine this on your own using the following steps and a kernel debugger such as WinDbg in local debugging mode:

1. Obtain your processor frequency as Windows has detected it. You can use the value stored in the PRCB's MHz field, which you can display with the `!cpuinfo` command. Here is a sample output of a four-processor system running at 2794 megahertz (MHz):

[Click here to view code image](#)

```
lkd> !cpuinfo
```

CP	F/M/S	Manufacturer	MHz	PRCB	Signature	MSR	8B	Signature
0	6,60,3	GenuineIntel	2794	ffffffff	00000000			
>ffffffff00000000<a3cd3fff								
1	6,60,3	GenuineIntel	2794	ffffffff	00000000			a3cd3fff
2	6,60,3	GenuineIntel	2794	ffffffff	00000000			a3cd3fff
3	6,60,3	GenuineIntel	2794	ffffffff	00000000			a3cd3fff

2. Convert the number to hertz (Hz). This is the number of CPU clock cycles that occur each second on your system—in this case, 2,794,000,000 cycles per second.
3. Obtain the clock interval on your system by using `clockres`. This measures how long it takes before the clock fires. On the sample system used here, this interval was 15.625 msec.
4. Convert this number to the number of times the clock interval timer fires each second. One second equals 1,000 ms, so divide the number derived in step 3 by 1,000. In this case, the timer fires every 0.015625 seconds.
5. Multiply this count by the number of cycles each second that you obtained in step 2. In this case, 43,656,250 cycles have elapsed after each clock interval.
6. Remember that each quantum unit is one-third of a clock interval, so divide the number of cycles by 3. This gives you 14,528,083, or 0xDE0C13 in hexadecimal. This is the number of clock cycles each quantum unit should take on a system running at 2,794 MHz with a clock interval of around 15.6 ms.
7. To verify your calculation, dump the value of `KiCyclesPerClockQuantum` on your system. It should match (or be close enough because of rounding errors).

[Click here to view code image](#)

```
lkd> dd nt!KiCyclesPerClockQuantum L1
8149755c  00de0c10
```

Controlling the quantum

You can change the thread quantum for all processes, but you can choose only one of two settings: short (two clock ticks, which is the default for client machines) or long (12 clock ticks, which is the default for server systems).



Note

By using the job object on a system running with long quanta, you can select other quantum values for the processes in the job.

To change this setting, right-click the **This PC** icon on the desktop. Alternatively, in Windows Explorer, choose **Properties**, click the **Advanced System Settings** label, click the **Advanced** tab, click the **Settings** button in the **Performance** section, and click yet another **Advanced** tab. [Figure 4-10](#) shows the resulting dialog box.

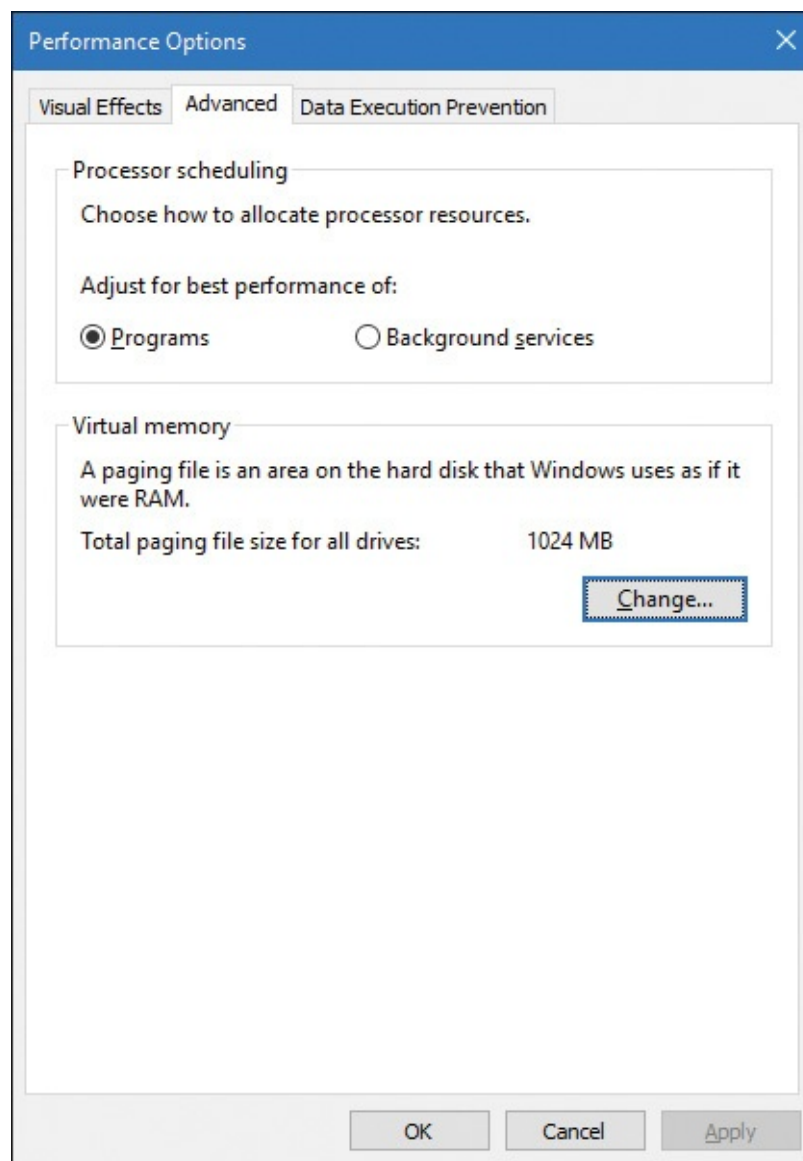


FIGURE 4-10 Quantum configuration in the Performance Options dialog box.

This dialog box contains two key options:

- **Programs** This setting designates the use of short, variable quanta, which is the default for client versions of Windows (and other client-like versions, such as mobile, XBOX, HoloLens, and so on).

If you install Terminal Services on a server system and configure the server as an application server, this setting is selected so that the users on the terminal server have the same quantum settings that would normally be set on a desktop or client system. You might also select this manually if you were running Windows Server as your desktop OS.

- **Background Services** This setting designates the use of long, fixed quanta—the default for server systems. The only reason you might select this option on a workstation system is if you were using the workstation as a server system. However, because changes in this option take effect immediately, it might make sense to use it if the machine is about to run a background or server-style workload. For example, if a long-running computation, encoding, or modeling simulation needs to run overnight, you could select the Background Services option at night and return the system to Programs mode in the morning.

Variable quanta

When variable quanta are enabled, the variable quantum table (`PspVariableQuantums`), which holds an array of six quantum numbers, is loaded into the `PspForegroundQuantum` table (a three-element array) that is used by the `PspComputeQuantum` function. Its algorithm will pick the appropriate quantum index based on whether the process is a foreground process—that is, whether it contains the thread that owns the foreground window on the desktop. If this is not the case, an index of 0 is chosen, which corresponds to the default thread quantum described earlier. If it is a foreground process, the quantum index corresponds to the priority separation.

This priority separation value determines the priority boost (described in the upcoming section “[Priority boosts](#)”) that the scheduler will apply to foreground threads, and it is thus paired with an appropriate extension of the quantum. For each extra priority level (up to 2), another quantum is given to the thread. For example, if the thread receives a boost of one priority level, it receives an extra quantum as well. By default, Windows sets the maximum possible priority boost to foreground threads, meaning that the priority separation will be 2, which means quantum index 2 is selected in the variable quantum table. This leads to the thread receiving two extra quanta, for a total of three quanta.

[Table 4-2](#) describes the exact quantum value (recall that this is stored in a unit representing one-third of a clock tick) that will be selected based on the quantum index and which quantum configuration is in use.

	Short Quantum Index			Long Quantum Index		
Variable	6	12	18	12	24	36
Fixed	18	18	18	36	36	36

TABLE 4-2 Quantum values

Thus, when a window is brought into the foreground on a client system, all the threads in the process containing the thread that owns the foreground window have their quanta tripled. Threads in the foreground process run with a quantum of six clock ticks, whereas threads in other processes have the default client quantum of two clock ticks. In this way, when you switch away from a CPU-intensive process, the new foreground process will get proportionally more of the CPU. This is because when its threads run, they will have a longer turn than background threads (again, assuming the thread priorities are the same in both the foreground and background processes).

Quantum settings registry value

The user interface that controls quantum settings described earlier modifies the registry value Win32-PrioritySeparation in the key HKLM\SYSTEM\CurrentControlSet\Control\PriorityControl. In addition to specifying the relative length of thread quantums (short or long), this registry value also defines whether variable quantums should be used, as well as the priority separation (which, as you’ve seen, will determine the quantum index used when variable quantums are enabled). This value consists of 6 bits divided into the three 2-bit fields shown in [Figure 4-11](#).



FIGURE 4-11 Fields of the Win32PrioritySeparation registry value.

The fields shown in [Figure 4-11](#) can be defined as follows:

- **Short vs. Long** A value of 1 specifies long quantums, and a value of 2 specifies short ones. A setting of 0 or 3 indicates that the default appropriate for the system will be used (short for client systems, long for server systems).
- **Variable vs. Fixed** A value of 1 means to enable the variable quantum table based on the algorithm shown in the “[Variable quantums](#)” section. A setting of 0 or 3 means that the default appropriate for the system will be used (variable for client systems, fixed for server systems).
- **Priority Separation** This field (stored in the kernel variable PsPrioritySeparation) defines the priority separation (up to 2), as explained in the “[Variable quantums](#)” section.

When you use the Performance Options dialog box (refer to [Figure 4-10](#)), you can choose from only two combinations: short quantums with foreground quantums tripled, or long quantums with no quantum changes for foreground threads. However, you can select other combinations by modifying the Win32PrioritySeparation registry value directly.

Threads that are part of a process running in the idle process priority class always receive a single thread quantum, ignoring any sort of quantum configuration settings, whether set by default or set through the registry.

On Windows Server systems configured as application servers, the initial value of the Win32PrioritySeparation registry value will be hex 26, which is identical to the value set by the Optimize Performance for Programs option in the Performance Options dialog box. This selects quantum and priority-boost behavior like that on Windows client systems, which is appropriate for a server used primarily to host users’ applications.

On Windows client systems and on servers not configured as application servers, the initial value of the Win32PrioritySeparation registry setting will be 2. This provides values of 0 for the Short vs. Long and Variable vs. Fixed Bit fields, relying on the default behavior of the system (depending on whether it is a client system or a server system) for these options. However, it provides a value of 2 for the Priority Separation field. After the registry value has been changed via the Performance Options dialog box, it cannot be restored to this original value other than by modifying the registry directly.

Using a local kernel debugger, you can see how the two quantum configuration settings, Programs and Background Services, affect the PsPrioritySeparation and PspForegroundQuantum tables, as well as modify the QuantumReset value of threads on the system. Take the following steps:

1. Open the **System** utility in Control Panel or right-click the **This PC** icon on the desktop and choose **Properties**.
2. Click the **Advanced System Settings** label, click the **Advanced** tab, click the **Settings** button in the Performance section, and click the second **Advanced** tab.
3. Select the **Programs** option and click **Apply**. Keep this dialog box open for the duration of the experiment.
4. Dump the values of PsPrioritySeparation and PspForegroundQuantum, as shown here. The values shown are what you should see on a Windows system after making the change in steps 1–3. Notice how the variable short quantum table is being used and that a priority boost of 2 will apply to foreground applications:

[Click here to view code image](#)

```
lkd> dd nt!PsPrioritySeparation L1
fffff803'75e0e388  00000002
lkd> db nt!PspForegroundQuantum L3

fffff803'76189d28  06 0c 12
```

5. Look at the QuantumReset value of any process on the system. As noted, this is the default full quantum of each thread on the system when it is replenished. This value is cached into each thread of the process, but the KPROCESS structure is easier to look at. Notice in this case it is 6, because WinDbg, like most other applications, gets the quantum set in the first entry of the PspForegroundQuantum table:

[Click here to view code image](#)

```
lkd> .process
Implicit process is now fffffe001'4f51f080
lkd> dt nt!_KPROCESS fffffe001'4f51f080 QuantumReset
+0x1bd QuantumReset : 6 ''
```

6. Change the Performance option to **Background Services** in the dialog box you opened in steps 1 and 2.
7. Repeat the commands shown in steps 4 and 5. You should see the values change in a manner consistent with our discussion in this section:

[Click here to view code image](#)

```
lkd> dd nt!PsPrioritySeparation L1
fffff803'75e0e388  00000000
lkd> db nt!PspForegroundQuantum L3
fffff803'76189d28  24 24 24
lkd> dt nt!_KPROCESS fffffe001'4f51f080 QuantumReset
+0x1bd QuantumReset : 36 '$'
```

Priority boosts

The Windows scheduler periodically adjusts the current (dynamic) priority of threads through an internal priority-boosting mechanism. In many cases, it does so to decrease various latencies (that is, to make threads respond faster to the events they are waiting on) and increase responsiveness. In others, it applies these boosts to prevent inversion and starvation scenarios. Here are some of the boost scenarios that will be described in this section (and their purpose):

- Boosts due to scheduler/dispatcher events (latency reduction)
- Boosts due to I/O completion (latency reduction)
- Boosts due to user interface (UI) input (latency reduction/responsiveness)
- Boosts due to a thread waiting on an executive resource (ERESOURCE) for too long (starvation avoidance)
- Boosts when a thread that's ready to run hasn't been running for some time (starvation and priority-inversion avoidance)

Like any scheduling algorithms, however, these adjustments aren't perfect, and they might not benefit all applications.



Note

Windows never boosts the priority of threads in the real-time range (16 through 31). Therefore, scheduling is always predictable with respect to other threads in this range. Windows assumes that if you're using the real-time thread priorities, you know what you're doing.

Client versions of Windows also include a pseudo-boosting mechanism that occurs during multimedia playback. Unlike the other priority boosts, multimedia-playback boosts are managed by a kernel-mode driver called the Multimedia Class Scheduler Service (mmcss.sys). They are not really boosts, however. The driver merely sets new priorities for the threads as needed. Therefore, none of the rules regarding boosts apply. We'll first cover the typical kernel-managed priority boosts and then talk about MMCSS and the kind of "boosting" it performs.

Boosts due to scheduler/dispatcher events

Whenever a dispatch event occurs, the `KiExitDispatcher` routine is called. Its job is to process the deferred ready list by calling `KiProcessThreadWaitList` and then call `KzCheckForThreadDispatch` to check whether any threads on the current processor should not be scheduled. Whenever such an event occurs, the caller can also specify which type of boost should be applied to the thread, as well as what priority increment the boost should be associated with. The following scenarios are considered as `AdjustUnwait` dispatch events because they deal with a dispatcher (synchronization) object entering a signaled state, which might cause one or more threads to wake up:

- An asynchronous procedure call (APC; described in [Chapter 6](#) and in more detail in Chapter 8 in Part 2) is queued to a thread.
- An event is set or pulsed.
- A timer was set, or the system time was changed, and timers had to be reset.
- A mutex was released or abandoned.

- A process exited.
- An entry was inserted in a queue (KQUEUE), or the queue was flushed.
- A semaphore was released.
- A thread was alerted, suspended, resumed, frozen, or thawed.
- A primary UMS thread is waiting to switch to a scheduled UMS thread.

For scheduling events associated with a public API (such as `SetEvent`), the boost increment applied is specified by the caller. Windows recommends certain values to be used by developers, which will be described later. For alerts, a boost of 2 is applied (unless the thread is put in an alert wait by calling `KeAlertThreadByThreadId`, in which case the applied boost is 1), because the alert API does not have a parameter allowing a caller to set a custom increment.

The scheduler also has two special `AdjustBoost` dispatcher events, which are part of the lock-ownership priority mechanism. These boosts attempt to fix situations in which a caller that owns the lock at priority x ends up releasing the lock to a waiting thread at priority $\leq x$. In this situation, the new owner thread must wait for its turn (if running at priority x), or worse, it might not even get to run at all if its priority is lower than x . This means the releasing thread continues its execution, even though it should have caused the new owner thread to wake up and take control of the processor. The following two dispatcher events cause an `AdjustBoost` dispatcher exit:

- An event is set through the `KeSetEventBoostPriority` interface, which is used by the `ERESOURCE` reader-writer kernel lock.
- A gate is set through the `KeSignalGate` interface, which is used by various internal mechanisms when releasing a gate lock.

Unwait boosts

Unwait boosts attempt to decrease the latency between a thread waking up due to an object being signaled (thus entering the ready state) and the thread actually beginning its execution to process the unwait (thus entering the running state). Generally speaking, it is desirable that a thread that wakes up from a waiting state would be able to run as soon as possible.

The various Windows header files specify recommended values that kernel-mode callers of APIs such as `KeReleaseMutex`, `KeSetEvent` and `KeReleaseSemaphore` should use, which correspond to definitions such as `MUTANT_INCREMENT`, `SEMAPHORE_INCREMENT`, and `EVENT_INCREMENT`. These three definitions have always been set to 1 in the headers, so it is safe to assume that most unwaits on these objects result in a boost of 1. In the user-mode API, an increment cannot be specified, nor do the native system calls such as `NtSetEvent` have parameters to specify such a boost. Instead, when these APIs call the underlying Ke interface, they automatically use the default `_INCREMENT` definition. This is also the case when mutexes are abandoned or timers are reset due to a system time change: The system uses the default boost that normally would have been applied when the mutex would have been released. Finally, the APC boost is completely up to the caller. Soon, you'll see a specific usage of the APC boost related to I/O completion.



Note

Some dispatcher objects don't have boosts associated with them. For example, when a timer is set or expires, or when a process is signaled, no boost is applied.

All these boosts of 1 attempt to solve the initial problem by assuming that both the releasing and waiting threads are running at the same priority. By boosting the waiting thread by one priority level, the waiting thread should preempt the releasing thread as soon as the operation completes. Unfortunately, on uniprocessor systems, if this assumption does not hold, the boost might not do much. For example, if the waiting thread is at priority 4 and the releasing thread is at priority 8, waiting at priority 5 won't do much to reduce latency and force preemption. On multiprocessor systems, however, due to the stealing and balancing algorithms, this higher-priority thread may have a better chance of getting picked up by another logical processor. This is due to a design choice made in the initial NT architecture, which is to not track lock ownership (except a few locks). This means the scheduler can't be sure who really owns an event and if it's really being used as a lock. Even with lock-ownership tracking, ownership is not usually passed (to avoid convoy issues) other than in the executive resource case, explained in an upcoming section.

For certain kinds of lock objects that use events or gates as their underlying synchronization object, the lock-ownership boost resolves the dilemma. Also, on a multiprocessor machine, the ready thread might get picked up on another processor (due to the processor-distribution and load-balancing schemes you'll see later), and its high priority might increase the chances of it running on that secondary processor instead.

Lock-ownership boosts

Because the executive-resource (ERESOURCE) and critical-section locks use underlying dispatcher objects, releasing these locks results in an unwait boost as described earlier. On the other hand, because the high-level implementation of these objects tracks the owner of the lock, the kernel can make a more informed decision as to what kind of boost should be applied by using the `AdjustBoost` reason. In these kinds of boosts, `AdjustIncrement` is set to the current priority of the releasing (or setting) thread, minus any graphical user interface (GUI) foreground separation boost. In addition, before the `KiExitDispatcher` function is called, `KiRemoveBoostThread` is called by the event and gate code to return the releasing thread back to its regular priority. This step is needed to avoid a lock-convoy situation, in which two threads repeatedly passing the lock between one another get ever-increasing boosts.



Note

Pushlocks, which are unfair locks because ownership of the lock in a contended acquisition path is not predictable (rather, it's random, like a spinlock), do not apply priority boosts due to lock ownership. This is because doing so only contributes to preemption and priority proliferation, which isn't required because the lock becomes immediately free as soon as it is released (bypassing the normal wait/unwait path).

Other differences between the lock-ownership boost and unwait boost will be exposed in the way the scheduler actually applies boosting, which is the subject of the next section.

Priority boosting after I/O completion

Windows gives temporary priority boosts upon completion of certain I/O operations so that threads that were waiting for an I/O have more of a chance to run right away and process whatever was being waited for. Although you'll find recommended boost values in the Windows Driver Kit (WDK) header files (by searching for `#define IO_` in `Wdm.h` or `Ntddk.h`), the actual value for the boost is up to the device driver. (These values are listed in [Table 4-3](#).) It is the device driver that specifies the boost when it completes an I/O request on its call to the kernel function, `IoCompleteRequest`. In [Table 4-3](#), notice that I/O requests to devices that warrant better responsiveness have higher boost values.

Device	Boost
Disk, CD-ROM, parallel, video	1
Network, mailslot, named pipe, serial	2
Keyboard, mouse	6
Sound	8

TABLE 4-3 Recommended boost values



Note

You might intuitively expect better responsiveness from your video card or disk than a boost of 1. However, the kernel is in fact trying to optimize for *latency*, to which some devices (as well as human sensory inputs) are more sensitive than others. To give you an idea, a sound card expects data around every 1 ms to play back music without perceptible glitches, while a video card needs to output at only 24 frames per second, or about once every 40 ms, before the human eye can notice glitches.

As hinted earlier, these I/O completion boosts rely on the unwait boosts seen in the previous section. [Chapter 6](#) shows the mechanism of I/O completion in depth. For now, the important detail is that the kernel implements the signaling code in the `IoCompleteRequest` API through the use of either an APC (for asynchronous I/O) or through an event (for synchronous I/O). When a driver passes in—for example, `IO_DISK_INCREMENT` to `IoCompleteRequest` for an asynchronous disk read—the kernel calls `KeInsertQueueApc` with the boost parameter set to `IO_DISK_INCREMENT`. In turn, when the thread's wait is broken due to the APC, it receives a boost of 1.

Be aware that the boost values given in [Table 4-3](#) are merely recommendations by Microsoft. Driver developers are free to ignore them, and certain specialized drivers can use their own values. For example, a driver handling ultrasound data from a medical device, which must notify a user-mode visualization application of new data, would probably use a boost value of 8 as well, to satisfy the same latency as a sound card. In most cases, however, due to the way Windows driver stacks are built (again, see [Chapter 6](#) for more information), driver developers often write *minidrivers*, which call into a Microsoft-owned driver that supplies its own boost to `IoCompleteRequest`. For example, RAID or SATA controller card developers typically call `StorPortCompleteRequest` to complete processing their requests. This call does not have any parameter for a boost value, because the `Storport.sys` driver fills in the right value when calling the kernel. Additionally, whenever any file system driver (identified by setting its device type to `FILE_DEVICE_DISK_FILE_SYSTEM` or

FILE_DEVICE_NETWORK_FILE_SYSTEM) completes its request, a boost of `IO_DISK_INCREMENT` is always applied if the driver passed in `IO_NO_INCREMENT` (0) instead. So this boost value has become less of a recommendation and more of a requirement enforced by the kernel.

Boosts during waiting on executive resources

When a thread attempts to acquire an executive resource (`ERESOURCE`; see Chapter 8 in Part 2 for more information on kernel-synchronization objects) that is already owned exclusively by another thread, it must enter a wait state until the other thread has released the resource. To limit the risk of deadlocks, the executive performs this wait in intervals of 500 ms instead of doing an infinite wait on the resource. At the end of these 500 ms, if the resource is still owned, the executive attempts to prevent CPU starvation by acquiring the dispatcher lock, boosting the owning thread or threads to 15 (if the original owner priority is less than the waiter's and not already 15), resetting their quantum, and performing another wait.

Because executive resources can be either shared or exclusive, the kernel first boosts the exclusive owner and then checks for shared owners and boosts all of them. When the waiting thread enters the wait state again, the hope is that the scheduler will schedule one of the owner threads, which will have enough time to complete its work and release the resource. Note that this boosting mechanism is used only if the resource doesn't have the Disable Boost flag set, which developers can choose to set if the priority-inversion mechanism described here works well with their usage of the resource.

Additionally, this mechanism isn't perfect. For example, if the resource has multiple shared owners, the executive boosts all those threads to priority 15. This results in a sudden surge of high-priority threads on the system, all with full quantum. Although the initial owner thread will run first (because it was the first to be boosted and therefore is first on the ready list), the other shared owners will run next because the waiting thread's priority was not boosted. Only after all the shared owners have had a chance to run and their priority has been decreased below the waiting thread will the waiting thread finally get its chance to acquire the resource. Because shared owners can promote or convert their ownership from shared to exclusive as soon as the exclusive owner releases the resource, it's possible for this mechanism not to work as intended.

Priority boosts for foreground threads after waits

As will be described shortly, whenever a thread in the foreground process completes a wait operation on a kernel object, the kernel boosts its current (not base) priority by the current value of `PSPrioritySeparation`. (The windowing system is responsible for determining which process is considered to be in the foreground.) As described earlier in this chapter in the section "[Controlling the quantum](#)," `PSPrioritySeparation` reflects the quantum-table index used to select quantum for the threads of foreground applications. However, in this case, it is being used as a priority boost value.

The reason for this boost is to improve the responsiveness of interactive applications. By giving the foreground application a small boost when it completes a wait, it has a better chance of running right away, especially when other processes at the same base priority might be running in the background.

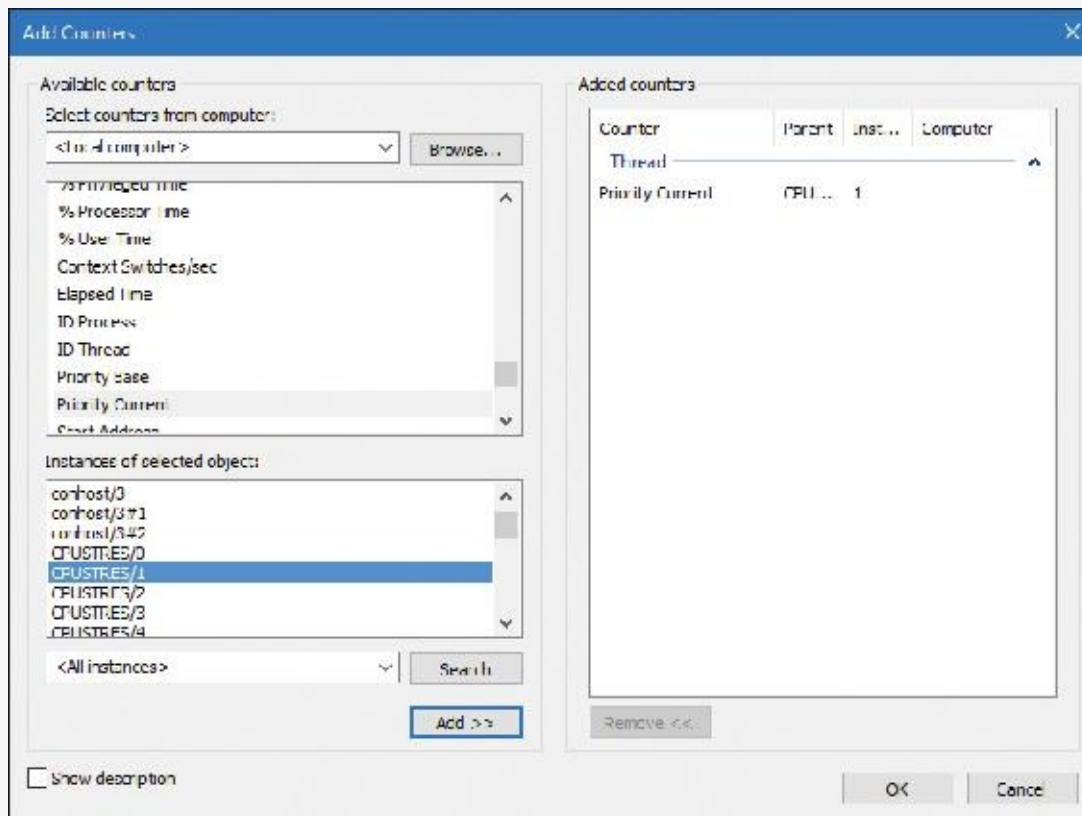
EXPERIMENT: Watching foreground priority boosts and decays

Using the CPU Stress tool, you can watch priority boosts in action. Take the following steps:

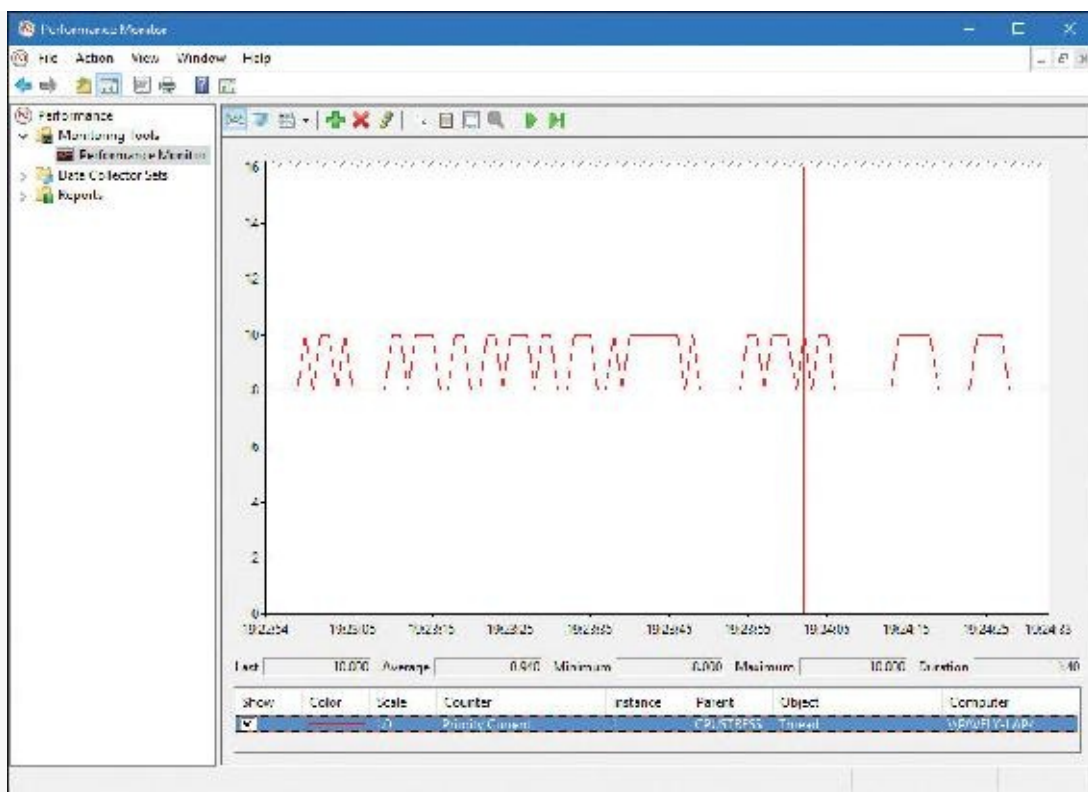
1. Open the System utility in Control Panel or right-click the **This Computer** icon on the desktop and choose **Properties**.
2. Click the **Advanced System Settings** label, click the **Advanced** tab, click the **Settings** button

in the Performance section, and click the **Advanced** tab.

3. Select the **Programs** option. This gives PsPrioritySeparation a value of 2.
4. Run CPU Stress, right-click thread 1, and choose **Busy** from the context menu.
5. Start the Performance Monitor tool.
6. Click the **Add Counter** toolbar button or press **Ctrl+I** to open the Add Counters dialog box.
7. Select the **Thread** object and then select the **Priority Current** counter.
8. In the Instances box, select **<All Instances>** and click **Search**.
9. Scroll down to the CPUSTRES process, select the second thread (thread 1; the first thread is the GUI thread) and click the **Add** button. You should see something like this:



10. Click **OK**.
11. Right-click the counter and select **Properties**.
12. Click the **Graph** tab and change the maximum vertical scale to 16. Then click **OK**.
13. Bring the CPUSTRES process to the foreground. You should see the priority of the CPUSTRES thread being boosted by 2 and then decaying back to the base priority. CPUSTRES periodically receives a boost of 2 because the thread you're monitoring is sleeping about 25 percent of the time and then waking up. (This is the Busy activity level.) The boost is applied when the thread wakes up. If you set the activity level to Maximum, you won't see any boosts because Maximum in CPUSTRES puts the thread into an infinite loop. Therefore, the thread doesn't invoke any wait functions and therefore doesn't receive any boosts.



14. When you've finished, exit Performance Monitor and CPU Stress.

Priority boosts after GUI threads wake up

Threads that own windows receive an additional boost of 2 when they wake up because of windowing activity such as the arrival of window messages. The windowing system (Win32k.sys) applies this boost when it calls `KeSetEvent` to set an event used to wake up a GUI thread. The reason for this boost is similar to the previous one: to favor interactive applications.

EXPERIMENT: Watching priority boosts on GUI threads

You can see the windowing system apply its boost of 2 for GUI threads that wake up to process window messages by monitoring the current priority of a GUI application and moving the mouse across the window. Just follow these steps:

1. Open the **System** utility in Control Panel.
2. Click the **Advanced System Settings** label, click the **Advanced** tab, click the **Settings** button in the Performance section, and click the **Advanced** tab.
3. Select the **Programs** option. This gives `PsPrioritySeparation` a value of 2.
4. Run Notepad.
5. Start the Performance Monitor tool.
6. Click the **Add Counter** toolbar button or press **Ctrl+I** to open the Add Counters dialog box.
7. Select the **Thread** object and then select the **Priority Current** counter.
8. In the Instances box, type **Notepad**. Then click **Search**.
9. Scroll down to the Notepad/0 entry, click it, click the **Add** button, and then click **OK**.
10. As in the previous experiment, change the maximum vertical scale to 16. You should see the priority of thread 0 in Notepad at 8 or 10. (Because Notepad entered a wait state shortly after it received the boost of 2 that threads in the foreground process receive, it might not yet have decayed from 10 to 8.)
11. With Performance Monitor in the foreground, move the mouse across the Notepad window. (Make both windows visible on the desktop.) Notice that the priority sometimes remains at 10 and sometimes at 9, for the reasons just explained.



Note

You won't likely catch Notepad at 8. This is because it runs so little after receiving the GUI thread boost of 2 that it never experiences more than one priority level of decay before waking up again. (This is due to additional windowing activity and the fact that it receives the boost of 2 again.)

-
12. Bring Notepad to the foreground. You should see the priority rise to 12 and remain there. This is because the thread is receiving two boosts: the boost of 2 applied to GUI threads when they wake up to process windowing input and an additional boost of 2 because Notepad is in the foreground. (Or, you may see it drop to 11 if it experiences the normal priority decay that occurs for boosted threads on the quantum end.)
 13. Move the mouse over Notepad while it's still in the foreground. You might see the priority drop to 11 (or maybe even 10) as it experiences the priority decay that normally occurs on boosted threads as they complete their turn. However, the boost of 2 that is applied because it's the foreground process remains as long as Notepad remains in the foreground.
 14. Exit Performance Monitor and Notepad.
-

Priority boosts for CPU starvation

Imagine the following situation: A priority 7 thread is running, preventing a priority 4 thread from ever receiving CPU time. However, a priority 11 thread is waiting for some resource that the priority 4 thread has locked. But because the priority 7 thread in the middle is eating up all the CPU time, the priority 4 thread will never run long enough to finish whatever it's doing and release the resource blocking the priority 11 thread. This scenario is known as *priority inversion*.

What does Windows do to address this situation? An ideal solution (at least in theory) would be to track locks and owners and boost the appropriate threads so that forward progress can be made. This idea is implemented with a feature called *Autoboost*, described later in this chapter in the section “[Autoboost](#).” However, for general starvation scenarios, the following mitigation is used.

You saw how the code responsible for executive resources manages this scenario by boosting the owner threads so that they can have a chance to run and release the resource. However, executive resources are only one of the many synchronization constructs available to developers, and the boosting technique will not apply to any other primitive. Therefore, Windows also includes a generic CPU starvation-relief mechanism as part of a thread called the *balance-set manager*. (This is a system thread that exists primarily to perform memory-management functions and is described in more detail in [Chapter 5](#).) Once per second, this thread scans the ready queues for any threads that have been in the ready state (that is, haven't run) for approximately 4 seconds. If it finds such a thread, the balance-set manager boosts the thread's priority to 15 and sets the quantum target to an equivalent CPU clock cycle count of 3 quantum units. After the quantum expires, the thread's priority decays immediately to its original base priority. If the thread wasn't finished and a higher-priority thread is ready to run, the decayed thread returns to the ready queue, where it again becomes eligible for another boost if it remains there for another 4 seconds.

The balance-set manager doesn't actually scan all the ready threads every time it runs. To minimize the CPU time it uses, it scans only 16 ready threads. If there are more threads at that priority level, it remembers where it left off and picks up again on the next pass. Also, it boosts only 10 threads per pass. If it finds more than 10 threads meriting this particular boost (which indicates an unusually busy system), it stops the scan and picks up again on the next pass.



Note

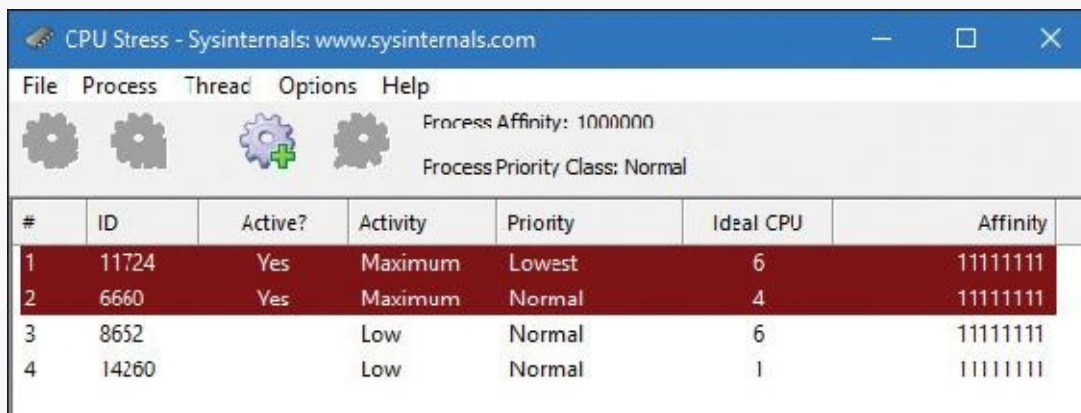
As mentioned, scheduling decisions in Windows are not affected by the number of threads and are made in constant time. Because the balance-set manager must scan ready queues manually, this operation depends on the number of threads on the system; more threads require more scanning time. However, the balance-set manager is not considered part of the scheduler or its algorithms and is simply an extended mechanism to increase reliability. Additionally, because of the cap on threads and queues to scan, the performance impact is minimized and predictable in a worst-case scenario.

EXPERIMENT: Watching priority boosts for CPU starvation

Using the CPU Stress tool, you can watch priority boosts in action. In this experiment, you'll see CPU usage change when a thread's priority is boosted. Take the following steps:

1. Run CPUTRES.exe.
2. The activity level of thread 1 is Low. Change it to **Maximum**.
3. The thread priority of thread 1 is Normal. Change it to **Lowest**.

4. Click thread 2. Its activity level is Low. Change it to **Maximum**.
5. Change the process affinity mask to a single logical processor. To do so, open the **Process** menu and choose **Affinity**. (It doesn't matter which processor.) Alternatively, use Task Manager to make the change. The screen should look something like this:



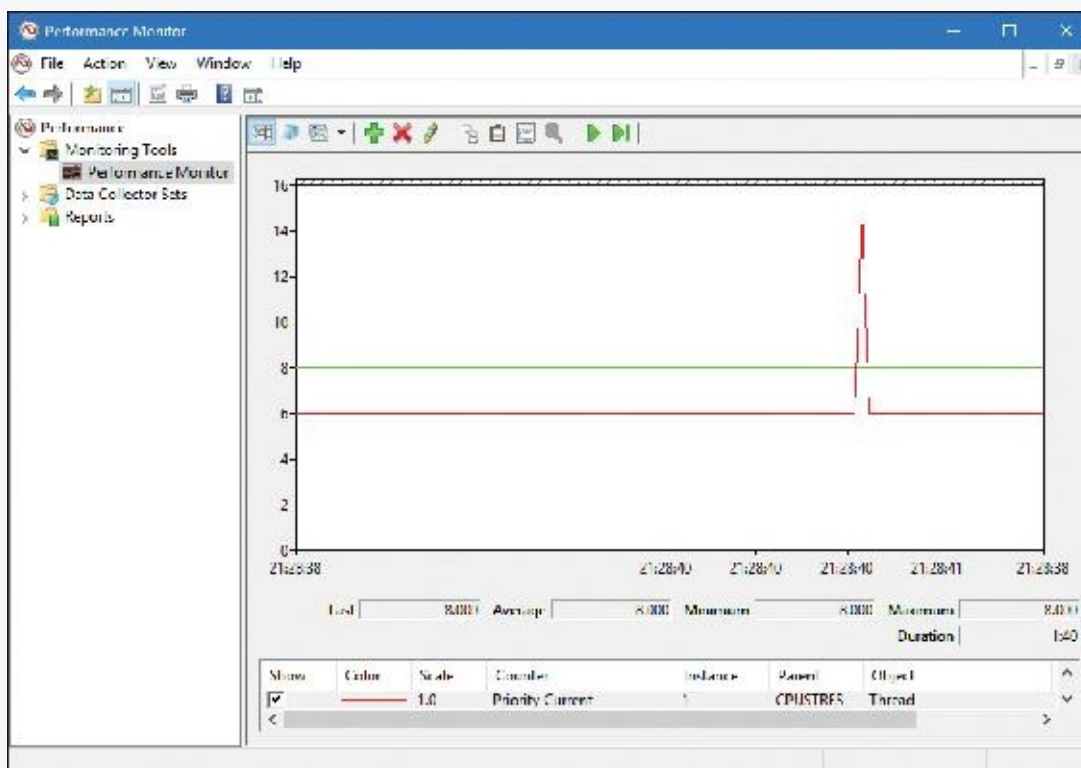
CPU Stress - Sysinternals: www.sysinternals.com

File Process Thread Options Help

Process Affinity: 10000000
Process Priority Class: Normal

#	ID	Active?	Activity	Priority	Ideal CPU	Affinity
1	11724	Yes	Maximum	Lowest	6	11111111
2	6660	Yes	Maximum	Normal	4	11111111
3	8652		Low	Normal	6	11111111
4	14260		Low	Normal	1	11111111

6. Start the Performance Monitor tool.
7. Click the **Add Counter** toolbar button or press **Ctrl+I** to open the Add Counters dialog box.
8. Select the **Thread** object and then select the **Priority Current** counter.
9. In the Instances box, type **CPUSTRES** and click **Search**.
10. Select threads 1 and 2 (thread 0 is the GUI thread), click the **Add** button, and click **OK**.
11. Change the vertical scale maximum to 16 for both counters.
12. Because Performance Monitor refreshes once per second, you may miss the priority boosts. To help with that, press **Ctrl+F** to freeze the display. Then force updates to occur more frequently by pressing and holding down **Ctrl+U**. With some luck, you may see a priority boost for the lower-priority thread to level 15 like so:



13. Exit Performance Monitor and CPU Stress.

Applying boosts

Back in `KiExitDispatcher`, you saw that `KiProcessThreadWaitList` is called to process any threads in the deferred ready list. It is here that the boost information passed by the caller is processed. This is done by looping through each `DeferredReady` thread, unlinking its wait blocks (only `Active` and `Bypassed` blocks are unlinked), and setting two key values in the kernel's thread control block: `Adjust-Reason` and `AdjustIncrement`. The reason is one of the two `Adjust` possibilities seen earlier, and the increment corresponds to the boost value.

`KiDeferredReadyThread` is then called. This makes the thread ready for execution by running two algorithms: the quantum and priority selection algorithm (which you are about to see in two parts) and the processor selection algorithm (which is shown in the "[Processor selection](#)" section later in this chapter).

Let's first look at when the algorithm applies boosts, which happens only in cases when a thread is not in the real-time priority range. For an `AdjustUnwait` boost, it will be applied only if the thread is not already experiencing an unusual boost and only if the thread has not disabled boosting by calling `SetThreadPriorityBoost`, which sets the `DisableBoost` flag in the `KTHREAD`. Another situation that can disable boosting in this case is if the kernel has realized that the thread has actually exhausted its quantum (but the clock interrupt did not fire to consume it) and it has come out of a wait that lasted less than two clock ticks.

If these situations are not currently true, the new priority of the thread will be computed by adding the `AdjustIncrement` to the thread's current base priority. Additionally, if the thread is known to be part of a foreground process (meaning that the memory priority is set to `MEMORY_PRIORITY_FOREGROUND`, which is configured by `Win32k.sys` when focus changes), this is where the priority-separation boost (`PSPrioritySeparation`) is applied by adding its value on top of the new priority. This is also known as the *foreground priority boost*, which was explained earlier.

Finally, the kernel checks whether this newly computed priority is higher than the current priority of the thread, and it limits this value to an upper bound of 15 to avoid crossing into the real-time range. It then sets this value as the thread's new current priority. If any foreground separation boost was applied, it sets this value in the `ForegroundBoost` field of the `KTHREAD`, which results in a `PriorityDecrement` equal to the separation boost.

For `AdjustBoost` boosts, the kernel checks whether the thread's current priority is lower than the `AdjustIncrement` (recall this is the priority of the setting thread) and whether the thread's current priority is below 13. If so, and priority boosts have not been disabled for the thread, the `AdjustIncrement` priority is used as the new current priority, limited to a maximum of 13. Meanwhile, the `UnusualBoost` field of the `KTHREAD` contains the boost value, which results in a `PriorityDecrement` equal to the lock-ownership boost.

In all cases where a `PriorityDecrement` is present, the quantum of the thread is also recomputed to be the equivalent of only one clock tick, based on the value of `KiLockQuantumTarget`. This ensures that foreground and unusual boosts will be lost after one clock tick instead of the usual two (or other configured value), as will be shown in the next section. This also happens when an `AdjustBoost` is requested but the thread is running at priority 13 or 14 or with boosts disabled.

After this work is complete, `AdjustReason` is now set to `AdjustNone`.

Removing boosts

Removing boosts is done in `KiDeferredReadyThread` as boosts and quantum recomputations are being applied (as shown in the previous section). The algorithm first begins by checking the type of adjustment being done.

For an `AdjustNone` scenario, which means the thread became ready perhaps due to a preemption, the thread's quantum will be recomputed if it already hit its target but the clock interrupt has not yet noticed, as long as the thread was running at a dynamic priority level. Additionally, the thread's priority will be recomputed. For an `AdjustUnwait` or `AdjustBoost` scenario on a non-real-time thread, the kernel checks whether the thread silently exhausted its quantum (as in the prior section). If so, or if the thread was running with a base priority of 14 or higher, or if no `PriorityDecrement` is present and the thread has completed a wait that lasted longer than two clock ticks, the quantum of the thread is recomputed, as is its priority.

Priority recomputation happens on non-real-time threads. It's done by taking the thread's current priority, subtracting its foreground boost, subtracting its unusual boost (the combination of these last two items is the `PriorityDecrement`), and finally subtracting 1. This new priority is bounded with the base priority as the lowest bound and any existing priority decrement is zeroed out (clearing unusual and foreground boosts). This means that in the case of a lock-ownership boost or any of the other unusual boosts explained, the entire boost value is now lost. On the other hand, for a regular `AdjustUnwait` boost, the priority naturally trickles down by 1 due to the subtraction by 1. This lowering eventually stops when the base priority is hit due to the lower bound check.

There is another instance where boosts must be removed, which goes through the `KiRemoveBoostThread` function. This is a special-case boost removal that occurs due to the lock-ownership boost rule, which specifies that the setting thread must lose its boost when donating its current priority to the waking thread (to avoid a lock convoy). It is also used to undo the boost due to targeted deferred procedure calls (DPCs) as well as the boost against `ERESOURCE` lock-starvation boost. The only special detail about this routine is that when computing the new priority, it takes special care to separate the `ForegroundBoost` and `UnusualBoost` components of the `PriorityDecrement` to maintain any GUI foreground-separation boost that the thread accumulated. This behavior, which appeared starting with Windows 7, ensures that threads relying on the lock-ownership boost do not behave erratically when running in the foreground, or vice-versa.

[Figure 4-12](#) displays an example of how normal boosts are removed from a thread as it experiences quantum end.

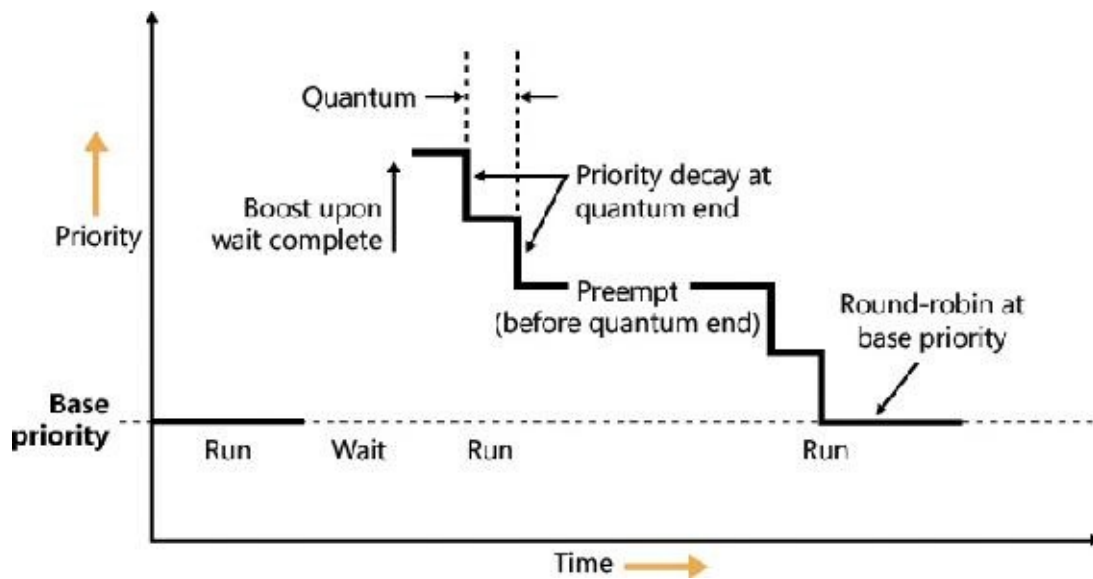


FIGURE 4-12 Priority boosting and decay.

Priority boosts for multimedia applications and games

Although Windows' CPU-starvation priority boosts might be enough to get a thread out of an abnormally long wait state or potential deadlock, they simply cannot deal with the resource requirements imposed by a CPU-intensive application such as Windows Media Player or a 3D computer game.

Skipping and other audio glitches have long been a common source of irritation among Windows users. The Windows user-mode audio stack exacerbates this situation because it offers even more chances for preemption. To address this, client versions of Windows use the MMCSS driver (described earlier in this chapter), implemented in %SystemRoot%\System32\Drivers\MMCSS.sys. Its purpose is to ensure glitch-free multimedia playback for applications that register with it.



Note

Windows 7 implements MMCSS as a service (rather than a driver). This posed a potential risk, however. If the MMCSS managing thread blocked for any reason, the threads managed by it would retain their real-time priorities, potentially causing system-wide starvation. The solution was to move the code to the kernel where the managing thread (and other resources used by MMCSS) could not be touched. There are other benefits to being a kernel driver, such as holding a direct pointer to process and thread objects rather than IDs or handles. This bypasses searches based on IDs or handles and allows faster communication with the scheduler and Power Manager.

Client applications can register with MMCSS by calling `AvSetMmThreadCharacteristics` with a task name that must match one of the subkeys under `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Multimedia\SystemProfile\Tasks`. (The list can be modified by OEMs to include other specific tasks as appropriate.) Out of the box, the following tasks exist:

- Audio
- Capture
- Distribution
- Games
- Low Latency

- Playback
- Pro Audio
- Window Manager

Each of these tasks includes information about the various properties that differentiate them. The most important one for scheduling is called the *Scheduling Category*, which is the primary factor determining the priority of threads registered with MMCSS. [Table 4-4](#) shows the various scheduling categories.

Category	Priority	Description
High	23–26	Pro Audio threads running at a higher priority than any other thread on the system except for critical system threads
Medium	16–22	The threads part of a foreground application such as Windows Media Player
Low	8–15	All other threads that are not part of the previous categories
Exhausted	4–6	Threads that have exhausted their share of the CPU and will continue running only if no other higher-priority threads are ready to run

TABLE 4-4 Scheduling categories

The main mechanism behind MMCSS boosts the priority of threads inside a registered process to the priority level matching their scheduling category and relative priority within this category for a guaranteed period. It then lowers those threads to the exhausted category so that other, non-multimedia threads on the system can also get a chance to execute.

By default, multimedia threads get 80 percent of the CPU time available, while other threads receive 20 percent. (Based on a sample of 10 ms, that would be 8 ms and 2 ms, respectively.) You can change this percentage by modifying the `SystemResponsiveness` registry value under the `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Multimedia\SystemProfile` key. The value can range from 10 to 100 percent (20 is the default; setting a value lower than 10 evaluates to 10), which indicates the CPU percentage guaranteed to the system (not the registered audio apps). MMCSS scheduling thread runs at priority 27 because they need to preempt any Pro Audio threads to lower their priority to the exhausted category.

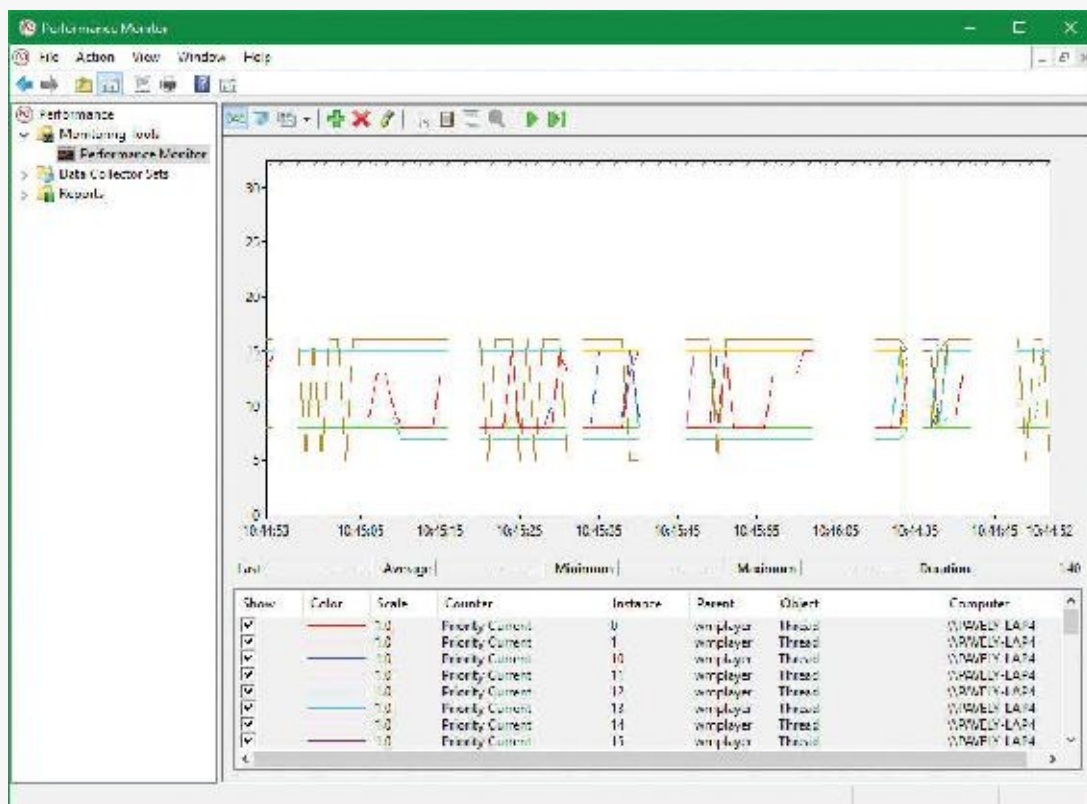
As discussed, changing the relative thread priorities within a process does not usually make sense, and no tool allows this because only developers understand the importance of the various threads in their programs. On the other hand, because applications must manually register with MMCSS and provide it with information about what kind of thread this is, MMCSS does have the necessary data to change these relative thread priorities—and developers are well aware that this will happen.

EXPERIMENT: MMCSS priority boosting

In this experiment, you'll see the effects of MMCSS priority boosting.

1. Run Windows Media Player (`wmplayer.exe`). (Other playback programs might not take advantage of the API calls required to register with MMCSS.)
2. Play some audio content.
3. Using Task Manager or Process Explorer, set the affinity of the `Wmplayer.exe` process so that it runs on only one CPU.
4. Start the Performance Monitor tool.
5. Using Task Manager, change Performance Monitor's priority class to **Realtime** so it will have a better chance of recording activity.

6. Click the **Add Counter** toolbar button or press **Ctrl+I** to open the Add Counters dialog box.
7. Select the **Thread** object and then select the **Priority Current**.
8. In the Instances box, type **Wmplayer**, click **Search**, and then select all its threads.
9. Click the **Add** button and click **OK**.
10. Open the **Action** menu and choose **Properties**.
11. On the Graph tab, change the maximum vertical scale to 32. You should see one or more priority-16 threads inside Wmplayer, which will be constantly running unless there is a higher-priority thread requiring the CPU after they are dropped to the exhausted category.
12. Run CPU Stress.
13. Set the activity level of thread 1 to **Maximum**.
14. The priority of thread 1 is Normal. Change it to **Time Critical**.
15. Change the CPUTRES priority class to **High**.
16. Change the CPUTRES affinity to use the same CPU used for Wmplayer. The system should slow down considerably, but the music playback should continue. Every so often, you'll be able to get back some responsiveness from the rest of the system.
17. In Performance Monitor, notice that the WmPlayer priority 16 threads drop from time to time as shown here:



MMCSS' functionality does not stop at simple priority boosting, however. Because of the nature of network drivers on Windows and the NDIS stack, DPCs are quite common mechanisms for delaying work after an interrupt has been received from the network card. Because DPCs run at an IRQL level higher than user-mode code (see [Chapter 6](#) for more information on DPCs and IRQLs), long-running network card driver code can still interrupt media playback—for example, during network transfers or when playing a game.

MMCSS sends a special command to the network stack, telling it to throttle network packets during the

duration of the media playback. This throttling is designed to maximize playback performance—at the cost of some small loss in network throughput (which would not be noticeable for network operations usually performed during playback, such as playing an online game). The exact mechanisms behind it do not belong to any area of the scheduler, so we'll leave them out of this description.

MMCSS also supports a feature called *deadline scheduling*. The idea is that an audio-playing program does not always need the highest priority level in its category. If such a program uses buffering (obtaining audio data from disk or network) and then plays the buffer while building the next buffer, deadline scheduling allows a client thread to indicate a time when it must get the high priority level to avoid glitches, but live with a slightly lower priority (within its category) in the meantime. A thread can use the `AvTaskIndexYield` function to indicate the next time it must be allowed to run, specifying the time it needs to get the highest priority within its category. Until that time arrives, it gets the lowest priority within its category, potentially freeing more CPU time to the system.

Autoboost

Autoboost is a framework targeted at the priority-inversion problem described in the previous section. The idea is to track lock owners and lock waiters in such a way that would allow boosting the appropriate threads' priorities (I/O priority as well if needed) to allow threads to make forward progress. The lock information is stored in a static array of `KLOCK_ENTRY` objects inside the `KTHREAD` structure. The current implementation uses a maximum of six entries. Each `KLOCK_ENTRY` maintains two binary trees: one for locks owned by the thread and the other for locks waited on by the thread. These trees are keyed by priority so that constant time is required to determine the highest priority to which boosting should be applied. If boost is required, the owner's priority is set to the waiter's priority. It may also boost I/O priority if these were issued with low priority. (See [Chapter 6](#) for more on I/O priority.) As with all priority boosts, the maximum priority achievable by Autoboost is 15. (The priority of real-time threads is never boosted.)

Current implementation uses the Autoboost framework for pushlocks and guarded mutexes synchronization primitives, which are exposed to kernel code only. (See Chapter 8 in Part 2 for more on these objects.) The framework is also used by some executive components for specialized cases. Future versions of Windows may implement Autoboost for user-mode accessible objects that have an ownership concept, such as critical sections.

Context switching

A thread's context and the procedure for context switching vary depending on the processor's architecture. A typical context switch requires saving and reloading the following data:

- Instruction pointer
- Kernel stack pointer
- A pointer to the address space in which the thread runs (the process's page table directory)

The kernel saves this information from the old thread by pushing it onto the current (old thread's) kernel-mode stack, updating the stack pointer, and saving the stack pointer in the old thread's `KTHREAD` structure. The kernel stack pointer is then set to the new thread's kernel stack, and the new thread's context is loaded. If the new thread is in a different process, it loads the address of its page table directory into a special processor register so that its address space is available. (See the description of address translation in [Chapter 5](#).) If a kernel APC that needs to be delivered is pending, an interrupt at IRQL 1 is requested. (For more information on APCs, see Chapter 8 in Part 2.) Otherwise, control passes to the new thread's restored instruction pointer and the new thread resumes execution.

Direct Switch

Windows 8 and Server 2012 introduced an optimization called Direct Switch, that allows a thread to donate its quantum and boost to another thread, which is then immediately scheduled on the same processor. In synchronous client/server scenarios, this can produce significant throughput improvements because the client/server threads are not migrated to other processors that may be idle or parked. Another way to think about this is that at any given time, only the client or the server thread is running, so the thread scheduler should treat them as a single logical thread. [Figure 4-13](#) shows the effect of using Direct Switch.

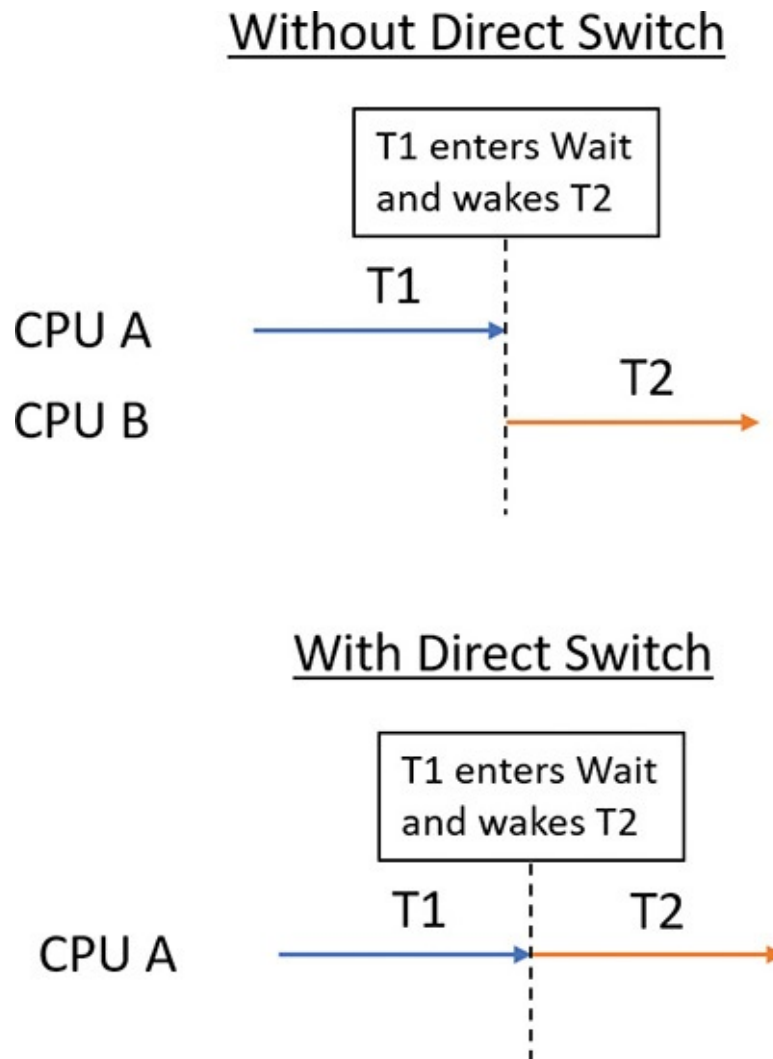


FIGURE 4-13 Direct Switch.

The scheduler has no way of knowing that the first thread (T1 in [Figure 4-13](#)) is about to enter a wait state after signaling some synchronization object that the second thread (T2) is waiting on. Therefore, a special function must be called to let the scheduler know that this is the case (atomic signal and wait).

If possible, the `KiDirectSwitchThread` function performs the actual switch. It's called by `KiExitDispatcher` if passed a flag indicating to use Direct Switch if possible. Priority donation, in which the first thread's priority is "donated" to the second thread (if the latter's priority is lower than the former), is applied if specified by yet another bit flag to `KiExitDispatcher`. In the current implementation, these two flags are always specified together (or none at all), meaning in any Direct Switch attempt, priority donation is attempted as well. Direct Switch can fail—for example, if the target thread's affinity precludes it from running on the current processor. However, if it succeeds, the quantum of the first thread is transferred to the target thread and the first thread loses its remaining quantum.

Direct Switch is currently used in the following scenarios:

- If a thread calls the `SignalObjectAndWait` Windows API (or its kernel equivalent `NtSignalAndWaitForSingleObject`)
- ALPC (described in Chapter 8 in Part 2)
- Synchronous remote procedure call (RPC) calls
- COM remote calls (currently MTA [multithreaded apartment] to MTA only)

Scheduling scenarios

Windows answers the question of “Who gets the CPU?” based on thread priority, but how does this approach work in practice? The following sections illustrate just how priority-driven preemptive multitasking works on the thread level.

Voluntary switch

A thread might voluntarily relinquish use of the processor by entering a wait state on some object (such as an event, a mutex, a semaphore, an I/O completion port, a process, a thread, and so on) by calling one of the Windows wait functions such as `WaitForSingleObject` or `WaitForMultipleObjects`. (Waiting for objects is described in more detail in Chapter 8 in Part 2.)

[Figure 4-14](#) illustrates a thread entering a wait state and Windows selecting a new thread to run. In [Figure 4-14](#), the top block (thread) is voluntarily relinquishing the processor so that the next thread in the ready queue can run. (This is represented by the halo it has when in the Running column.) Although it might appear from this figure that the relinquishing thread’s priority is being reduced, it’s not. It’s just being moved to the wait queue of the objects the thread is waiting for.

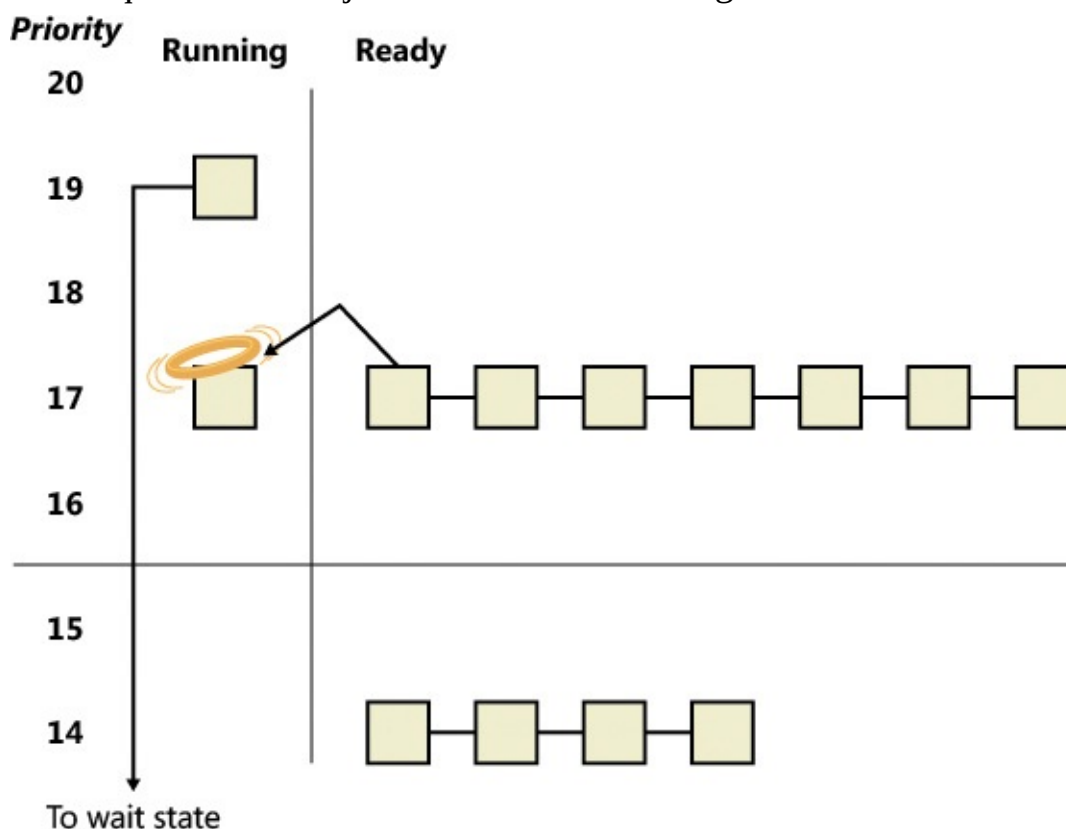


FIGURE 4-14 Voluntary switching.

Preemption

In this scheduling scenario, a lower-priority thread is preempted when a higher-priority thread becomes ready to run. This situation might occur for a couple of reasons:

- A higher-priority thread's wait completes (the event that the other thread was waiting for has occurred).
- A thread priority is increased or decreased.

In either of these cases, Windows must determine whether the currently running thread should continue to run or be preempted to allow a higher-priority thread to run.



Note

Threads running in user mode can preempt threads running in kernel mode. The mode in which the thread is running doesn't matter; the thread priority is the determining factor.

When a thread is preempted, it is put at the head of the ready queue for the priority it was running at (see [Figure 4-15](#)).

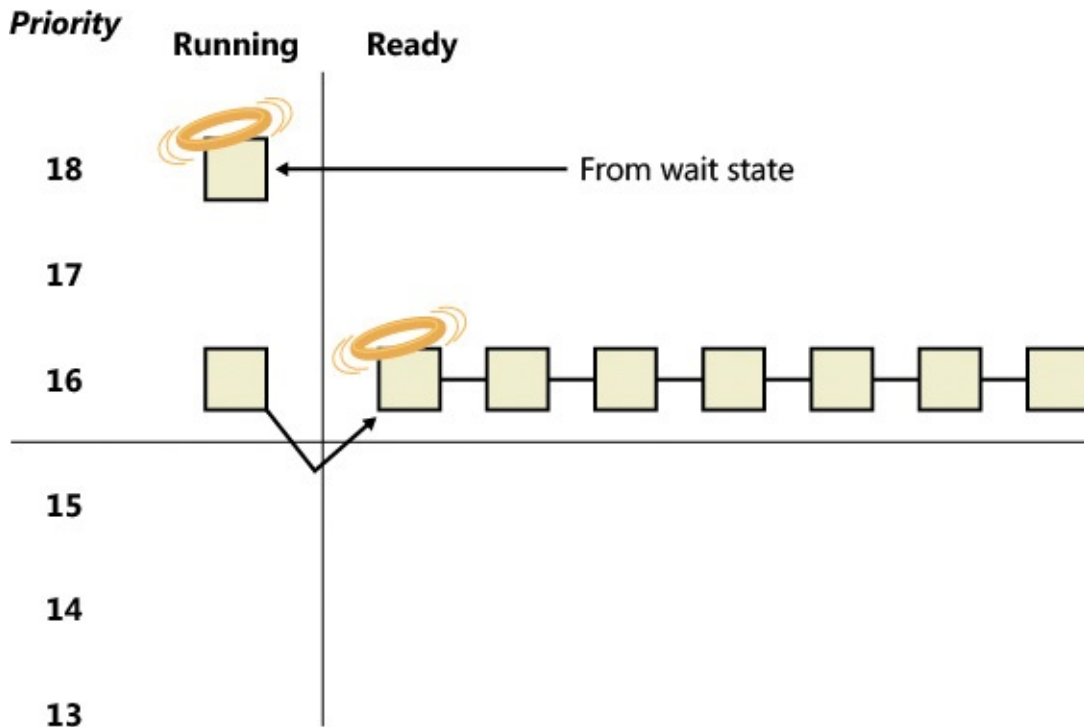


FIGURE 4-15 Preemptive thread scheduling.

In [Figure 4-15](#), a thread with priority 18 emerges from a wait state and repossesses the CPU, causing the thread that had been running (at priority 16) to be bumped to the head of the ready queue. Notice that the bumped thread doesn't go to the end of the queue. Rather, it goes to the beginning. When the preempting thread has finished running, the bumped thread can complete its quantum.

Quantum end

When the running thread exhausts its CPU quantum, Windows must determine whether the thread's priority should be decremented and then whether another thread should be scheduled on the processor.

If the thread priority is reduced (for example, because of some boost it received before), Windows looks for a more appropriate thread to schedule, such as one in a ready queue with a higher priority than

the new priority for the currently running thread. If the thread priority isn't reduced and there are other threads in the ready queue at the same priority level, Windows selects the next thread in the ready queue at that same priority level. It then moves the previously running thread to the tail of that queue, giving it a new quantum value and changing its state from running to ready. This is illustrated in [Figure 4-16](#). If no other thread of the same priority is ready to run, the thread gets to run for another quantum.

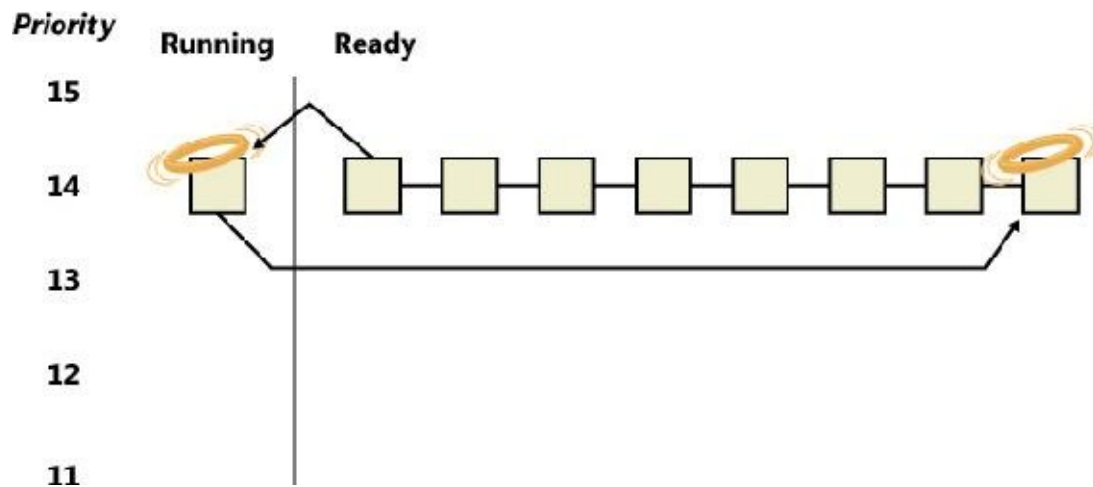


FIGURE 4-16 Quantum-end thread scheduling.

As you saw, instead of simply relying on a clock interval timer-based quantum to schedule threads, Windows uses an accurate CPU clock cycle count to maintain quantum targets. Windows also uses this count to determine whether quantum end is currently appropriate for the thread—something that might have happened previously and is important to discuss.

Using a scheduling model that relies only on the clock interval timer, the following situation can occur:

- Threads A and B become ready to run during the middle of an interval. (Scheduling code runs not just at each clock interval, so this is often the case.)
- Thread A starts running but is interrupted for a while. The time spent handling the interrupt is charged to the thread.
- Interrupt processing finishes and thread A starts running again, but it quickly hits the next clock interval. The scheduler can assume only that thread A had been running all this time and now switches to thread B.
- Thread B starts running and has a chance to run for a full clock interval (barring preemption or interrupt handling).

In this scenario, thread A was unfairly penalized in two different ways. First, the time it spent handling a device interrupt was counted against its own CPU time, even though the thread probably had nothing to do with the interrupt. (Interrupts are handled in the context of whichever thread was running at the time, as discussed in [Chapter 6](#).) It was also unfairly penalized for the time the system was idling inside that clock interval before it was scheduled. [Figure 4-17](#) illustrates this scenario.

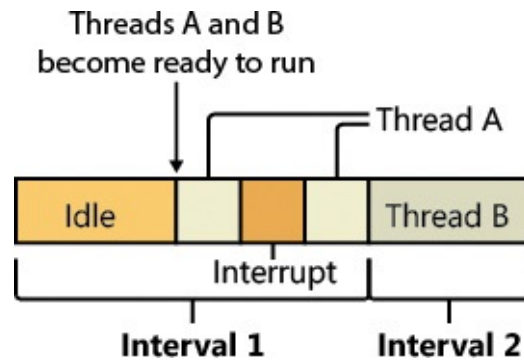


FIGURE 4-17 Unfair time slicing in pre-Vista versions of Windows.

Windows keeps an accurate count of the exact number of CPU clock cycles spent doing work that the thread was scheduled to do (which means excluding interrupts). It also keeps a quantum target of clock cycles that should have been spent by the thread at the end of its quantum. Therefore, both of the unfair decisions that would have been made against thread A as described in the preceding paragraph will not happen in Windows. Instead, the following situation occurs:

- Threads A and B become ready to run during the middle of an interval.
- Thread A starts running but is interrupted for a while. The CPU clock cycles spent handling the interrupt are not charged to the thread.
- Interrupt processing finishes and thread A starts running again, but it quickly hits the next clock interval. The scheduler looks at the number of CPU clock cycles charged to the thread and compares them to the expected CPU clock cycles that should have been charged at quantum end.
- Because the former number is much smaller than it should be, the scheduler assumes that thread A started running in the middle of a clock interval and might have been additionally interrupted.
- Thread A gets its quantum increased by another clock interval, and the quantum target is recalculated. Thread A now has its chance to run for a full clock interval.
- At the next clock interval, thread A has finished its quantum, and thread B now gets a chance to run.

[Figure 4-18](#) illustrates this scenario.

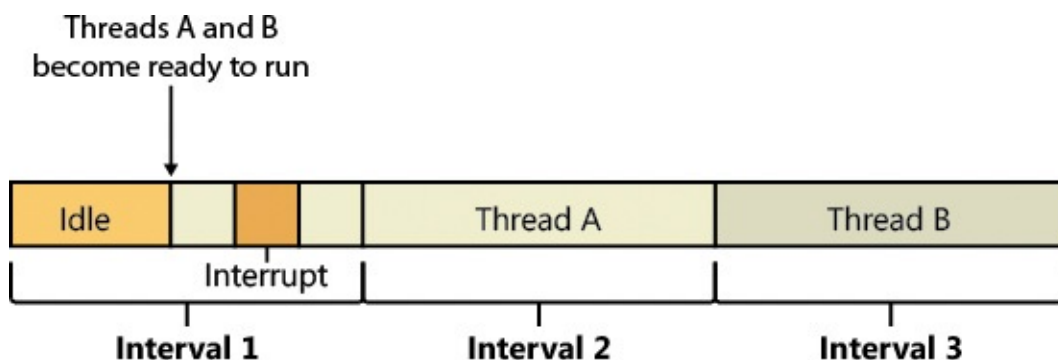


FIGURE 4-18 Fair time slicing in current versions of Windows.

Termination

When a thread finishes running (either because it returned from its main routine, called `ExitThread`, or was killed with `TerminateThread`), it moves from the running state to the terminated state. If there are no handles open on the thread object, the thread is removed from the process thread list and the associated data structures are deallocated and released.

Idle threads

When no runnable thread exists on a CPU, Windows dispatches that CPU's idle thread. Each CPU has its own dedicated idle thread. This is because on a multiprocessor system, one CPU can be executing a thread while other CPUs might have no threads to execute. Each CPU's idle thread is found via a pointer in that CPU's PRCB.

All the idle threads belong to the idle process. The idle process and idle threads are special cases in many ways. They are, of course, represented by `EPROCESS/KPROCESS` and `ETHREAD/KTHREAD` structures, but they are not executive manager processes and thread objects. Nor is the idle process on the system process list. (This is why it does not appear in the output of the kernel debugger's `!process 0 0` command.) However, the idle thread or threads and their process can be found in other ways.

EXPERIMENT: Displaying the structures of the idle threads and idle process

You can find the idle thread and process structures in the kernel debugger via the `!pcr` command. (*PCR* is short for *processor control region*.) This command displays a subset of information from the PCR and from the associated PRCB. `!pcr` takes a single numeric argument, which is the number of the CPU whose PCR is to be displayed. The boot processor is processor 0. It is always present, so `!pcr 0` should always work. The following output shows the results of this command from a local kernel debugging session for a 64-bit, eight-processor system:

[Click here to view code image](#)

```
lkd> !pcr
KPCR for Processor 0 at ffffff80174bd0000:
  Major 1 Minor 1
  NtTib.ExceptionList: ffffff80176b4a000
    NtTib.StackBase: ffffff80176b4b070
    NtTib.StackLimit: 000000000108e3f8
  NtTib.SubSystemTib: ffffff80174bd0000
    NtTib.Version: 0000000074bd0180
    NtTib.UserPointer: ffffff80174bd07f0
    NtTib.SelfTib: 00000098af072000

    SelfPcr: 0000000000000000
    Prcb: ffffff80174bd0180
    Irql: 0000000000000000
    IRR: 0000000000000000
    IDR: 0000000000000000
  InterruptMode: 0000000000000000
    IDT: 0000000000000000
    GDT: 0000000000000000
    TSS: 0000000000000000

  CurrentThread: fffffb882fa27c080
    NextThread: 0000000000000000
    IdleThread: fffff80174c4c940

  DpcQueue:
```

This output shows that CPU 0 was executing a thread other than its idle thread at the time the memory dump was obtained because the CurrentThread and IdleThread pointers are different. (On a multi-CPU system you can try !pcr 1, !pcr 2, and so on, until you run out. Observe that each IdleThread pointer is different.)

Now use the !thread command on the indicated idle thread address:

[Click here to view code image](#)

```
lkd> !thread fffff80174c4c940
THREAD fffff80174c4c940  Cid 0000.0000  Peb: 0000000000000000
Win32Thread:
0000000000000000 RUNNING on processor 0
Not impersonating
DeviceMap                fffff800a52e17ce0
Owning
Process                  fffff80174c4b940      Image:          Idle
Attached
Process                  fffffb882e7ec7640      Image:          System
Wait Start TickCount      1637993          Ticks: 30
(0:00:00:00.468)
Context Switch Count      25908837      IdealProcessor: 0
UserTime                  00:00:00.000
KernelTime                05:51:23.796
Win32 Start Address nt!KiIdleLoop (0xfffff801749e0770)
Stack Init fffff80176b52c90 Current fffff80176b52c20
Base fffff80176b53000 Limit fffff80176b4d000 Call
0000000000000000
Priority 0 BasePriority 0 PriorityDecrement 0 IoPriority 0
PagePriority 5
```

Finally, use the !process command on the owning process shown in the preceding output. For brevity, we'll add a second parameter value of 3, which causes !process to emit only minimal information for each thread:

[Click here to view code image](#)

```
lkd> !process fffff80174c4b940 3
PROCESS fffff80174c4b940
  SessionId: none  Cid: 0000    Peb: 00000000  ParentCid: 0000
  DirBase: 001aa000  ObjectTable:
fffff800a52e14040  HandleCount: 2011.
  Image: Idle
  VadRoot fffffb882e7e1ae70 Vads 1 Clone 0 Private 7. Modified
1627. Locked 0.
  DeviceMap 0000000000000000
  Token                fffff800a52e17040
  ElapsedTime          07:07:04.015
  UserTime              00:00:00.000
  KernelTime            00:00:00.000
```

```

QuotaPoolUsage[PagedPool]          0
QuotaPoolUsage[NonPagedPool]       0
Working Set Sizes (now,min,max)    (7, 50, 450) (28KB, 200KB,
1800KB)
PeakWorkingSetSize                  1
VirtualSize                         0 Mb
PeakVirtualSize                     0 Mb
PageFaultCount                      2
MemoryPriority                       BACKGROUND
BasePriority                         0
CommitCharge                        0

```

```

      THREAD fffff80174c4c940  Cid 0000.0000  Teb:
0000000000000000
Win32Thread: 0000000000000000 RUNNING on processor 0
      THREAD fffff9d81e230ccc0  Cid 0000.0000  Teb:
0000000000000000
Win32Thread: 0000000000000000 RUNNING on processor 1
      THREAD fffff9d81e1bd9cc0  Cid 0000.0000  Teb:
0000000000000000
Win32Thread: 0000000000000000 RUNNING on processor 2
      THREAD fffff9d81e2062cc0  Cid 0000.0000  Teb:
0000000000000000
Win32Thread: 0000000000000000 RUNNING on processor 3
      THREAD fffff9d81e21a7cc0  Cid 0000.0000  Teb:
0000000000000000
Win32Thread: 0000000000000000 RUNNING on processor 4
      THREAD fffff9d81e22ebcc0  Cid 0000.0000  Teb:
0000000000000000
Win32Thread: 0000000000000000 RUNNING on processor 5
      THREAD fffff9d81e2428cc0  Cid 0000.0000  Teb:
0000000000000000
Win32Thread: 0000000000000000 RUNNING on processor 6
      THREAD fffff9d81e256bcc0  Cid 0000.0000  Teb:
0000000000000000
Win32Thread: 0000000000000000 RUNNING on processor 7

```

These process and thread addresses can also be used with `dt nt!_EPROCESS`, `dt nt!_KTHREAD`, and other such commands.

The preceding experiment shows some of the anomalies associated with the idle process and its threads. The debugger indicates an **Image** name of **Idle** (which comes from the **EPROCESS** structure's **ImageFileName** member), but various Windows utilities report the idle process using different names. Task Manager and Process Explorer call it *System Idle Process*, while `tlst` calls it *System Process*. The process ID and thread IDs (the client IDs, or **Cid** in the debugger's output) are 0, as are the PEB and TEB pointers and potentially many other fields in the idle process or its threads. Because the idle process has no user-mode address space and its threads execute no user-mode code, they have no need of the

various data required to manage a user-mode environment. Also, the idle process is not an object-manager process object, and its idle threads are not object-manager thread objects. Instead, the initial idle thread and idle process structures are statically allocated and used to bootstrap the system before the process manager and the object manager are initialized. Subsequent idle thread structures are allocated dynamically (as simple allocations from a non-paged pool, bypassing the object manager) as additional processors are brought online. Once process management initializes, it uses the special variable `PsIdleProcess` to refer to the idle process.

Perhaps the most interesting anomaly regarding the idle process is that Windows reports the priority of the idle threads as 0. In reality, however, the values of the idle threads' priority members are irrelevant because these threads are selected for dispatching only when there are no other threads to run. Their priority is never compared with that of any other thread. Nor is it used to put an idle thread on a ready queue, as idle threads are never part of any ready queues. (Only one thread per Windows system is actually running at priority 0—the zero page thread, explained in [Chapter 5](#).)

Just as the idle threads are special cases in terms of selection for execution, they are also special cases for preemption. The idle thread's routine, `KiIdleLoop`, performs a number of operations that preclude its being preempted by another thread in the usual fashion. When no non-idle threads are available to run on a processor, that processor is marked as idle in its `PRCB`. After that, if a thread is selected for execution on the idle processor, the thread's address is stored in the `NextThread` pointer of the idle processor's `PRCB`. The idle thread checks this pointer on each pass through its loop.

Although some details of the flow vary between architectures (this is one of the few routines written in assembly and not in C), the basic sequence of operations of the idle thread is as follows:

1. The idle thread briefly enables interrupts, allowing any pending interrupts to be delivered, and then disables them (using the `STI` and `CLI` instructions on x86 and x64 processors). This is desirable because significant parts of the idle thread execute with interrupts disabled.
2. On the debug build on some architectures, the idle thread checks whether there is a kernel debugger trying to break into the system. If so, it gives it access.
3. The idle thread checks whether any DPCs (described in [Chapter 6](#)) are pending on the processor. DPCs could be pending if a DPC interrupt was not generated when they were queued. If DPCs are pending, the idle loop calls `KiRetiredDpcList` to deliver them. This will also perform timer expiration, as well as deferred ready processing; the latter is explained in the upcoming [“Multiprocessor systems”](#) section. `KiRetiredDpcList` must be entered with interrupts disabled, which is why interrupts are left disabled at the end of step 1. `KiRetiredDpcList` exits with interrupts disabled as well.
4. The idle thread checks whether quantum end processing has been requested. If so, `KiQuantum-End` is called to process the request.
5. The idle thread checks whether a thread has been selected to run next on the processor. If so, it dispatches that thread. This could be the case if, for example, a DPC or timer expiration processed in step 3 resolved the wait of a waiting thread, or if another processor selected a thread for this processor to run while it was already in the idle loop.
6. If requested, the idle thread checks for threads ready to run on other processors and, if possible, schedules one of them locally. (This operation is explained in the upcoming [“Idle scheduler”](#) section).
7. The idle thread calls the registered power-management processor idle routine (in case any power-management functions need to be performed), which is either in the processor power driver (such as

intelppm.sys) or in the HAL if such a driver is unavailable.

Thread suspension

Threads can be suspended and resumed explicitly with the `SuspendThread` and `ResumeThread` API functions, respectively. Every thread has a suspend count, which is incremented by suspension and decremented by resuming. If the count is 0, the thread is free to execute. Otherwise, it will not execute.

Suspension works by queuing a kernel APC to the thread. When the thread is switched in to execute, the APC is executed first. This puts the thread in a wait state on event that is signaled when the thread is finally resumed.

This suspension mechanism has a noticeable drawback if the thread is in a wait state while a suspension request comes in, because it means that the thread needs to wake up just to be suspended. This can result in a kernel stack inswap (if the thread's kernel stack was swapped out). Windows 8.1 and Server 2012 R2 added a mechanism called *Lightweight Suspend* to allow for the suspension of a thread that is in a wait state not by using the APC mechanism, but by directly manipulating the thread's object in memory and marking it as suspended.

(Deep) freeze

Freezing is a mechanism by which processes enter a suspended state that cannot be changed by calling `ResumeThread` on threads in the process. This is useful when the system needs to suspend a UWP app. This happens when a Windows app goes to the background—for example, because another app comes to the foreground in Tablet mode or the app is minimized in Desktop mode. In this case, the system gives to the app roughly five seconds to do work, typically to save application state. Saving state is important because Windows apps may be killed without any notice if memory resources become low. If the app is killed, the state can be reloaded on the next launch and the user would have the perception that the app never really went away. Freezing a process means suspending all threads in such a way that `ResumeThread` is not able to wake. A flag in the `KTHREAD` structure indicates whether a thread is frozen. For a thread to be able to execute, its suspend count must be 0 and the `frozen` flag must be clear.

Deep freeze adds another constraint: Newly created threads in the process cannot start as well. For example, if a call to `CreateRemoteThreadEx` is used to create a new thread in a deep-frozen process, the thread will be frozen before actually starting. This is the typical usage of the freezing capability.

Process- and thread-freezing functionality is not exposed directly to user mode. It is used internally by the Process State Manager (PSM) service that is responsible for issuing the requests to the kernel for deep freezing and thawing (unfreezing).

You can also freeze processes using jobs. The ability to freeze and unfreeze a job is not publicly documented, but it's possible to do using the standard `NtSetInformationJobObject` system call. This is typically used for Windows apps, as all Windows apps processes are contained in jobs. Such a job may contain a single process (the Windows app itself), but it can also contain background task-hosting processes related to the same Window app so that freezing or thawing (unfreezing) all processes under that job can be done in a single stroke. (See Chapter 8 in Part 2 for more on Windows apps.)

EXPERIMENT: Deep freeze

In this experiment, you'll watch deep freeze happening by debugging a virtual machine.

1. Open WinDbg with admin privileges and attach to a virtual machine running Windows 10.

2. Press **Ctrl+Break** to break into the VM.

3. Set a breakpoint when deep freeze begins with a command to show the process that is frozen:

[Click here to view code image](#)

```
bp nt!PsFreezeProcess "!process -1 0; g"
```

4. Enter the **g** (go) command or press **F5**. You should see many deep freeze occurrences.

5. Start the Cortana UI from the taskbar and then close the UI. After about 5 seconds you should see something like the following:

[Click here to view code image](#)

```
PROCESS 8f518500 SessionId: 2 Cid: 12c8 Peb:
03945000 ParentCid: 02ac
DirBase: 054007e0 ObjectTable: b0a8a040 HandleCount: 988.
Image: SearchUI.exe
```

6. Now break into the debugger and show more info on that process:

[Click here to view code image](#)

```
1: kd> !process 8f518500 1
PROCESS 8f518500 SessionId: 2 Cid: 12c8 Peb:
03945000 ParentCid: 02ac
DeepFreeze
DirBase: 054007e0 ObjectTable: b0a8a040 HandleCount: 988.
Image: SearchUI.exe
VadRoot 95c1ffd8 Vads 405 Clone 0 Private 7682. Modified
201241. Locked 0.
DeviceMap a12509c0
Token b0a65bd0
ElapsedTime 04:02:33.518
UserTime 00:00:06.937
KernelTime 00:00:00.703
QuotaPoolUsage[PagedPool] 562688
QuotaPoolUsage[NonPagedPool] 34392
Working Set Sizes (now,min,max) (20470, 50, 345) (81880KB,
200KB, 1380KB)
PeakWorkingSetSize 25878
VirtualSize 367 Mb
PeakVirtualSize 400 Mb
PageFaultCount 307764
MemoryPriority BACKGROUND
BasePriority 8
CommitCharge 8908
Job 8f575030
```

7. Notice the **DeepFreeze** attribute written by the debugger. Also notice that the process is part of a job. Use the **!job** command to see more details:

[Click here to view code image](#)

```
1: kd> !job 8f575030
```

```
Job at 8f575030
```

Basic Accounting Information

```
TotalUserTime:          0x0
TotalKernelTime:         0x0
TotalCycleTime:          0x0
ThisPeriodTotalUserTime: 0x0
ThisPeriodTotalKernelTime: 0x0
TotalPageFaultCount:     0x0
TotalProcesses:          0x1
ActiveProcesses:         0x1
FreezeCount:              1
BackgroundCount:         0
TotalTerminatedProcesses: 0x0
PeakJobMemoryUsed:        0x38e2
PeakProcessMemoryUsed:    0x38e2
```

Job Flags

```
[cpu rate control]
[frozen]
[wake notification allocated]
[wake notification enabled]
[timers virtualized]
[job swapped]
```

```
Limit Information (LimitFlags: 0x0)
```

```
Limit Information (EffectiveLimitFlags: 0x3000)
```

CPU Rate Control

```
Rate = 100.00%
Scheduling Group: a469f330
```

8. The job is under CPU rate control (see the section “[CPU rate limits](#)” later in this chapter for more on CPU rate control) and is frozen. Detach from the VM and close the debugger.

Thread selection

Whenever a logical processor needs to pick the next thread to run, it calls the `KiSelectNextThread` scheduler function. This can happen in a variety of scenarios:

- A hard affinity change has occurred, making the currently running or standby thread ineligible for execution on its selected logical processor. Therefore, another must be chosen.
- The currently running thread reached its quantum end, and the Symmetric Multithreading (SMT) set it was running on has become busy while other SMT sets within the ideal node are fully idle. (*Symmetric Multithreading* is the technical name for the hyper-threading technology described in [Chapter 2](#).) The scheduler performs a quantum-end migration of the current thread, so another must be chosen.
- A wait operation has finished, and there were pending scheduling operations in the wait status register (in other words, the priority and/or affinity bits were set).

In these scenarios, the behavior of the scheduler is as follows:

- The scheduler calls `KiSelectReadyThreadEx` to search for the next ready thread that the processor should run and check whether one was found.
- If a ready thread was not found, the idle scheduler is enabled, and the idle thread is selected for execution. If a ready thread *was* found, it is put in the ready state in the local or shared ready queue, as appropriate.

The `KiSelectNextThread` operation is performed only when the logical processor needs to pick—but not yet run—the next schedulable thread (which is why the thread will enter the Ready state). Other times, however, the logical processor is interested in immediately running the next ready thread or performing another action if one is not available (instead of going idle), such as when the following occurs:

- A priority change causes the current standby or running thread to no longer be the highest-priority ready thread on its selected logical processor, meaning that a higher priority ready thread must now run.
- The thread has explicitly yielded with `YieldProcessor` or `NtYieldExecution` and another thread might be ready for execution.
- The quantum of the current thread has expired, and other threads at the same priority level need their chance to run as well.
- A thread has lost its priority boost, causing a similar priority change to the scenario just described.
- The idle scheduler is running and needs to check whether a ready thread has not appeared in the interval between when the idle scheduling was requested and the idle scheduler ran.

A simple way to remember the difference between which routine runs is to check whether the logical processor *must* run a different thread (in which case `KiSelectNextThread` is called) or if it *should if possible* run a different thread (in which case `KiSelectReadyThreadEx` is called). In either case, because each processor belongs to a shared ready queue (pointed to by the `KPRCB`), `KiSelectReady-ThreadEx` can simply check the current logical processor's (LP's) queues, removing the first highest-priority thread that it finds unless this priority is lower than the one of the currently running thread (depending on whether the current thread is still allowed to run, which would not be the case in the `KiSelectNextThread` scenario). If there is no higher-priority thread (or no threads are ready at all), no thread is returned.

Idle scheduler

Whenever the idle thread runs, it checks whether idle scheduling has been enabled. If so, the idle thread begins scanning other processors' ready queues for threads it can run by calling `KiSearchForNewThread`. The run-time costs associated with this operation are not charged as idle thread time, but are instead charged as interrupt and DPC time (charged to the processor), so idle scheduling time is considered system time. The `KiSearchForNewThread` algorithm, which is based on the functions described earlier in this section, is explained shortly.

Multiprocessor systems

On a uniprocessor system, scheduling is relatively simple: The highest-priority thread that wants to run is always running. On a multiprocessor system, it is more complex. This is because Windows attempts to schedule threads on the most optimal processor for the thread, taking into account the thread's preferred and previous processors as well as the configuration of the multiprocessor system. Therefore, although Windows attempts to schedule the highest-priority runnable threads on all available CPUs, it guarantees only to be running one of the highest-priority threads somewhere. With shared ready queues (for threads with no affinity restrictions), the guarantee is stronger. Each shared group of processors is running at least one of the highest-priority threads.

Before we describe the specific algorithms used to choose which threads run where and when, let's examine the additional information Windows maintains to track thread and processor state on multiprocessor systems and the three different types of multiprocessor systems supported by Windows (SMT, multicore, and NUMA).

Package sets and SMT sets

Windows uses five fields in the KPRCB to determine correct scheduling decisions when dealing with logical processor topologies. The first field, `CoresPerPhysicalProcessor`, determines whether this logical processor is part of a multicore package. It's computed from the `CPUID` returned by the processor and rounded to a power of 2. The second field, `LogicalProcessorsPerCore`, determines whether the logical processor is part of an SMT set, such as on an Intel processor with hyper-threading enabled, and is also queried through `CPUID` and rounded. Multiplying these two numbers yields the number of logical processors per package, or an actual physical processor that fits into a socket. With these numbers, each PRCB can then populate its `PackageProcessorSet` value. This is the affinity mask describing which other logical processors within this group (because packages are constrained to a group) belong to the same physical processor. Similarly, the `CoreProcessorSet` value connects other logical processors to the same core, also called an *SMT set*. Finally, the `GroupSetMember` value defines which bitmask within the current processor group identifies this very logical processor. For example, the logical processor 3 normally has a `GroupSetMember` value of 8 (which equals 2 to the third power).

EXPERIMENT: Viewing logical processor information

You can examine the information Windows maintains for SMT processors using the `!smt` command in the kernel debugger. The following output is from a quad-core Intel Core i7 system with SMT (eight logical processors):

[Click here to view code image](#)

```
lkd> !smt
SMT Summary:
-----

KeActiveProcessors:
*****_-----
(0000000000000000ff)
IdleSummary:
_****_ *_-----
(00000000000000009e)
  No PRCB          SMT
Set                                     APIC Id
  0 ffffff803d7546180 **_-----
(000000000000000003) 0x00000000
  1 fffffba01cb31a180 **_-----
(000000000000000003) 0x00000001
  2 fffffba01cb3dd180 --*_-----
(00000000000000000c) 0x00000002
  3 fffffba01cb122180 --*_-----
(00000000000000000c) 0x00000003
  4 fffffba01cb266180 ---*_-----
(000000000000000030) 0x00000004
  5 fffffba01cabd6180 ---*_-----
(000000000000000030) 0x00000005
  6 fffffba01cb491180 -----**_-----
(0000000000000000c0) 0x00000006
  7 fffffba01cb5d4180 -----**_-----
(0000000000000000c0) 0x00000007

Maximum cores per physical processor:      8
Maximum logical processors per core:       2
```

NUMA systems

Another type of multiprocessor system supported by Windows is one with a non-uniform memory architecture (NUMA). In a NUMA system, processors are grouped together in smaller units called *nodes*. Each node has its own processors and memory and is connected to the larger system through a cache-coherent interconnect bus. These systems are called *non-uniform* because each node has its own local high-speed memory. Although any processor in any node can access all of memory, node-local memory is much faster to access.

The kernel maintains information about each node in a NUMA system in a data structure called **KNODE**. The kernel variable **KeNodeBlock** is an array of pointers to the **KNODE** structures for each node. You can reveal the format of the **KNODE** structure using the **dt** command in the kernel debugger, as shown here:

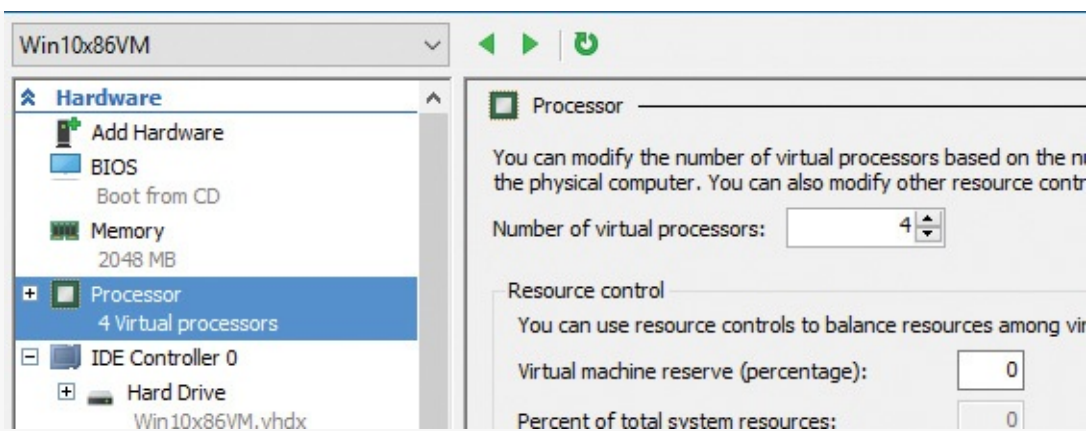
[Click here to view code image](#)

```
lkd> dt nt!_KNODE
+0x000 IdleNonParkedCpuSet : Uint8B
+0x008 IdleSmtSet          : Uint8B
+0x010 IdleCpuSet          : Uint8B
+0x040 DeepIdleSet         : Uint8B
+0x048 IdleConstrainedSet  : Uint8B
+0x050 NonParkedSet        : Uint8B
+0x058 ParkLock            : Int4B
+0x05c Seed                : Uint4B
+0x080 SiblingMask         : Uint4B
+0x088 Affinity             : _GROUP_AFFINITY
+0x088 AffinityFill        : [10] UChar
+0x092 NodeNumber          : Uint2B
+0x094 PrimaryNodeNumber   : Uint2B
+0x096 Stride              : UChar
+0x097 Spare0              : UChar
+0x098 SharedReadyQueueLeaders : Uint8B
+0x0a0 ProximityId         : Uint4B
+0x0a4 Lowest              : Uint4B
+0x0a8 Highest             : Uint4B
+0x0ac MaximumProcessors   : UChar
+0x0ad Flags               : _flags
+0x0ae Spare10             : UChar
+0x0b0 HeteroSets          : [5] _KHETERO_PROCESSOR_SET
```

EXPERIMENT: Viewing NUMA information

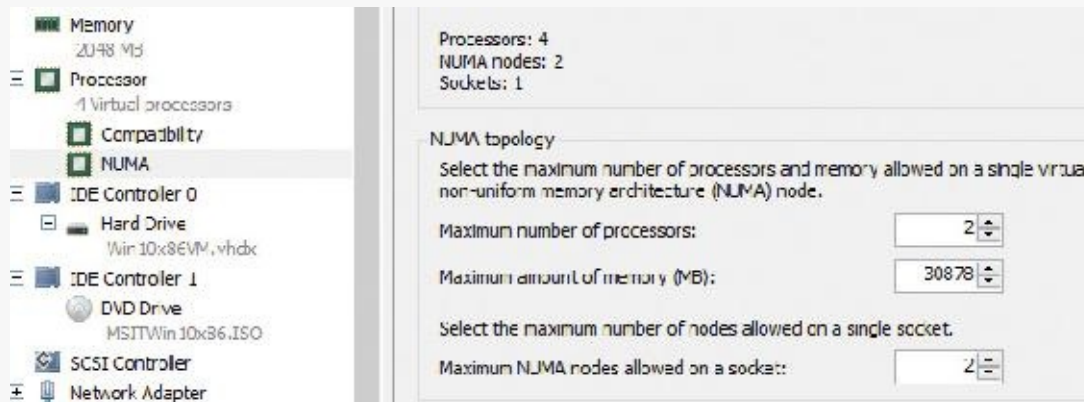
You can examine the information Windows maintains for each node in a NUMA system using the **!numa** command in the kernel debugger. To experiment with NUMA systems even when such hardware is not available, it's possible to configure a Hyper-V virtual machine to include more than one NUMA node that the guest VM will use. To configure a Hyper-V VM to use NUMA, do the following. (You will need a host machine with more than four logical processors.)

1. Click **Start**, type **hyper**, and click the **Hyper-V Manager** option.
2. Make sure the VM is powered off. Otherwise the following changes cannot be made.
3. Right-click the VM in Hyper-V Manager and select **Settings** to open the VM's settings.
4. Click the **Memory** node and make sure **Dynamic Memory** is unchecked.
5. Click the **Processor** node and enter **4** in the **Number of Virtual Processors** box:



6. Expand the **Processor** node and select the **NUMA** sub-node.

7. Enter 2 in the **Maximum Number of Processors** and **Maximum NUMA Nodes Allowed on a Socket** boxes:



8. Click **OK** to save the settings.

9. Power up the VM.

10. Use a kernel debugger to issue the **!numa** command. Here's an example of output for the previously configured VM:

[Click here to view code image](#)

```
2: kd> !numa
NUMA Summary:
-----
    Number of NUMA nodes : 2
    Number of Processors : 4
unable to get nt!MmAvailablePages
MmAvailablePages      : 0x00000000
KeActiveProcessors    :
*** ----- (0000000f)

    NODE 0 (FFFFFFFF820510C0):
    Group              : 0 (Assigned, Committed, Assignment
Adjustable)
    ProcessorMask      : ** -----
(00000003)
    ProximityId        : 0
    Capacity           : 2
    Seed               : 0x00000001
```

```

IdleCpuSet      : 00000003
IdleSmtSet      : 00000003
NonParkedSet    : 00000003
Unable to get MiNodeInformation

    NODE 1 (FFFFFFFF8719E0C0):
    Group        : 0 (Assigned, Committed, Assignment
Adjustable)
    ProcessorMask : --**-----
(0000000c)
    ProximityId   : 1
    Capacity      : 2
    Seed          : 0x00000003
    IdleCpuSet    : 00000008
    IdleSmtSet    : 00000008
    NonParkedSet  : 0000000c
Unable to get MiNodeInformation

```

Applications that want to gain the most performance out of NUMA systems can set the affinity mask to restrict a process to the processors in a specific node, although Windows already restricts nearly all threads to a single NUMA node due to its NUMA-aware scheduling algorithms.

How the scheduling algorithms account for NUMA systems is covered later in this chapter in the section “[Processor selection](#).” (The optimizations in the memory manager to take advantage of node-local memory are covered in [Chapter 5](#).)

Processor group assignment

While querying the topology of the system to build the various relationships between logical processors, SMT sets, multicore packages, and physical sockets, Windows assigns processors to an appropriate group that will describe their affinity (through the extended affinity mask seen earlier). This work is done by the `KePerformGroupConfiguration` routine, which is called during initialization before any other phase 1 work is done. The steps of this process are as follows:

1. The function queries all detected nodes (`KeNumberNodes`) and computes the capacity of each node—that is, how many logical processors can be part of the node. This value is stored in `MaximumProcessors` in the `KeNodeBlock` array, which identifies all NUMA nodes on the system. If the system supports NUMA proximity IDs, the proximity ID is queried for each node and saved in the node block.
2. The NUMA distance array is allocated (`KeNodeDistance`) and the distance between each NUMA node is computed.

The next series of steps deal with specific user-configuration options that override default NUMA assignments. For example, consider a system with Hyper-V installed and with the hypervisor configured to auto-start. If the CPU does not support the extended hypervisor interface, then only one processor group will be enabled, and all NUMA nodes (that can fit) will be associated with group 0. Therefore, in this case, Hyper-V cannot take advantage of machines with more than 64 processors.

3. The function checks whether any static group assignment data was passed by the loader (and thus

configured by the user). This data specifies the proximity information and group assignment for each NUMA node.



Note

Users dealing with large NUMA servers who might need custom control of proximity information and group assignments for testing or validation purposes can enter this data through the **Group Assignment** and **Node Distance** registry values. These are found in the **HKLM\SYSTEM\CurrentControlSet\Control\NUMA** registry key. The exact format of this data includes a count followed by an array of proximity IDs and group assignments, which are all 32-bit values.

4. Before treating this data as valid, the kernel queries the proximity ID to match the node number and then associates group numbers as requested. It then makes sure that NUMA node 0 is associated with group 0, and that the capacity for all NUMA nodes is consistent with the group size. Finally, the function checks how many groups still have remaining capacity.
-



Note

NUMA node 0 is always assigned to group 0, no matter what.

5. The kernel dynamically attempts to assign NUMA nodes to groups while respecting any statically configured nodes if passed in as just described. Normally, the kernel tries to minimize the number of groups created, combining as many NUMA nodes as possible per group. However, if this behavior is not desired, it can be configured differently with the **/MAXGROUP** loader parameter, configured through the **maxgroup** BCD option. Turning this value on overrides the default behavior and causes the algorithm to spread as many NUMA nodes as possible into as many groups as possible, while respecting that the currently implemented group limit is 20. If there is only one node, or if all nodes can fit into a single group (and **maxgroup** is off), the system performs the default setting of assigning all nodes to group 0.
6. If there is more than one node, Windows checks the static NUMA node distances (if any). It then sorts all the nodes by their capacity so that the largest nodes come first. In the group-minimization mode, the kernel figures out the maximum processors there can be by adding up all the capacities. By dividing that by the number of processors per group, the kernel assumes there will be this many total groups on the machine (limited to a maximum of 20). In group-maximization mode, the initial estimate is that there will be as many groups as nodes (limited again to 20).
7. The kernel begins the final assignment process. All fixed assignments from earlier are now committed and groups are created for those assignments.
8. All the NUMA nodes are reshuffled to minimize the distance between the different nodes within a group. In other words, closer nodes are put in the same group and sorted by distance.
9. The same process is performed for any dynamically configured node to group assignments.
10. Any remaining empty nodes are assigned to group 0.

Logical processors per group

Generally, Windows assigns 64 processors per group. But you can also customize this configuration by using different load options such as the `/GROUPSIZE` option, which is configured through the `groupsize` BCD element. By specifying a number that is a power of 2, you can force groups to contain fewer processors than normal for purposes such as testing group awareness in the system. For example, a system with eight logical processors can be made to appear to have one, two, or four groups. To force the issue, the `/FORCEGROUPAWARE` option (BCD element `groupaware`) causes the kernel to avoid group 0 whenever possible, assigning the highest group number available in actions such as thread and DPC affinity selection and process group assignment. You should avoid setting a group size of 1 because this will force almost all applications on the system to behave as if they're running on a uniprocessor machine. This is because the kernel sets the affinity mask of a given process to span only one group until the application requests otherwise (which most applications will not do).

In the edge case where the number of logical processors in a package cannot fit into a single group, Windows adjusts these numbers so that a package can fit into a single group. It does so by shrinking the `CoresPerPhysicalProcessor` number and, if the SMT cannot fit, the `LogicalProcessorsPerCore` number. The exception to this rule is if the system actually contains multiple NUMA nodes within a single package (uncommon, but possible). In these multi-chip modules (MCMs), two sets of cores as well as two memory controllers are on the same die/package. If the ACPI Static Resource Affinity Table (SRAT) defines the MCM as having two NUMA nodes, Windows might associate the two nodes with two different groups (depending on group-configuration algorithms). In this scenario, the MCM package would span more than one group.

Other than causing significant driver and application compatibility problems—which they are designed to identify and root out, when used by developers—these options have an even greater impact on the machine: They force NUMA behaviors even on a non-NUMA machine. This is because Windows will never allow a NUMA node to span multiple groups, as was shown in the assignment algorithms. So, if the kernel is creating artificially small groups, those two groups must each have their own NUMA node. For example, on a quad-core processor with a group size of 2, this will create two groups, and thus two NUMA nodes, which will be subnodes of the main node. This will affect scheduling and memory-management policies in the same way a true NUMA system would, which can be useful for testing.

Logical processor state

In addition to the shared and local ready queues and summaries, Windows maintains two bitmasks that track the state of the processors on the system. (How these bitmasks are used is explained in the upcoming “[Processor selection](#)” section.) Following are the bitmasks that Windows maintains:

- **KeActiveProcessors** This is the active processor mask, which has a bit set for each usable processor on the system. These might be fewer than the number of actual processors if the licensing limits of the version of Windows running supports fewer than the number of available physical processors. Use the `KeRegisteredProcessors` variable to see how many processors are actually licensed on the machine. In this instance, *processors* refers to physical packages.
- **KeMaximumProcessors** This is the maximum number of logical processors (including all future possible dynamic processor additions) bounded within the licensing limit. It also reveals any platform limitations that are queried by calling the HAL and checking with the ACPI SRAT table, if any.

Part of the node's data (KNODE) is the set of idle CPUs in this node (the `IdleCpuSet` member), idle CPUs that are not parked (`IdleNonParkedCpuSet`), and idle SMT sets (`IdleSmtSet`).

Scheduler scalability

On a multiprocessor system, one processor might need to modify another processor's per-CPU scheduling data structures—for example, inserting a thread that would like to run on a certain processor. For this reason, you synchronize these structures by using a per-PRCB queued spinlock, which is held at `DISPATCH_LEVEL`. Thus, thread selection can occur while locking only an individual processor's PRCB. If needed, one more processor's PRCB can also be locked, such as in scenarios of thread stealing (described later). Thread context switching is also synchronized by using a finer-grained per-thread spinlock.

There is also a per-CPU list of threads in the deferred ready state (`DeferredReadyListHead`). These represent threads that are ready to run but have not yet been readied for execution; the actual ready operation has been deferred to a more appropriate time. Because each processor manipulates only its own per-processor deferred ready list, this list is not synchronized by the PRCB spinlock. The deferred ready thread list is processed by `KiProcessDeferredReadyList` after a function has already done modifications to process or thread affinity, priority (including due to priority boosting), or quantum values.

This function calls `KiDeferredReadyThread` for each thread on the list, which performs the algorithm shown later in this chapter in the “[Processor selection](#)” section. This could cause the thread to run immediately; to be put on the ready list of the processor; or, if the processor is unavailable, to be potentially put on a different processor's deferred ready list, in a standby state or immediately executed. This property is used by the core parking engine when parking a core: All threads are put into the deferred ready list, and it is then processed. Because `KiDeferredReadyThread` skips parked cores (as will be shown), it causes all of this processor's threads to wind up on other processors.

Affinity

Each thread has an affinity mask that specifies the processors on which the thread is allowed to run. The thread affinity mask is inherited from the process affinity mask. By default, all processes (and therefore all threads) begin with an affinity mask that is equal to the set of all active processors on their assigned group. In other words, the system is free to schedule all threads on any available processor within the group associated with the process. However, to optimize throughput, partition workloads to a specific set of processors, or both, applications can choose to change the affinity mask for a thread. This can be done at several levels:

- By calling the `SetThreadAffinityMask` function to set the affinity for an individual thread.
- By calling the `SetProcessAffinityMask` function to set the affinity for all the threads in a process.
- Task Manager and Process Explorer provide a GUI to this function. To access it, right-click a process and choose **Set Affinity**. In addition, the Psexec tool (from Sysinternals) provides a command-line interface to this function. (See the `-a` switch in its help output.)
- By making a process a member of a job that has a job-wide affinity mask set using the `SetInformationJobObject` function (described in [Chapter 3](#)).
- By specifying an affinity mask in the image header when compiling the application.



Tip

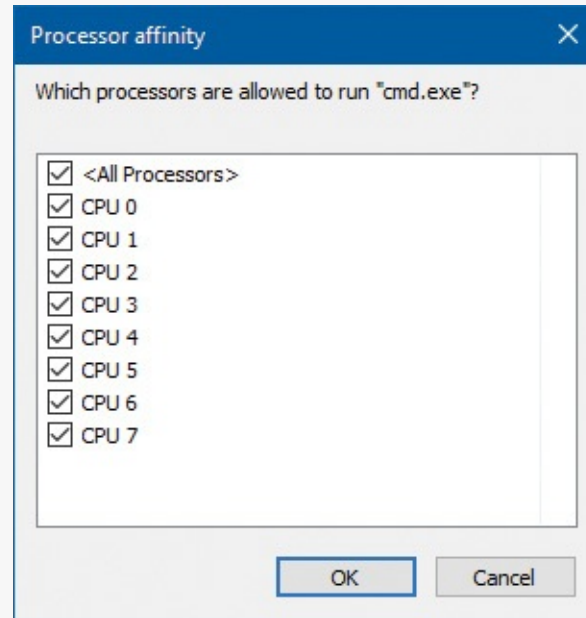
For a detailed specification of the Windows images format, search for *Portable Executable and Common Object File Format Specification* on <http://msdn.microsoft.com>.

An image can also have the `uniprocessor` flag set at link time. If this flag is set, the system chooses a single processor at process-creation time (`MmRotatingProcessorNumber`) and assigns that as the process affinity mask, starting with the first processor and then going round-robin across all the processors within the group. For example, on a dual-processor system, the first time an image marked with the `uniprocessor` flag is launched, it is assigned to CPU 0; the second time, CPU 1; the third time, CPU 0; the fourth time, CPU 1; and so on. This flag can be useful as a temporary workaround for programs that have multithreaded synchronization bugs that surface on multiprocessor systems due to race conditions but not on uniprocessor systems. If an image exhibits such symptoms and is unsigned, you can add the flag manually editing the image header with a Portable Executable (PE) image-editing tool. A better solution, also compatible with signed executables, is to use the Microsoft Application Compatibility Toolkit and add a shim to force the compatibility database to mark the image as uniprocessor-only at launch time.

EXPERIMENT: Viewing and changing process affinity

In this experiment, you will modify the affinity settings for a process and see that process affinity is inherited by new processes:

1. Run the command prompt (Cmd.exe).
2. Run Task Manager or Process Explorer and find the Cmd.exe process in the process list.
3. Right-click the process and select **Set Affinity**. A list of processors should be displayed. For example, on a system with eight logical processes, you will see this:



4. Select a subset of the available processors on the system and click **OK**. The process's threads are now restricted to run on the processors you just selected.
5. At the command prompt, type **Notepad** to run Notepad.exe.
6. Go back to Task Manager or Process Explorer and find the new Notepad process.
7. Right-click the process and choose **Affinity**. You should see the same list of processors you chose for the command-prompt process. This is because processes inherit their affinity settings from their parent.

Windows won't move a running thread that could run on a different processor from one CPU to a second processor to permit a thread with an affinity for the first processor to run on the first processor. For example, consider this scenario: CPU 0 is running a priority 8 thread that can run on any processor, and CPU 1 is running a priority 4 thread that can run on any processor. A priority 6 thread that can run on only CPU 0 becomes ready. What happens? Windows won't move the priority 8 thread from CPU 0 to CPU 1 (preempting the priority 4 thread) so that the priority 6 thread can run; the priority 6 thread must stay in the ready state. Therefore, changing the affinity mask for a process or thread can result in threads getting less CPU time than they normally would because Windows is restricted from running the thread on certain processors. Therefore, setting affinity should be done with extreme care. In most cases, it is optimal to let Windows decide which threads run where.

Extended affinity mask

To support more than 64 processors, which is the limit enforced by the original affinity mask structure (composed of 64 bits on a 64-bit system), Windows uses an extended affinity mask, `KAFFINITY_EX`. This is an array of affinity masks, one for each supported processor group (currently defined at 20). When the scheduler needs to refer to a processor in the extended affinity masks, it first de-references the correct bitmask by using its group number and then accesses the resulting affinity directly. In the kernel API, extended affinity masks are not exposed; instead, the caller of the API inputs the group number as a parameter and receives the legacy affinity mask for that group. In the Windows API, on the other hand, only information about a single group can usually be queried, which is the group of the currently running thread (which is fixed).

The extended affinity mask and its underlying functionality are also how a process can escape the boundaries of its original assigned processor group. By using the extended affinity APIs, threads in a process can choose affinity masks on other processor groups. For example, if a process has four threads and the machine has 256 processors, thread 1 can run on processor 4, thread 2 can run on processor 68, thread 3 on processor 132, and thread 4 on processor 196, if each thread set an affinity mask of 0x10 (0b10000 in binary) on groups 0, 1, 2, and 3. Alternatively, the threads can each set an affinity of all 1 bits (0xFFFF...) for their given group, and the process then can execute its threads on any available processor on the system (with the limitation that each thread is restricted to running within its own group only).

You can take advantage of extended affinity at creation time by specifying a group number in the thread attribute list (`PROC_THREAD_ATTRIBUTE_GROUP_AFFINITY`) when creating a new thread or by calling `SetThreadGroupAffinity` on an existing thread.

System affinity mask

Windows drivers usually execute in the context of the calling thread or an arbitrary thread (that is, not in the safe confines of the System process). Therefore, currently running driver code might be subject to affinity rules set by the application developer. These are not currently relevant to the driver code and might even prevent correct processing of interrupts and other queued work. Driver developers therefore have a mechanism to temporarily bypass user thread affinity settings, by using the `KeSetSystemAffinityThread(Ex)/KeSetSystemGroupAffinityThread` and `KeRevertToUser-AffinityThread(Ex)/KeRevertToUserGroupAffinityThread` APIs.

Ideal and last processor

Each thread has three CPU numbers stored in the kernel thread control block:

- **Ideal processor** This is the preferred processor that this thread should run on.
- **Last processor** This is the processor the thread last ran on.
- **Next processor** This is the processor that the thread will be or is already running on.

The ideal processor for a thread is chosen when a thread is created using a seed in the process control block. The seed is incremented each time a thread is created so that the ideal processor for each new thread in the process rotates through the available processors on the system. For example, the first thread in the first process on the system is assigned an ideal processor of 0 and the second thread in that process is assigned an ideal processor of 1. However, the next process in the system has its first thread's ideal processor set to 1, the second to 2, and so on. In that way, the threads within each process are spread across the processors. On SMT systems (hyper-threading), the next ideal processor is selected from the

next SMT set. For example, on a quad-core, hyper-threaded system, ideal processors for threads in a certain process could be 0, 2, 4, 6, 0, ...; 3, 5, 7, 1, 3, ...; etc. In this way, the threads are spread evenly across the physical processors.

Note that this assumes the threads within a process are doing an equal amount of work. This is typically not the case in a multithreaded process, which normally has one or more housekeeping threads and numerous worker threads. Therefore, a multithreaded application that wants to take full advantage of the platform might find it advantageous to specify the ideal processor numbers for its threads by using the `SetThreadIdealProcessor` function. To take advantage of processor groups, developers should call `SetThreadIdealProcessorEx` instead, which allows selection of a group number for the affinity.

In 64-bit Windows, the `Stride` field in the `KNODE` is used to balance the assignment of newly created threads within a process. The stride is a scalar number that represents the number of affinity bits within a given NUMA node that must be skipped to attain a new independent logical processor slice, where *independent* means on another core (if dealing with an SMT system) or another package (if dealing with a non-SMT but multicore system). Because 32-bit Windows doesn't support large processor-configuration systems, it doesn't use a stride. It simply selects the next processor number, trying to avoid sharing the same SMT set if possible.

Ideal node

On NUMA systems, when a process is created, an ideal node for the process is selected. The first process is assigned to node 0, the second process to node 1, and so on. Then the ideal processors for the threads in the process are chosen from the process's ideal node. The ideal processor for the first thread in a process is assigned to the first processor in the node. As additional threads are created in processes with the same ideal node, the next processor is used for the next thread's ideal processor, and so on.

CPU sets

You've seen how affinity (sometimes referred to as *hard affinity*) can limit threads to certain processors, which is always honored by the scheduler. The ideal processor mechanism tries to run threads on their ideal processors (sometimes referred to as *soft affinity*), generally expecting to have the thread's state be part of the processor's cache. The ideal processor may or may not be used, and it does not prevent the thread from being scheduled on other processors. Both these mechanisms don't work on system-related activity, such as system threads activity. Also, there is no easy way to set hard affinity to all processes on a system in one stroke. Even walking the process would not work. System processes are generally protected from external affinity changes because they require the `PROCESS_SET_INFORMATION` access right, which is not granted for protected processes.

Windows 10 and Server 2016 introduce a mechanism called *CPU sets*. These are a form of affinity that you can set for use by the system as a whole (including system threads activity), processes, and even individual threads. For example, a low-latency audio application may want to use a processor exclusively while the rest of the system is diverted to use other processors. CPU sets provide a way to achieve that.

The documented user mode API is somewhat limited at the time of this writing. `GetSystemCpuSetInformation` returns an array of `SYSTEM_CPU_SET_INFORMATION` that contains data for each CPU set. In the current implementation, a CPU set is equivalent to a single CPU. This means the returned array's length is the number of logical processors on the system. Each CPU set is identified by its ID, which is arbitrarily selected to be 256 (0x100) plus the CPU index (0, 1, ...). These IDs are the ones that must be passed to `SetProcessDefaultCpuSets` and `SetThreadSelectedCpuSets` functions to set default CPU sets for a process and a CPU set for a

specific thread, respectively.

An example for setting thread CPU set would be for an “important” thread that should not be interrupted if possible. This thread could have a CPU set that contains one CPU, while setting the default process CPU set to include all other CPUs.

One missing function in the Windows API is the ability to reduce the system CPU set. This can be achieved by a call to the `NtSetSystemInformation` system call. For this to succeed, the caller must have `SeIncreaseBasePriorityPrivilege`.

EXPERIMENT: CPU sets

In this experiment, you’ll view and modify CPU sets and watch the resulting effects.

1. Download the `CpuSet.exe` tool from the book’s downloadable resources.
2. Open an administrative command window and navigate to the directory where `CPUSET.exe` exists.
3. At the command window, type **cpuset.exe** without arguments to see the current system CPU sets. The output should be similar to the following:

[Click here to view code image](#)

```
System CPU Sets
-----
Total CPU Sets: 8

CPU Set 0
  Id: 256 (0x100)
  Group: 0
  Logical Processor: 0
  Core: 0
  Last Level Cache: 0
  NUMA Node: 0
  Flags: 0 (0x0)  Parked: False  Allocated: False  Realtime:
False  Tag: 0

CPU Set 1
  Id: 257 (0x101)
  Group: 0
  Logical Processor: 1
  Core: 0
  Last Level Cache: 0
  NUMA Node: 0
  Flags: 0 (0x0)  Parked: False  Allocated: False  Realtime:
False  Tag: 0
...
```

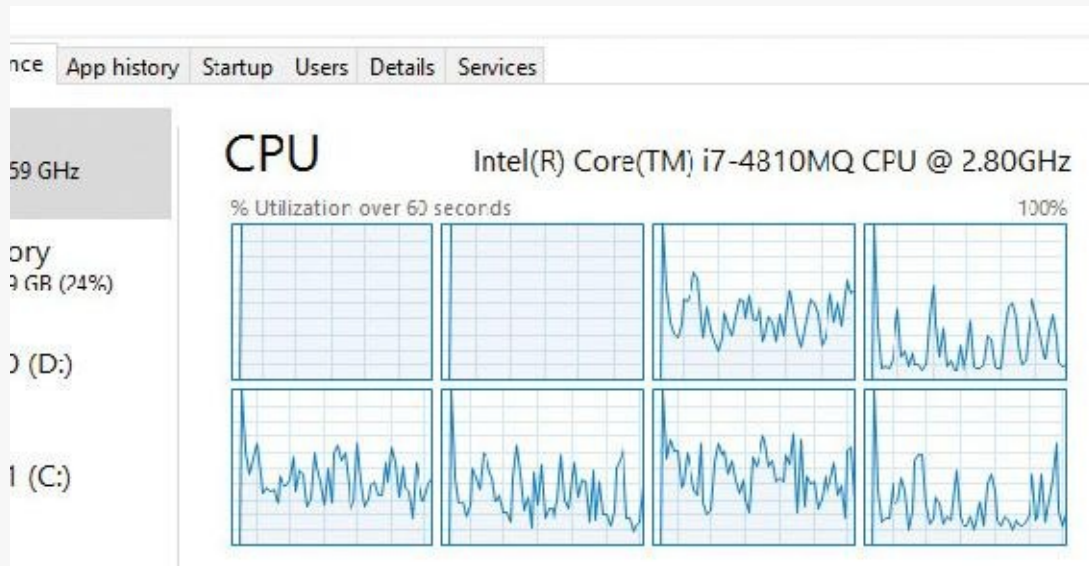
4. Run `CPUSTRES.exe` and configure it to run a thread or two with maximum activity level. (Aim for something around 25 percent CPU usage.)
5. Open Task Manager, click the **Performance** tab, and select the **CPU** label.
6. Change the CPU graph view to show individual processors (if the view is configured for

overall utilization).

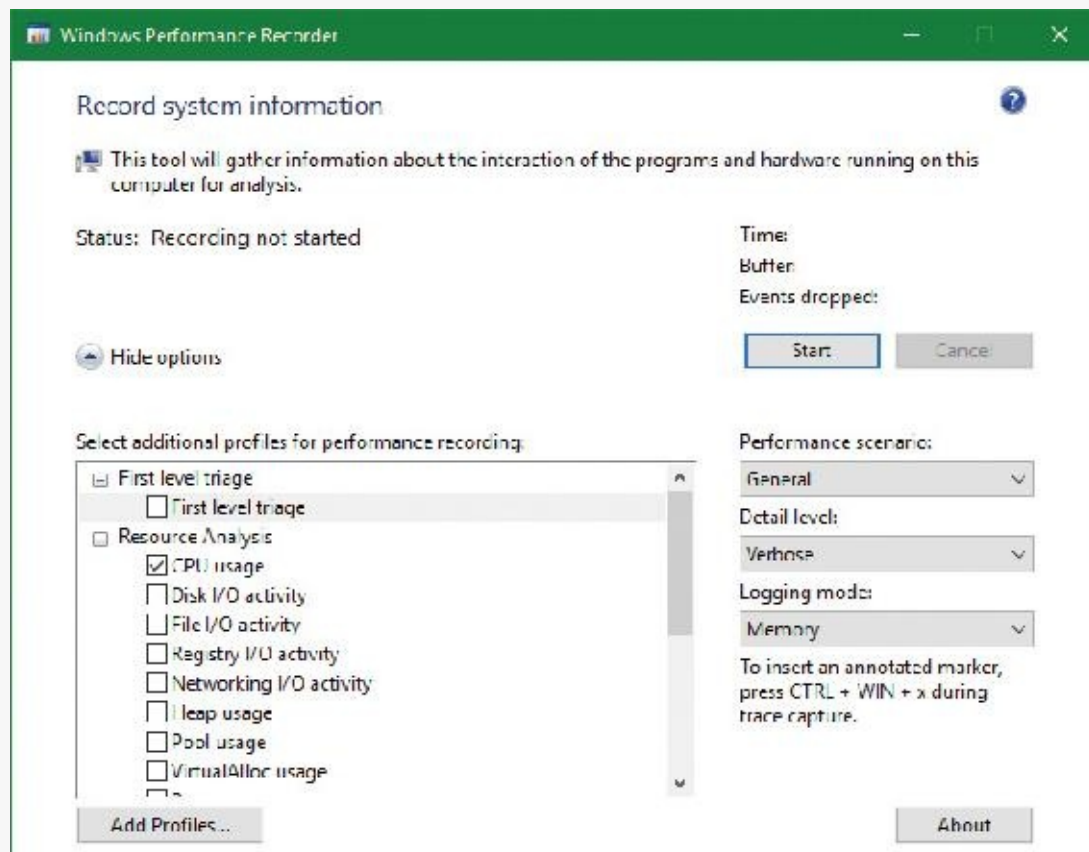
- At the command window, run the following command, replacing the number after -p with the process ID of the CPUSTRES process on your system:

CpuSet.exe -p 18276 -s 3

The -s argument specifies the processor mask to set as the default for the process. Here, 3 means CPU 0 and 1. You should see Task Manager hard at work on these two CPUs:

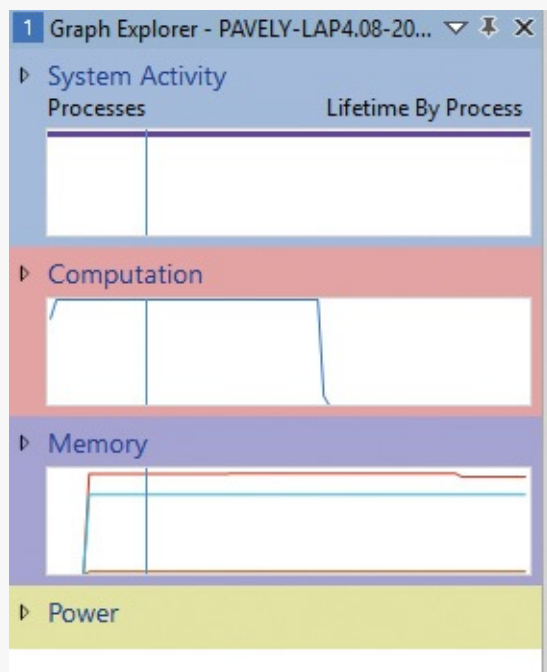


- Let's look at CPU 0 more closely to see what threads it's running. For this, you'll use Windows Performance Recorder (WPR) and Windows Performance Analyzer (WPA) from the Windows SDK. Click the **Start** button, type **WPR**, and select **Windows Performance Recorder**. Then accept the elevation prompt. You should see the following dialog box:

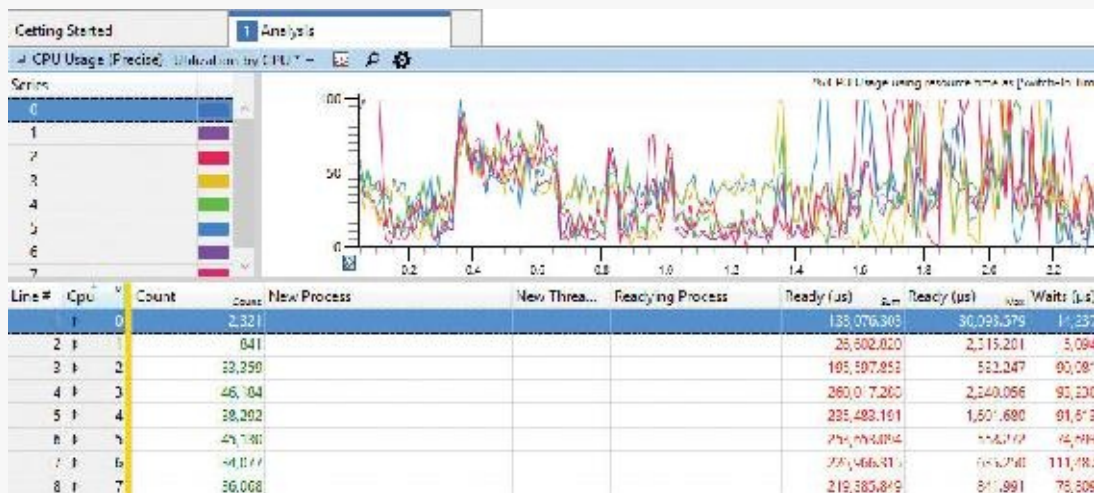


- The default is to record CPU usage, which is what we want. This tool records Event Tracing for Windows (ETW) events. (See Chapter 8 in Part 2 for more on ETW.) Click the **Start**

- button in the dialog box and, after a second or two click the same button, now labeled **Save**.
- WPR will suggest a location to save the recorded data. Accept it or change to some other file/folder.
 - After the file is saved, WPR suggests opening the file with WPA. Accept the suggestion.
 - The WPA tool opens and loads the saved file. (WPA is a rich tool, well beyond the scope of this book). On the left, you'll see the various categories of information captured, something like so:



- Expand the **Computation** node and then expand the **CPU Usage (Precise)** node.
- Double-click the **Utilization by CPU** graph. It should open in the main display:



- At the moment, we're interested in CPU 0. In the next step, you'll make CPU 0 work for CPUSTRES only. To begin, expand the CPU 0 node. You should see various processes, including CPUSTRES, but certainly not exclusively:

#	Cpu	Count	Count	New Process	Ne
1	0		2,321		
2				1 System (4)	
3				1 RuntimeBroker.exe (7596)	
4				1 System (4)	
5				1 RuntimeBroker.exe (7596)	
6				1 System (4)	
7				1 MsMpEng.exe (3948)	
8				1 System (4)	
9				1 SearchIndexer.exe (10784)	
10				1 System (4)	
11				1 CPUSTRES.exe (18276)	
12				1 conhost.exe (13580)	
13				1 System (4)	
14				1 conhost.exe (13580)	
15				1 System (4)	
16				1 SHSPipeHost.exe (12344)	
17				1 System (4)	
18				1 SHSPipeHost.exe (12344)	
19				1 System (4)	
20				1 SearchIndexer.exe (10784)	
21				1 System (4)	
22				1 SearchIndexer.exe (10784)	

16. Enter the following command to restrict the system to use all processors except the first. In this system, the number of processors is eight, so a full mask is 255 (0xff). Removing CPU 0 produces 254 (0xfe). Replace the mask with the correct one for your system:

CpuSet.exe -s 0xfe

17. The view in Task Manager should look about the same. Let's take a closer look at CPU 0. Run WPR again, and record a second or two with the same settings as before.
18. Open the trace in WPA and navigate to **Utilization by CPU**.
19. Expand CPU 0. You should see CPUSTRES almost exclusively, with the System process appearing occasionally:

#	Cpu	Count	Count	New Process
1	0		15	
2				1 CPUSTRES.exe (18276)
3				1 CPUSTRES.exe (18276)
4				1 CPUSTRES.exe (18276)
5				1 CPUSTRES.exe (18276)
6				1 System (4)
7				1 CPUSTRES.exe (18276)
8				1 CPUSTRES.exe (18276)
9				1 CPUSTRES.exe (18276)
0				1 System (4)
1				1 CPUSTRES.exe (18276)
2				1 CPUSTRES.exe (18276)
3				1 CPUSTRES.exe (18276)
4				1 CPUSTRES.exe (18276)
5				1 CPUSTRES.exe (18276)

20. Notice in the CPU Usage (in View) (ms) column that the time spent in the System process is very small (micro seconds). Clearly, CPU 0 is dedicated to the CPUSTRES process.

21. Run `CPUSET.exe` with no arguments again. The first set (CPU 0) is marked `Allocated: True` because it's now allocated to a particular process and not for general system use.
22. Close CPU Stress.
23. Enter the following command to restore the system CPU set to its default:

```
Cpuset -s 0
```

Thread selection on multiprocessor systems

Before covering multiprocessor systems in more detail, let's summarize the algorithms discussed earlier in the "[Thread selection](#)" section. They either continued executing the current thread (if no new candidate was found) or started running the idle thread (if the current thread had to block). However, there is a third algorithm for thread selection called `KiSearchForNewThread`, which was hinted at earlier. This algorithm is called in one specific instance: when the current thread is about to block due to a wait on an object, including when doing an `NtDelayExecutionThread` call, also known as the `Sleep API` in Windows.



Note

This shows a subtle difference between the commonly used `Sleep(1)` call, which makes the current thread block until the next timer tick, and the `SwitchToThread` call, which was shown earlier. The `Sleep(1)` call uses the algorithm about to be described, while the `SwitchToThread` call uses the previously shown logic.

`KiSearchForNewThread` initially checks whether there is already a thread that was selected for this processor (by reading the `NextThread` field). If so, it dispatches this thread immediately in the running state. Otherwise, it calls the `KiSelectReadyThreadEx` routine and, if a thread was found, performs the same steps.

If no thread was found, the processor is marked as idle (even though the idle thread is not yet executing) and a scan of the queues of other logical processors (shared) is initiated (unlike the other standard algorithms, which would now give up). If, however, the processor core is parked, the algorithm will not attempt to check other logical processors, as it is preferable to allow the core to enter the parking state instead of keeping it busy with new work.

Barring these two scenarios, the work-stealing loop now runs. This code looks at the current NUMA node and removes any idle processors (because they shouldn't have threads that need stealing). Then the code looks at the current CPU's shared ready queue and calls `KiSearchForNewThreadOnProcessor` in a loop. If no thread is found, the affinity is changed to the next group and the function is called again. This time, however, the target CPU points it to the next group's shared queue instead of the current one, causing this processor to find the best ready thread from the other processor group's ready queue. If this fails to find a thread to run, local queues of processors in that group are searched in the same manner. If this is unsuccessful, and if DFSS is enabled, a thread from the idle-only queue of the remote logical processor is released on the current processor instead, if possible.

If no candidate ready thread is found, the next-lower numbered logical processor is attempted, and so on, until all logical processors have been exhausted on the current NUMA node. In this case, the algorithm

keeps searching for the next-closest node, and so on, until all nodes in the current group have been exhausted. (Recall that Windows allows a given thread to have affinity only on a single group.) If this process fails to find any candidates, the function returns `NULL` and the processor enters the idle thread in the case of a wait (which will skip idle scheduling). If this work was already being done from the idle scheduler, the processor enters a sleep state.

Processor selection

We've described how Windows picks a thread when a logical processor needs to make a selection (or when a selection must be made for a given logical processor) and assumed the various scheduling routines have an existing database of ready threads to choose from. Now we'll see how this database gets populated in the first place—in other words, how Windows chooses which LP's ready queues to associate with a given ready thread. Having described the types of multiprocessor systems supported by Windows as well as thread-affinity and ideal processor settings, we're now ready to examine how this information is used for this purpose.

Choosing a processor for a thread when there are idle processors

When a thread becomes ready to run, the `KiDeferredReadyThread` scheduler function is called. This prompts Windows to perform two tasks:

- Adjust priorities and refresh quantum as needed (as explained in the “[Priority boosts](#)” section).
- Pick the best logical processor for the thread.

Windows first looks up the thread's ideal processor and then it computes the set of idle processors within the thread's hard affinity mask. This set is then pruned as follows:

1. Any idle logical processors that have been parked by the core-parking mechanism are removed. (See [Chapter 6](#) for more information on core parking.) If this causes no idle processors to remain, idle processor selection is aborted, and the scheduler behaves as if no idle processors were available (described in the next section).
2. Any idle logical processors that are not on the ideal node (defined as the node containing the ideal processor) are removed (unless this would cause all idle processors to be eliminated).
3. On an SMT system, any non-idle SMT sets are removed, even if this might cause the elimination of the ideal processor itself. In other words, Windows prioritizes a non-ideal, idle SMT set over an ideal processor.
4. Windows checks whether the ideal processor is among the remaining set of idle processors. If not, it must then find the most appropriate idle processor. To do this, it first checks whether the processor that the thread last ran on is part of the remaining idle set. If so, it considers this processor to be a temporary ideal processor and selects it. (Recall that the ideal processor attempts to maximize processor cache hits, and picking the last processor a thread ran on is a good way of doing so.) If the last processor is not part of the remaining idle set, Windows checks whether the current processor (that is, the processor currently executing this scheduling code) is part of this set. If so, it applies the same logic as before.
5. If neither the last nor the current processor is idle, Windows performs one more pruning operation, removing any idle logical processors that are not on the same SMT set as the ideal processor. If there are none left, Windows instead removes any processors not on the SMT set of the current processor (unless this, too, eliminates all idle processors). In other words, Windows prefers idle processors that share the same SMT set as the unavailable ideal processor and/or last processor it would've liked to pick in the first place. Because SMT implementations share the cache on the

core, this has nearly the same effect as picking the ideal or last processor from a caching perspective.

6. If after the previous step more than one processor remains in the idle set, Windows picks the lowest-numbered processor as the thread's current processor.

After a processor has been selected for the thread to run on, that thread is put in the standby state and the idle processor's PRCB is updated to point to this thread. If the processor is idle but not halted, a DPC interrupt is sent so that the processor handles the scheduling operation immediately. Whenever such a scheduling operation is initiated, `KiCheckForThreadDispatch` is called. It detects that a new thread has been scheduled on the processor and causes an immediate context switch if possible (as well as notifying Autoboost of the switch and delivering pending APCs). Alternatively, if no thread is pending, it causes a DPC interrupt to be sent.

Choosing a processor for a thread when there are no idle processors

If there are no idle processors when a thread wants to run, or if the only idle processors were eliminated by the first pruning (which got rid of parked idle processors), Windows first checks whether the latter situation has occurred. In this scenario, the scheduler calls `KiSelectCandidateProcessor` to ask the core-parking engine for the best candidate processor. The core-parking engine selects the highest-numbered processor that is unparked within the ideal node. If there are no such processors, the engine forcefully overrides the park state of the ideal processor and causes it to be unparked. Upon returning to the scheduler, it checks whether the candidate it received is idle; if so, it picks this processor for the thread, following the same last steps as in the previous scenario.

If this fails, Windows must decide whether to preempt the currently running thread. First, a target processor needs to be selected. The preference is in order of precedence: the ideal processor of the thread, the last processor the thread ran on, the first available processor in the current NUMA node, the closest processor on another NUMA node, and all these, barring affinity constraints, if any.

After a processor is selected, the next question is whether the new thread should preempt the current one on that processor. This is done by comparing the ranks of the two threads. This is an internal scheduling number that indicates the relative power of a thread based on its scheduling group and other factors. (See the section “[Group-based scheduling](#)” later in this chapter for a detailed discussion of group scheduling and rank.) If the rank of the new thread is zero (highest) or lower than the current thread's rank, or the ranks are equal but the priority of the new thread is higher than the currently executing one, then preemption should occur. The currently running thread is marked to be preempted, and Windows queues a DPC interrupt to the target processor to preempt the currently running thread in favor of this new thread.

If the ready thread cannot be run right away, it is moved into the ready state on the shared or local queue (as appropriate based on affinity constraints), where it will await its turn to run. As seen in the scheduling scenarios earlier, the thread will be inserted either at the head or the tail of the queue, based on whether it entered the ready state due to preemption.



Note

Regardless of the underlying scenario and various possibilities, threads are mostly put on their ideal processor's per-processor ready queues, guaranteeing the consistency of the algorithms that determine how a logical processor picks a thread to run.

Heterogeneous scheduling (big.LITTLE)

The kernel assumes an SMP system, as previously described. However, some ARM-based processors contain multiple cores that are not the same. A typical ARM CPU (for example, from Qualcomm) contains some powerful cores, which should run for short periods at a time (and consume more energy), and a set of weaker cores, which can run for longer periods (and consume less energy). This is sometimes called *big.LITTLE*.

Windows 10 introduced the ability to distinguish between these cores and schedule threads based on the core's size and policy, including the foreground status of the thread, its priority, and its expected run time. Windows initializes the set of processors when the Power Manager is initialized by calling `PopInitializeHeteroProcessors` (and if processors are hot-added to the system). The function allows the simulation of hetero systems (for example, for testing purposes) by adding keys under the registry key `HKLM\System\CurrentControlSet\Control\Session Manager\Kernel\KGroups` as follows:

- A key should use two decimal digits to identify a processor group number. (Recall that each group holds at most 64 processors.) For example, 00 is the first group, 01 is the second, etc. (On most systems, one group would suffice.)
- Each key should contain a `DWORD` value named `SmallProcessorMask` that is a mask of processors that would be considered small. For example, if the value is 3 (the first two bits are on) and the group has six total processors, that would mean processors 0 and 1 ($3 = 1 \text{ or } 2$) are small, while the other four processors are big. This is essentially the same as an affinity mask.

The kernel has several policy options that can be tweaked when dealing with hetero systems, stored in global variables. [Table 4-5](#) shows some of these variables and their meaning.

Variable Name	Meaning	Default Value
KiHeteroSystem	Is the system heterogeneous?	False
PopHeteroSystem	System hetero type: None (0) Simulated (1) EfficiencyClass (2) FavoredCore (3)	None (0)
PpmHeteroPolicy	Scheduling policy: None (0) Manual (1) SmallOnly (2) LargeOnly (3) Dynamic (4)	Dynamic (4)
KiDynamicHeteroCpuPolicyMask	Determine what is considered in assessing whether a thread is important	7 (foreground status = 1, priority = 2, expected run time = 4)
KiDefaultDynamicHeteroCpuPolicy	Behavior of Dynamic hetero policy (see above): All (0) (all available) Large (1) LargeOrIdle (2) Small (3) SmallOrIdle (4) Dynamic (5) (use priority and other metrics to decide) BiasedSmall (6) (use priority and other metrics, but prefer small) BiasedLarge (7)	Small (3)
KiDynamicHeteroCpuPolicyImportant	Policy for a dynamic thread that is deemed important (see possible values above)	LargeOrIdle (2)
KiDynamicHeteroCpuPolicyImportantShort	Policy for dynamic thread that is deemed important but run a short amount of time	Small (3)
KiDynamicCpuPolicyExpectedRunTime	Run-time value that is considered heavy	5,200 msec
KiDynamicHeteroCpuPolicyImportantPriority	Priority above which threads are considered important if priority-based dynamic policy is chosen	8

TABLE 4-5 Hetero kernel variables

Dynamic policies (refer to [Table 4-5](#)) must be translated to an importance value based on KiDynamic- HeteroPolicyMask and the thread's state. This is done by the KiConvertDynamicHeteroPolicy function, which checks, in order, the foreground state of the thread, its priority relative to KiDynamic-HeteroCpuPolicyImportantPriority, and its expected run time. If the thread is deemed important (if running time is the determining factor, then it could be short as well), the important-related policy is used for scheduling decisions. (In [Table 4-5](#), this would be KiDynamicHeteroCpuPolicyImportant-Short or KiDynamicHeteroCpuPolicyImportant.)

Group-based scheduling

The previous section described the standard thread-based scheduling implementation of Windows. Since its appearance in the first release of Windows NT (with scalability improvements done with each subsequent release), it has reliably served general user and server scenarios. However, because thread-based scheduling attempts to fairly share the processor or processors only among competing threads of the same priority, it does not account for higher-level requirements such as the distribution of threads to users and the potential for certain users to benefit from more overall CPU time at the expense of other users. This is problematic in terminal-services environments, in which dozens of users compete for CPU time. If only thread-based scheduling is used, a single high-priority thread from a given user has the potential to starve threads from all users on the machine.

Windows 8 and Server 2012 introduced a group-based scheduling mechanism, built around the concept of a scheduling group (KSCHEULING_GROUP). A scheduling group maintains a policy, scheduling parameters (described shortly), and a list of kernel scheduling control blocks (KSCBs), one per processor, that are part of the scheduling group. The flip side is that a thread points to a scheduling group it belongs to. If that pointer is null, it means the thread is outside any scheduling group's control. [Figure 4-19](#) shows the structure of a scheduling group. In this figure, threads T1, T2, and T3 belong to the scheduling group, while thread T4 does not.

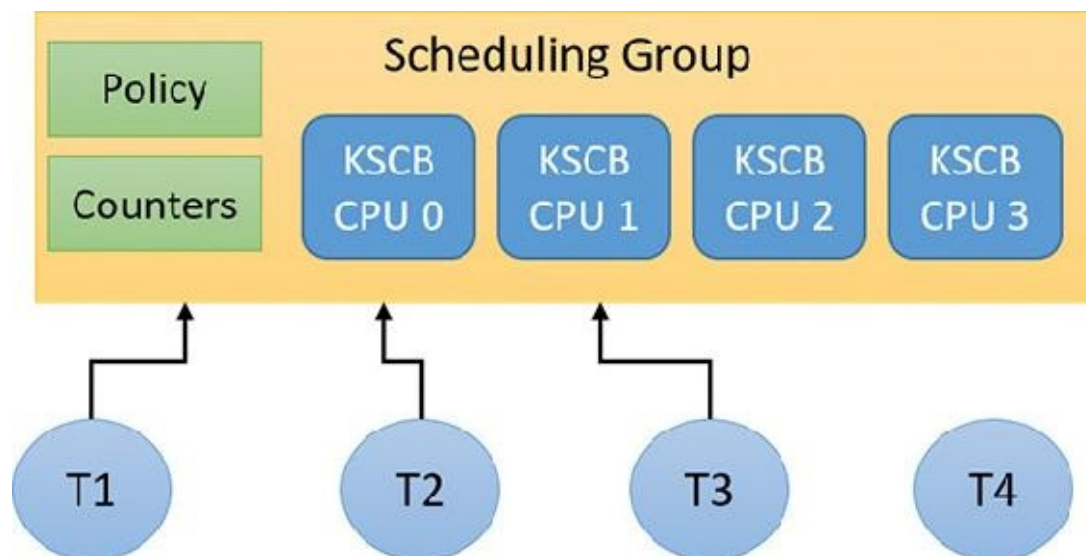


FIGURE 4-19 Scheduling group.

Here are some terms related to group scheduling:

- **Generation** This is the amount of time over which to track CPU usage.
- **Quota** This is the amount of CPU usage allowed to a group per generation. *Over quota* means the group has used up all its budget. *Under quota* means the group has not used its full budget.
- **Weight** This is the relative importance of a group, between 1 and 9, where the default is 5.
- **Fair-share scheduling** With this type of scheduling, idle cycles can be given to threads that are over quota if no under-quota threads want to run.

The KSCB structure contains CPU-related information as follows:

- Cycle usage for this generation
- Long-term average cycle usage, so that a burst of thread activity can be distinguished from a true hog
- Control flags such as hard capping, which means that even if CPU time is available above the

assigned quota, it will not be used to give the thread extra CPU time

- Ready queues, based on the standard priorities (0 to 15 only because real-time threads are never part of a scheduling group)

An important parameter maintained by a scheduling group is called *rank*, which can be considered a scheduling priority of the entire group of threads. A rank with a value of 0 is the highest. A higher-rank number means the group has used more CPU time and so is less likely to get more CPU time. Rank always trumps priority. This means that given two threads with different ranks, the lower value rank is preferred, regardless of priority. Equal-rank threads are compared based on priority. The rank is adjusted periodically as cycle usage increases.

Rank 0 is the highest (so it always wins out) against a higher number rank, and is implicit for some threads. This can indicate one of the following:

- The thread is not in any scheduling group (“normal” threads)
- Under-quota threads
- Real-time priority threads (16–31)
- Threads executing at `IRQL APC_LEVEL (1)` within a kernel critical or guarded region (see Chapter 8 in Part 2 for more on APCs and regions)

At various scheduling choices (for example, `KiQuantumEnd`), the decision of which thread to schedule next accounts for the scheduling group (if any) of the current and ready threads. If a scheduling group exists, the lowest value rank wins out, followed by priority (if ranks are equal), followed by the first arriving thread (if priorities are equal; round-robin at quantum end).

Dynamic fair share scheduling

Dynamic fair share scheduling (DFSS) is a mechanism that can be used to fairly distribute CPU time among sessions running on a machine. It prevents one session from potentially monopolizing the CPU if some threads running under that session have a relatively high priority and run a lot. It’s enabled by default on a Windows Server system that has the Remote Desktop role. However, it can be configured on any system, client or server. Its implementation is based on group scheduling described in the previous section.

During the very last parts of system initialization, as the registry `SOFTWARE` hive is initialized by `Smss.exe`, the process manager initiates the final post-boot initialization in `PsBootPhaseComplete`, which calls `PspIsDfssEnabled`. Here, the system decides which of the two CPU quota mechanisms (DFSS or legacy) will be employed. For DFSS to be enabled, the `EnableCpuQuota` registry value must be set to a non-zero value in both of the quota keys. The first of these is `HKLM\SOFTWARE\Policies\Microsoft\Windows\Session Manager\Quota System`, for the policy-based setting. The second is `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Quota System`, under the system key. This determines whether the system supports the functionality (which, by default, is set to `TRUE` on Windows Server with the Remote Desktop role).

If DFSS is enabled, the `PSCpuFairShareEnabled` global variable is set to `TRUE`, which makes all threads belong to scheduling groups (except session 0 processes). DFSS configuration parameters are read from the aforementioned keys by a call to `PspReadDfssConfigurationValues` and stored in global variables. These keys are monitored by the system. If modified, the notification callback calls `PspReadDfssConfigurationValues` again to update the configuration values. [Table 4-6](#) shows the values and their meaning.

Registry Value Name	Kernel Variable Name	Meaning	Default Value
DfssShortTermSharingMS	PsDfssShortTermSharingMS	The time it takes for the group rank to increase within a generation cycle	30 ms
DfssLongTermSharingMS	PsDfssLongTermSharingMS	The time it takes to jump from rank 0 to a non-zero rank when the threads exceed their quota within the generation cycle	15 ms
DfssGenerationLengthMS	PsDfssGenerationLengthMS	The generation time over which to track CPU usage	600 ms
DfssLongTermFraction1024	PsDfssLongTermFraction1024	The value used in a formula for an exponential moving average used for long-term cycles computation	512

TABLE 4-6 DFSS registry configuration parameters

After DFSS is enabled, whenever a new session is created (other than session 0), `MsSessionObjectCreate` allocates a scheduling group associated with the session with the default weight of 5, which is the middle ground between the minimum of 1 and the maximum of 9. A scheduling group manages either DFSS or CPU rate-control (see the next section) information based on a policy structure (`KSCEDULING_GROUP_POLICY`) that is part of a scheduling group. The `Type` member indicates whether it's configured for DFSS (`WeightBased=0`) or rate control (`RateControl=1`). `MsSessionObjectCreate` calls `KeInsertSchedulingGroup` to insert the scheduling group into a global system list (maintained in the global variable `KiSchedulingGroupList`, needed for weight recalculation if processors are hot-added). The resulting scheduling group is also pointed to by the `SESSION_OBJECT` structure for the particular session.

EXPERIMENT: DFSS in action

In this experiment, you'll configure a system to use DFSS and watch it “do its thing”.

1. Add the registry keys and values as described in this section to enable DFSS on the system. (You can try this experiment on a VM as well.) Then restart the system for the changes to take effect.
2. To make sure DFSS is active, open a live kernel debug session and inspect the value of `PsCpuFairShareEnabled` by issuing the following command. A value of 1 indicates DFSS is active.

[Click here to view code image](#)

```
lkd> db nt!PsCpuFairShareEnabled L1
fffff800'5183722a 01
```

3. In the debugger, look at the current thread. (It should be one of the threads running WinDbg.) Notice that the thread is part of a scheduling group and that its `KSCB` is not `NULL` because the thread was running at the time of display.

[Click here to view code image](#)

```
lkd> !thread
THREAD fffffd28c07231640 Cid 196c.1a60 Teb: 000000f897f4b000
Win32Thread:
fffffd28c0b9b0b40 RUNNING on processor 1
IRP List:
fffffd28c06dfac10: (0006,0118) Flags: 00060000 Mdl: 00000000
```

```

Not impersonating
DeviceMap                fffffac0d33668340
Owning
Process                  fffffd28c071fd080      Image:                windbg.c
Attached Process         N/A              Image:                N/A
Wait Start TickCount    6146              Ticks: 33
(0:00:00:00.515)
Context Switch Count    877              IdealProcessor: 0
UserTime                 00:00:00.468
KernelTime               00:00:00.156
Win32 Start Address 0x00007ff6ac53bc60
Stack Init fffffbf81ae85fc90 Current fffffbf81ae85f980
Base fffffbf81ae860000 Limit fffffbf81ae85a000 Call
0000000000000000
Priority 8 BasePriority 8 PriorityDecrement 0 IoPriority 2
PagePriority 5
Scheduling Group: fffffd28c089e7a40 KSCB: fffffd28c089e7c68 rank 0

```

4. Enter the **dt** command to view the scheduling group:

[Click here to view code image](#)

```

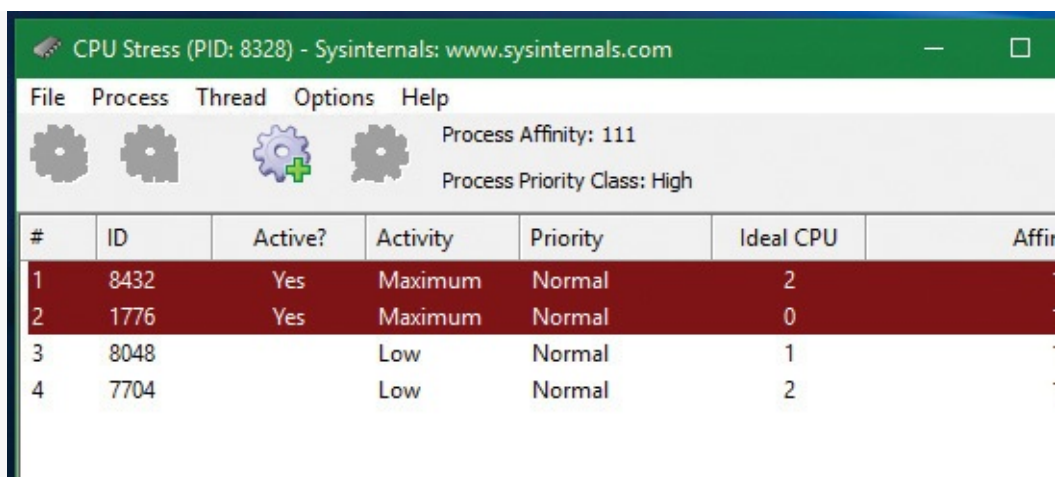
lkd> dt nt!_kscheduling_group fffffd28c089e7a40
+0x000 Policy                : _KSCHEDULING_GROUP_POLICY
+0x008 RelativeWeight        : 0x80
+0x00c ChildMinRate          : 0x2710
+0x010 ChildMinWeight        : 0
+0x014 ChildTotalWeight      : 0
+0x018 QueryHistoryTimeStamp : 0xfed6177
+0x020 NotificationCycles    : 0n0
+0x028 MaxQuotaLimitCycles   : 0n0
+0x030 MaxQuotaCyclesRemaining : 0n-73125382369
+0x038 SchedulingGroupList   : _LIST_ENTRY [
0xffffffff800'5179b110 -
0xfffffd28c'081b7078 ]
+0x038 Sibling                : _LIST_ENTRY [ 0xffffffff800'5179b110 -
0xfffffd28c'081b7078 ]
+0x048 NotificationDpc        : 0x0002eaa8'00000008e _KDPC
+0x050 ChildList              : _LIST_ENTRY [ 0xfffffd28c'062a7ab8 -
0xfffffd28c'05c0bab8 ]
+0x060 Parent                 : (null)
+0x080 PerProcessor           : [1] _KSCB

```

5. Create another local user on the machine.

6. Run CPU Stress in the current session.

7. Make a few threads run at maximum activity, but not enough to overwhelm the machine. For example, the following image shows two threads running at maximum activity on a three-processor virtual machine:



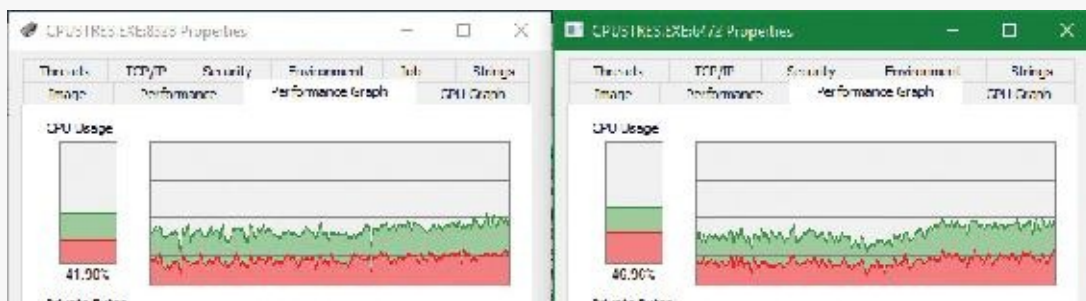
CPU Stress (PID: 8328) - Sysinternals: www.sysinternals.com

File Process Thread Options Help

Process Affinity: 111
Process Priority Class: High

#	ID	Active?	Activity	Priority	Ideal CPU	Affinity
1	8432	Yes	Maximum	Normal	2	
2	1776	Yes	Maximum	Normal	0	
3	8048		Low	Normal	1	
4	7704		Low	Normal	2	

8. Press **Ctrl+Alt+Del** and select **Switch User**. Then select and log in to the account for the other user you created.
9. Run CPU Stress again, making the same number of threads run with maximum activity.
10. For the CPUSTRES process, open the **Process** menu, choose **Priority Class**, and select **High** to change the process priority class. Without DFSS, that higher-priority process should consume most of the CPU. This is because there are four threads competing for three processors. One of these will lose out, and it should be from the lower-priority process.
11. Open Process Explorer, double-click both CPUSTRES processes, and select the **Performance Graph** tab.
12. Place both windows side by side. You should see the CPU consumed roughly evenly between the processes, even though their priorities are not the same:



13. Disable DFSS by removing the registry keys. Then restart the system.
14. Rerun the experiment. You should clearly see the difference with the higher-priority process receiving the most CPU time.

CPU rate limits

DFSS works by automatically placing new threads inside the session-scheduling group. This is fine for a terminal-services scenario, but is not good enough as a general mechanism to limit the CPU time of threads or processes.

The scheduling-group infrastructure can be used in a more granular fashion by using a job object. Recall from [Chapter 3](#) that a job can manage one or more processes. One of the limitations you can place on a job is a CPU rate control, which you do by calling `SetInformationJobObject` with `Job-ObjectCpuRateControlInformation` as the job information class and a structure of type `JOB_OBJECT_CPU_RATE_CONTROL_INFORMATION` containing the actual control data. The structure contains a set of flags that enable you to apply one of three settings to limit CPU time:

- **CPU rate** This value can be between 1 and 10000 and represents a percent multiplied by 100 (for

example, for 40 percent the value should be 4000).

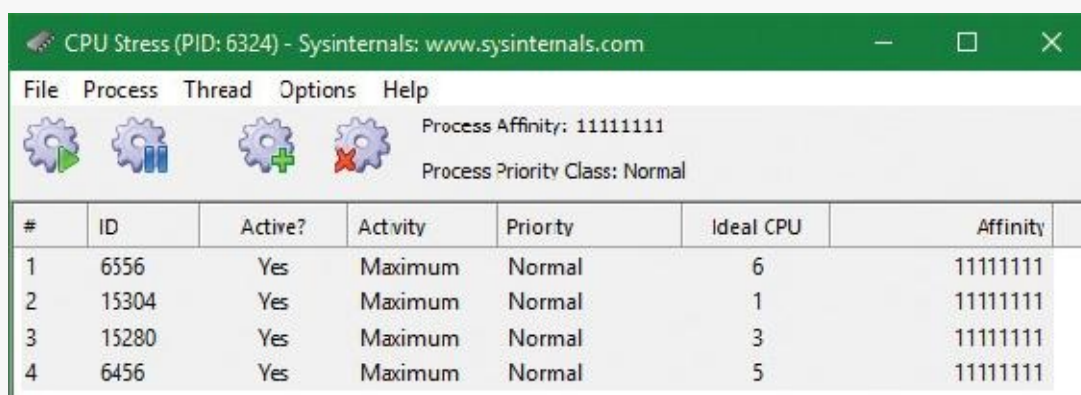
- **Weight-based** This value can be between 1 and 9, relative to the weight of other jobs. (DFSS is configured with this setting.)
- **Minimum and maximum CPU rates** These values are specified similarly to the first option. When the threads in the job reach the maximum percentage specified in the measuring interval (600 ms by default), they cannot get any more CPU time until the next interval begins. You can use a control flag to specify whether to use hard capping to enforce the limit even if there is spare CPU time available.

The net result of setting these limits is to place all threads from all processes that are in the job in a new scheduling group and configuring the group as specified.

EXPERIMENT: CPU rate limit

In this experiment, you'll look at CPU rate limit using a job object. It's best to perform this experiment on a virtual machine and attach to its kernel rather than using the local kernel because of a debugger bug at the time of writing.

1. Run CPU Stress on the test VM and configure a few threads to consume about 50 percent of CPU time. For example, on an eight-processor system, activate four threads that run with maximum activity level:

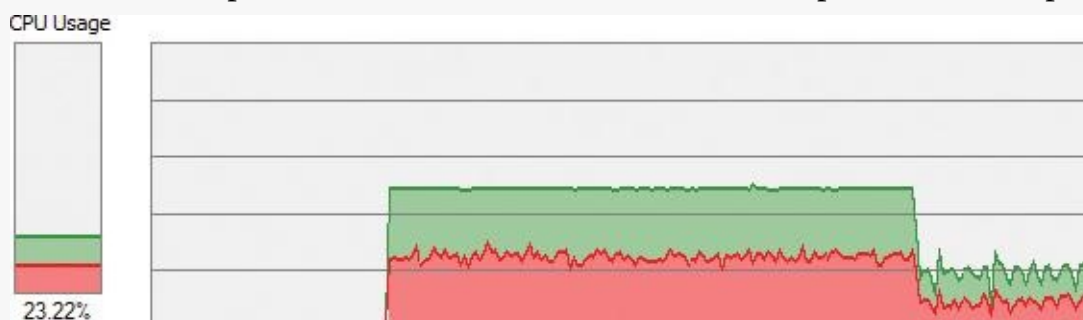


#	ID	Active?	Activity	Priority	Ideal CPU	Affinity
1	6556	Yes	Maximum	Normal	6	11111111
2	15304	Yes	Maximum	Normal	1	11111111
3	15280	Yes	Maximum	Normal	3	11111111
4	6456	Yes	Maximum	Normal	5	11111111

2. Open Process Explorer, find the CPUSTRES instance, open its properties, and select the **Performance Graph** tab. The CPU usage should be roughly 50 percent.
3. Download the CPULIMIT tool from the book's downloadable resources. This is a simple tool that allows you to limit the CPU usage of a single process through hard capping.
4. Run the command shown to limit the CPU usage to 20 percent for the CPUSTRES process. (Replace the number 6324 with your process ID.)

CpuLimit.exe 6324 20

5. Look at the Process Explorer window. You should see the drop to around 20 percent:



6. Open WinDbg on the host system.

7. Attach to the kernel of the test system and break into it.

8. Enter the following command to locate the CPUTRES process:

[Click here to view code image](#)

```
0: kd> !process 0 0 cpustres.exe
PROCESS fffff9e0629528080
    SessionId: 1  Cid: 18b4    Peb: 009e4000  ParentCid: 1c4c
    DirBase: 230803000  ObjectTable:
fffffd78d1af6c540  HandleCount: <Data
Not Accessible>
    Image: CPUTRES.exe
```

9. Type the following command to list basic information for the process:

[Click here to view code image](#)

```
0: kd> !process fffff9e0629528080 1
PROCESS fffff9e0629528080
    SessionId: 1  Cid: 18b4    Peb: 009e4000  ParentCid: 1c4c
    DirBase: 230803000  ObjectTable:
fffffd78d1af6c540  HandleCount: <Data
Not Accessible>
    Image: CPUTRES.exe
    VadRoot fffff9e0626582010 Vads 88 Clone 0 Private 450.
Modified 4. Locked 0.
    DeviceMap fffffd78cd8941640
    Token                                     fffffd78cfe3db050
    ElapsedTime                             00:08:38.438
    UserTime                                00:00:00.000
    KernelTime                              00:00:00.000
    QuotaPoolUsage[PagedPool]               209912
    QuotaPoolUsage[NonPagedPool]           11880
    Working Set Sizes (now,min,max) (3296, 50, 345) (13184KB,
200KB, 1380KB)
    PeakWorkingSetSize                      3325
    VirtualSize                             108 Mb
    PeakVirtualSize                         128 Mb
    PageFaultCount                          3670
    MemoryPriority                           BACKGROUND
    BasePriority                             8
    CommitCharge                             568
    Job                                     fffff9e06286539a0
```

10. Notice there is a non-NULL job object. Show its properties with the **!job** command. The tool creates a job (CreateJobObject), adds the process to the job (AssignProcessToJobObject), and calls SetInformationJobObject with the CPU rate information class and rate value of 2000 (20 percent).

[Click here to view code image](#)

```
0: kd> !job ffff9e06286539a0
```

```
Job at ffff9e06286539a0
```

```
Basic Accounting Information
```

```
TotalUserTime:          0x0
TotalKernelTime:         0x0
TotalCycleTime:          0x0
ThisPeriodTotalUserTime: 0x0
ThisPeriodTotalKernelTime: 0x0
TotalPageFaultCount:     0x0
TotalProcesses:          0x1
ActiveProcesses:         0x1
FreezeCount:              0
BackgroundCount:         0
TotalTerminatedProcesses: 0x0
PeakJobMemoryUsed:       0x248
PeakProcessMemoryUsed:   0x248
```

```
Job Flags
```

```
[close done]
[cpu rate control]
```

```
Limit Information (LimitFlags: 0x0)
```

```
Limit Information (EffectiveLimitFlags: 0x800)
```

```
CPU Rate Control
```

```
Rate = 20.00%
```

```
Hard Resource Cap
```

```
Scheduling Group: ffff9e0628d7c1c0
```

11. Rerun the CPULIMIT tool on the same process and again set the CPU rate to 20 percent. You should see the CPU consumption of CPUSTRES drop down to around 4 percent. This is because of job nesting. A new job is created, as is the process assigned to it, nested under the first job. The net result is 20 percent of 20 percent, which is 4 percent.

Dynamic processor addition and replacement

As you've seen, developers can fine-tune which threads are allowed to (and in the case of the ideal processor, should) run on which processor. This works fine on systems that have a constant number of processors during their run time. For example, desktop machines require shutting down the computer to make any sort of hardware changes to the processor or their count. Today's server systems, however, cannot afford the downtime that CPU replacement or addition normally requires. In fact, you may be required to add a CPU at times of high load that is above what the machine can support at its current level of performance. Having to shut down the server during a period of peak usage would defeat the purpose.

To address this requirement, the latest generation of server motherboards and systems support the addition of processors (as well as their replacement) while the machine is still running. The ACPI BIOS and related hardware on the machine have been specifically built to allow and be aware of this need, but OS participation is required for full support.

Dynamic processor support is provided through the HAL, which notifies the kernel of a new processor on the system through the `KeStartDynamicProcessor` function. This routine does similar work to that performed when the system detects more than one processor at startup and needs to initialize the

structures related to them. When a dynamic processor is added, various system components perform some additional work. For example, the memory manager allocates new pages and memory structures optimized for the CPU. It also initializes a new DPC kernel stack while the kernel initializes the global descriptor table (GDT), the interrupt dispatch table (IDT), the processor control region (PCR), the process control block (PRCB), and other related structures for the processor.

Other executive parts of the kernel are also called, mostly to initialize the per-processor look-aside lists for the processor that was added. For example, the I/O manager, executive look-aside list code, cache manager, and object manager all use per-processor look-aside lists for their frequently allocated structures.

Finally, the kernel initializes threaded DPC support for the processor and adjusts exported kernel variables to report the new processor. Different memory-manager masks and process seeds based on processor counts are also updated, and processor features need to be updated for the new processor to match the rest of the system—for example, enabling virtualization support on the newly added processor. The initialization sequence completes with the notification to the Windows Hardware Error Architecture (WHEA) component that a new processor is online.

The HAL is also involved in this process. It is called once to start the dynamic processor after the kernel is aware of it, and called again after the kernel has finished initialization of the processor. However, these notifications and callbacks only make the kernel aware and respond to processor changes. Although an additional processor increases the throughput of the kernel, it does nothing to help drivers.

To handle drivers, the system has a default executive callback object, `ProcessorAdd`, with which drivers can register for notifications. Similar to the callbacks that notify drivers of power state or system time changes, this callback allows driver code to, for example, create a new worker thread if desirable so that it can handle more work at the same time.

Once drivers are notified, the final kernel component called is the Plug and Play manager, which adds the processor to the system's device node and rebalances interrupts so that the new processor can handle interrupts that were already registered for other processors. CPU-hungry applications are also able to take advantage of the new processors.

However, a sudden change of affinity can have potentially breaking changes for a running application—especially when going from a single-processor to a multiprocessor environment—through the appearance of potential race conditions or simply misdistribution of work (because the process might have calculated the perfect ratios at startup, based on the number of CPUs it was aware of). As a result, applications do not take advantage of a dynamically added processor by default. They must request it.

The `SetProcessAffinityUpdateMode` and `QueryProcessAffinityUpdateMode` Windows APIs, which use the undocumented `NtSet/QueryInformationProcess` system call) tell the process manager that these applications should have their affinity updated (by setting the `AffinityUpdateEnable` flag in `EPROCESS`) or that they do not want to deal with affinity updates (by setting the `AffinityPermanent` flag in `EPROCESS`). This is a one-time change. After an application has told the system that its affinity is permanent, it cannot later change its mind and request affinity updates.

As part of `KeStartDynamicProcessor`, a new step has been added after interrupts are rebalanced: calling the process manager to perform affinity updates through `PsUpdateActiveProcessAffinity`. Some Windows core processes and services already have affinity updates enabled, while third-party software will need to be recompiled to take advantage of the new API call. The System process, Svchost processes, and Smss are all compatible with dynamic processor addition.

Worker factories (thread pools)

Worker factories are the internal mechanism used to implement user-mode thread pools. The legacy thread-pool routines were completely implemented in user mode inside the Ntdll.dll library. In addition, the Windows API provided several functions for developers to call, which provided waitable timers (CreateTimerQueue, CreateTimerQueueTimer, and friends), wait callbacks (RegisterWaitForSingleObject), and work item processing with automatic thread creation and deletion (QueueUserWorkItem), depending on the amount of work being done.

One issue with the old implementation was that only one thread pool could be created in a process, which made some scenarios difficult to implement. For example, trying to prioritize work items by building two thread pools which would serve a different set of requests was not directly possible. The other issue was the implementation itself, which was in user mode (in Ntdll.dll). Because the kernel can have direct control over thread scheduling, creation, and termination without the typical costs associated with doing these operations from user mode, most of the functionality required to support the user-mode thread pool implementation in Windows is now located in the kernel. This also simplifies the code that developers need to write. For example, creating a worker pool in a remote process can be done with a single API call instead of the complex series of virtual memory calls this normally requires. Under this model, Ntdll.dll merely provides the interfaces and high-level APIs required for interfacing with the worker factory kernel code.

This kernel thread pool functionality in Windows is managed by an object manager type called TpWorkerFactory, as well as four native system calls for managing the factory and its workers (NtCreateWorkerFactory, NtWorkerFactoryWorkerReady, NtReleaseWorkerFactoryWorker, and NtShutdownWorkerFactory); two query/set native calls (NtQueryInformationWorkerFactory and NtSetInformationWorkerFactory); and a wait call (NtWaitForWorkViaWorkerFactory). Just like other native system calls, these calls provide user mode with a handle to the TpWorkerFactory object, which contains information such as the name and object attributes, the desired access mask, and a security descriptor. Unlike other system calls wrapped by the Windows API, however, thread-pool management is handled by Ntdll.dll's native code. This means developers work with opaque descriptors: a TP_POOL pointer for a thread pool and other opaque pointers for object created from a pool, including TP_WORK (work callback), TP_TIMER (timer callback), TP_WAIT (wait callbacks), etc. These structures hold various pieces of information, such as the handle to the TpWorkerFactory object.

As its name suggests, the worker factory implementation is responsible for allocating worker threads (and calling the given user-mode worker thread entry point) and maintaining a minimum and maximum thread count (allowing for either permanent worker pools or totally dynamic pools) as well as other accounting information. This enables operations such as shutting down the thread pool to be performed with a single call to the kernel because the kernel has been the only component responsible for thread creation and termination.

Because the kernel dynamically creates new threads as needed (based on minimum and maximum numbers provided), this increases the scalability of applications using the new thread-pool implementation. A worker factory will create a new thread whenever all of the following conditions are met:

- Dynamic thread creation is enabled.
- The number of available workers is lower than the maximum number of workers configured for the factory (default of 500).

- The worker factory has bound objects (for example, an ALPC port that this worker thread is waiting on) or a thread has been activated into the pool.
- There are pending I/O request packets (IRPs; see [Chapter 6](#) for more information) associated with a worker thread.

In addition, it will terminate threads whenever they’ve become idle—that is, they haven’t processed any work item—for more than 10 seconds (by default). Furthermore, although developers have always been able to take advantage of as many threads as possible (based on the number of processors on the system) through the old implementation, it’s now possible for applications using thread pools to automatically take advantage of new processors added at run time. This is through its support for dynamic processors in Windows Server (as discussed earlier in this chapter).

Worker factory creation

The worker factory support is merely a wrapper to manage mundane tasks that would otherwise have to be performed in user mode (at a loss of performance). Much of the logic of the new thread-pool code remains in the Ntdll.dll side of this architecture. (Theoretically, by using undocumented functions, a different thread-pool implementation can be built around worker factories.) Also, it is not the worker factory code that provides the scalability, wait internals, and efficiency of work processing. Instead, it is a much older component of Windows: I/O completion ports or, more correctly, kernel queues (KQUEUE). In fact, when creating a worker factory, an I/O completion port must have already been created by user mode, and the handle needs to be passed in.

It is through this I/O completion port that the user-mode implementation will queue and wait for work—but by calling the worker factory system calls instead of the I/O completion port APIs. Internally, however, the “release” worker factory call (which queues work) is a wrapper around `IoSetIoCompletionEx`, which increases pending work, while the “wait” call is a wrapper around `IoRemoveIoCompletion`. Both these routines call into the kernel queue implementation. Therefore, the job of the worker factory code is to manage either a persistent, static, or dynamic thread pool; wrap the I/O completion port model into interfaces that try to prevent stalled worker queues by automatically creating dynamic threads; and simplify global cleanup and termination operations during a factory shutdown request (as well as easily block new requests against the factory in such a scenario).

The executive function that creates the worker factory, `NtCreateWorkerFactory`, accepts several arguments that allow customization of the thread pool, such as the maximum threads to create and the initial committed and reserved stack sizes. The `CreateThreadpool` Windows API, however, uses the default stack sizes embedded in the executable image (just like a default `CreateThread` would). The Windows API does not, however, provide a way to override these defaults. This is somewhat unfortunate, as in many cases thread-pool threads don’t require deep call stacks, and it would be beneficial to allocate smaller stacks.

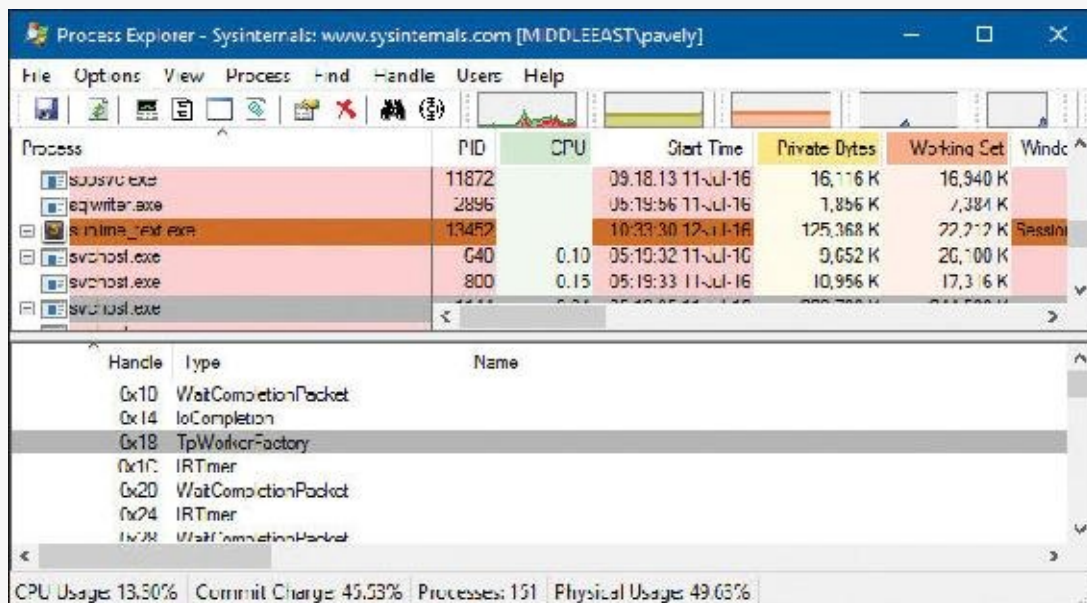
The data structures used by the worker factory implementation are not in the public symbols, but it is still possible to look at some worker pools, as you’ll see in the next experiment. Additionally, the `NtQueryInformationWorkerFactory` API dumps almost every field in the worker factory structure.

EXPERIMENT: Looking at thread pools

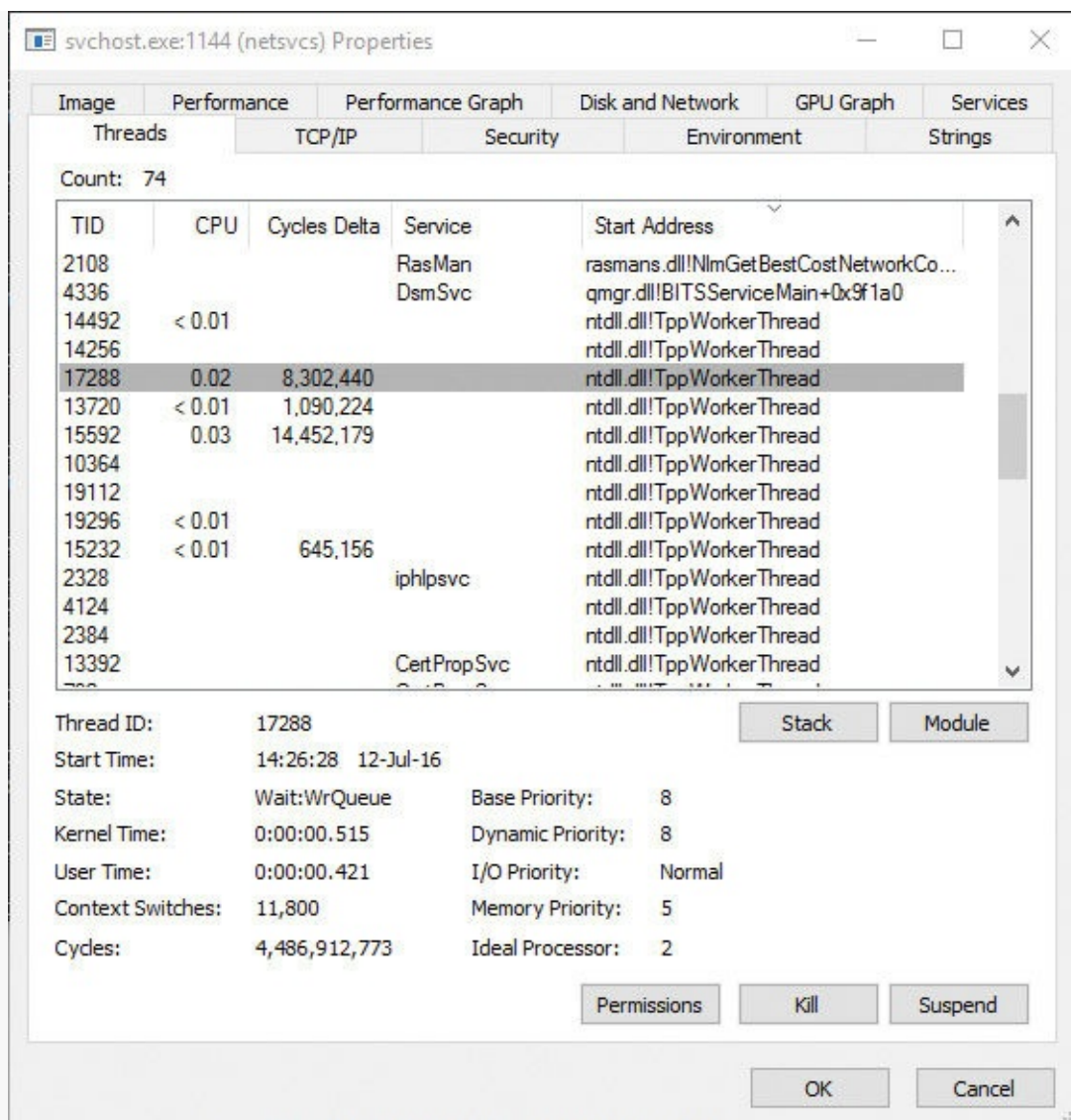
Because of the advantages of the thread-pool mechanism, many core system components and applications use it, especially when dealing with resources such as ALPC ports (to dynamically process incoming requests at an appropriate and scalable level). One of the ways to identify which

processes are using a worker factory is to look at the handle list in Process Explorer. Follow these steps to look at some details behind them:

1. Run Process Explorer.
2. Open the **View** menu and select **Show Unnamed Handles and Mappings**. (Unfortunately, worker factories aren't named by Ntdll.dll, so you need to take this step to see the handles.)
3. Select an instance of svchost.exe from the list of processes.
4. Open the **View** menu and choose **Show Lower Pane** to display the lower pane of the handle table.
5. Open the **View** menu, choose **Lower Pane View**, and select **Handles** to display the table in handle mode.
6. Right-click the lower pane column headers and choose **Select Columns**.
7. Make sure the **Type** and **Handle Value** columns are checked.
8. Click the **Type** header to sort by type.
9. Scroll down the handles, looking at the Type column, until you find a handle of type TpWorkerFactory.
10. Click the **Handle** header to sort by handle value. You should see something similar to the following screenshot. Notice how the TpWorkerFactory handle is immediately preceded by an IoCompletion handle. As discussed, this occurs because a handle to an I/O completion port on which work will be sent must be created before creating a worker factory.



11. Double-click the selected process in the list of processes, click the **Threads** tab, and click the **Start Address** column. You should see something similar to the following screenshot. Worker factory threads are easily identified by their Ntdll.dll's entry point, TppWorkerThread. (*Tpp* stands for *thread pool private*.)



If you look at other worker threads, you'll see some are waiting for objects such as events. A process can have multiple thread pools, and each thread pool can have a variety of threads doing completely unrelated tasks. It's up to the developer to assign work and to call the thread-pool APIs to register this work through Ntdll.dll.

Conclusion

This chapter examined the structure of threads, how they are created and managed, and how Windows decides which threads should run, for how long, and on which processor or processors. In the next chapter, you'll look at one of the most important aspects of any OS: memory management.